



TEXAS TECH UNIVERSITY
Department of Computer Science

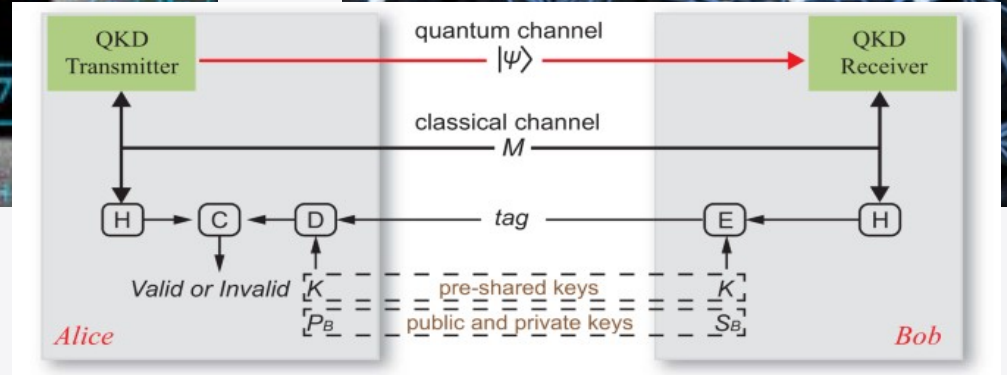
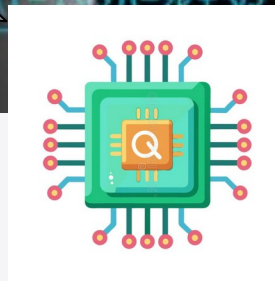
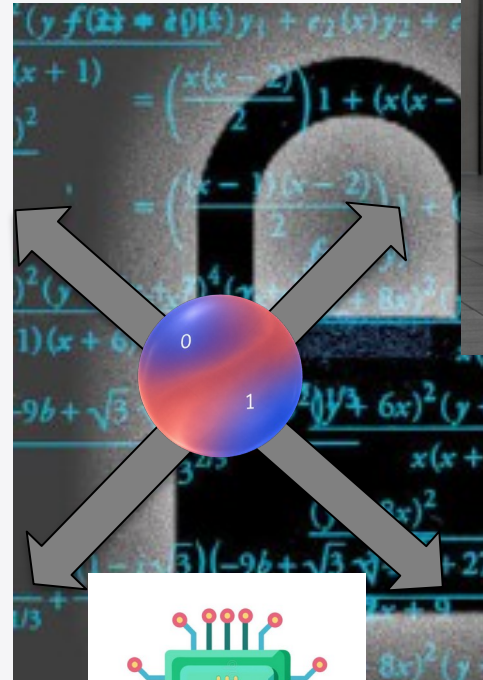
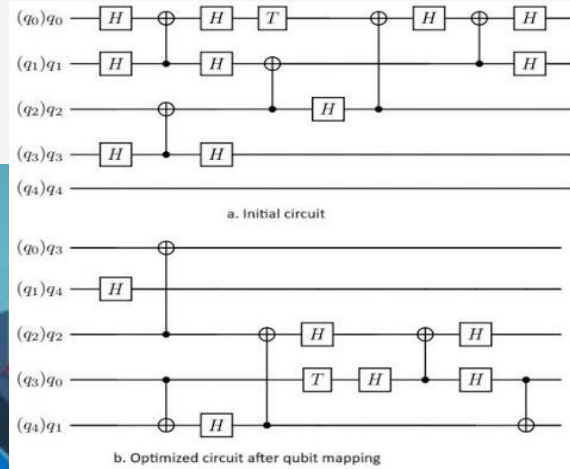
Quantum Computing and Circuit Simulation

Ziqing Guo

Thesis Committee: Dr. Ziwen Pan (Research Advisor)
Professor Zihao Zhan · Dr. Susan Mengel · Dr. Lu Wei

Dean's representative: Dr. Anne E. V. Gorden

Computing in Quantum

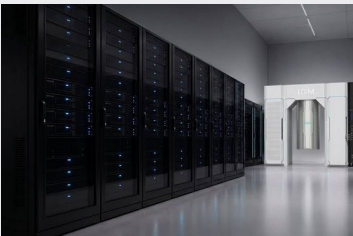


Applications & Research



01 · RQ1 · Scale

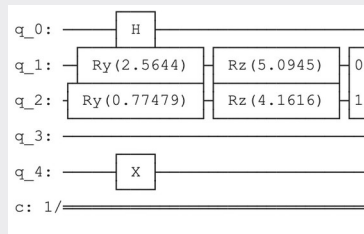
Simulation



GPU / QPU
simulation beyond
hardware reach

02 · RQ2 · QIP

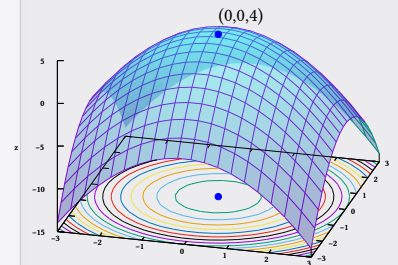
Algorithms



Structure from
problem and device,
not black-box search

03 · RQ4 · Synthesis

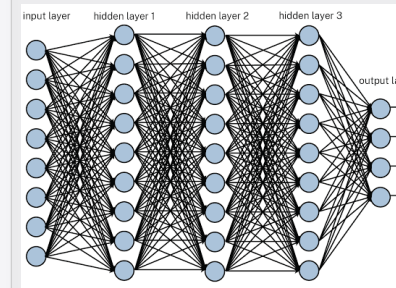
Optimization



AI/HPC synthesis
within fault-tolerant
budgets

04 · RQ3 · LLM

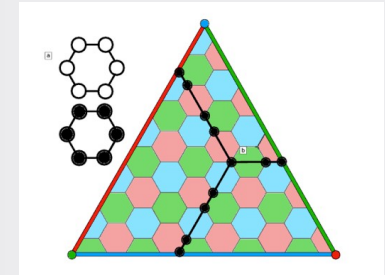
LLMs



AI for quantum

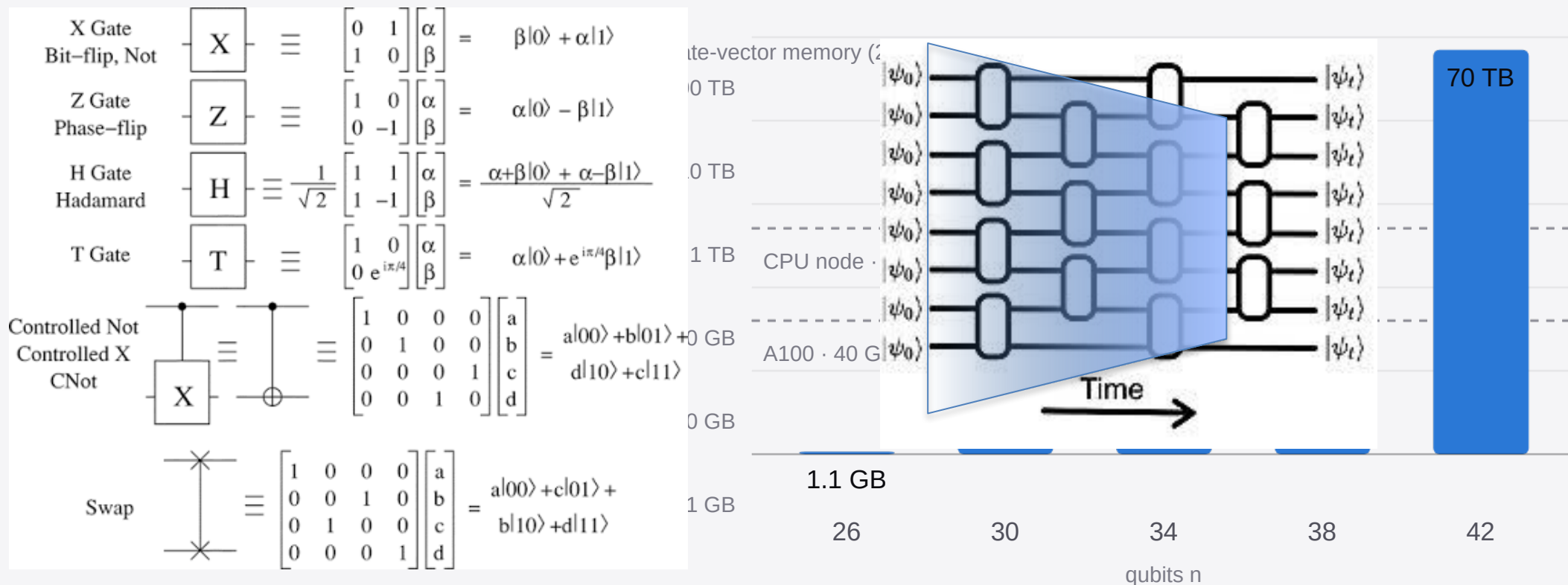
05 · RQ5 · Trust

QEC



Auditable QEC and
post-quantum
cryptography

Background: classical QC simulation



Nielsen, Michael A., and Isaac L. Chuang. *Quantum computation and quantum information*. Vol. 1. Cambridge: Cambridge university press, 2000.



Circuit simulators

Qirsp / qiskit / cudaq

Exact

Statevector / Density Matrices

Itensors, qiskit

Approx~

Tensor Network (MPS)

Stim / tsim

QEC

Stabilizer Pauli Frame Tracking /
extended T-gate Stabilizer
(quantum error correction)

$$\rho_{\text{mixture}} = \frac{1}{2}|00\rangle\langle 00| + \frac{1}{2}|11\rangle\langle 11|$$

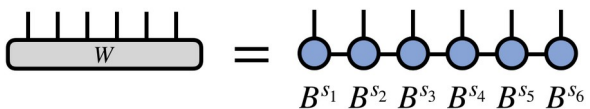
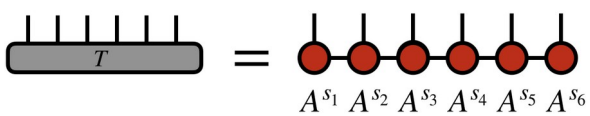
$$|\psi\rangle = a_{00}|00\rangle + a_{01}|01\rangle + a_{10}|10\rangle + a_{11}|11\rangle$$

$$\mathbf{v} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Ziqing Guo

$$\rho_{\text{Bell}} = \frac{1}{2} \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}$$

$$a_{i_1 \dots i_n} = A_1^{i_1} A_2^{i_2} \dots A_n^{i_n}$$



$$a_{00} = A_1^0 A_2^0 = \frac{1}{\sqrt{2}}, \quad a_{01} = A_1^0 A_2^1 = 0$$

TTU CS Department

$$P = \bigotimes_{j=1}^n X_j^{x_j} Z_j^{z_j}$$

$$X|0\rangle = |1\rangle, \quad Z|0\rangle = |0\rangle,$$

$$a = \frac{1 + e^{i\pi/4}}{2}, \quad b = \frac{1 - e^{i\pi/4}}{2}$$

$$T|+\rangle = \frac{|0\rangle + e^{i\pi/4}|1\rangle}{\sqrt{2}} = a|+\rangle + b|-\rangle,$$

Circuit simulators

Circuit Simulators Literature

DiaQ: Efficient State-Vector Quantum Simulation

Research article • Full

License: CC BY 4.0

The density-

Ulrich Schollwöck

¹Institu



We p
quantu
dynam
using c
show t
to class
a lower

Article

Sp

Abstract

The density-matrix renormalization group method (DMRG) has established itself over the last decade as the leading method for the simulation of the statics and dynamics of one-dimensional strongly correlated quantum lattice systems. In the further development of the method, the realization that DMRG operates on a highly interesting class of quantum states, so-called matrix product states (MPS), has allowed a much deeper understanding of the inner structure of the DMRG method, its further potential and its limitations. In this paper, I want to give a detailed

Q-GEAR: Improving quantum simulation framework

Ziqing Guo

NERSC, Lawrence Berkeley National
Laboratory, Berkeley, CA Texas Tech
University, TX, USA
ziqguo@ttu.edu

Jan Balewski

National Energy Research Scientific
Computing Center, Lawrence Berkeley
National Laboratory, CA, USA
balewski@lbl.gov

Ziwen Pan

Texas Tech University, TX, USA
ziwen.pan@ttu.edu

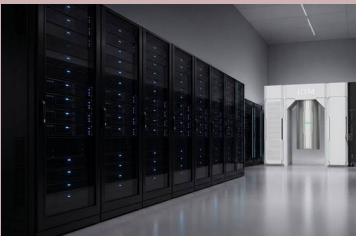
1. Introduction
2. Density-matr
3. DMRG and er
4. Matrix produ

Applications & Research



01 · RQ1 · Scale

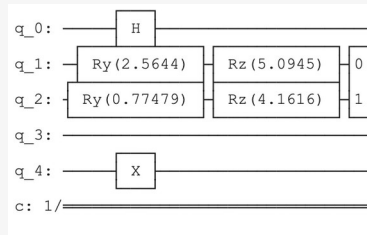
Simulation



GPU / QPU
simulation beyond
hardware reach

02 · RQ2 · QIP

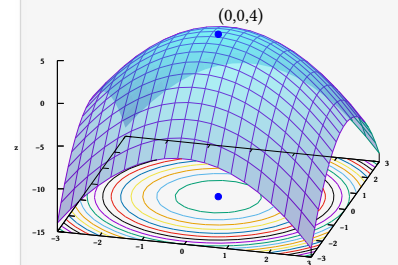
Algorithms



Structure from
problem and device,
not black-box search

03 · RQ4 · Synthesis

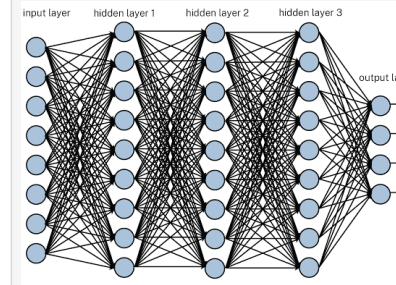
Optimization



AI/HPC synthesis
within fault-tolerant
budgets

04 · RQ3 · LLM

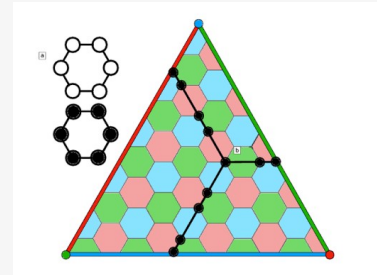
LLMs



AI for quantum

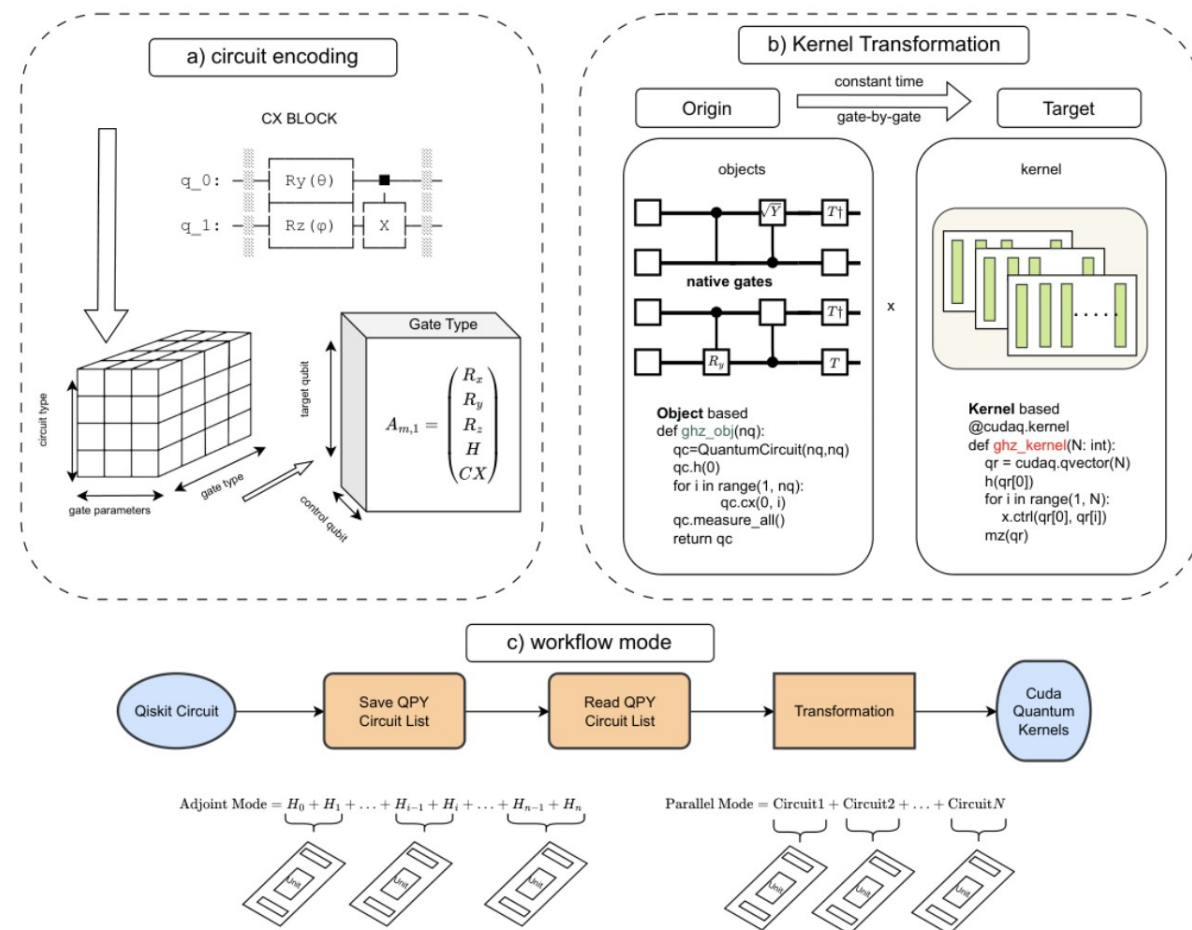
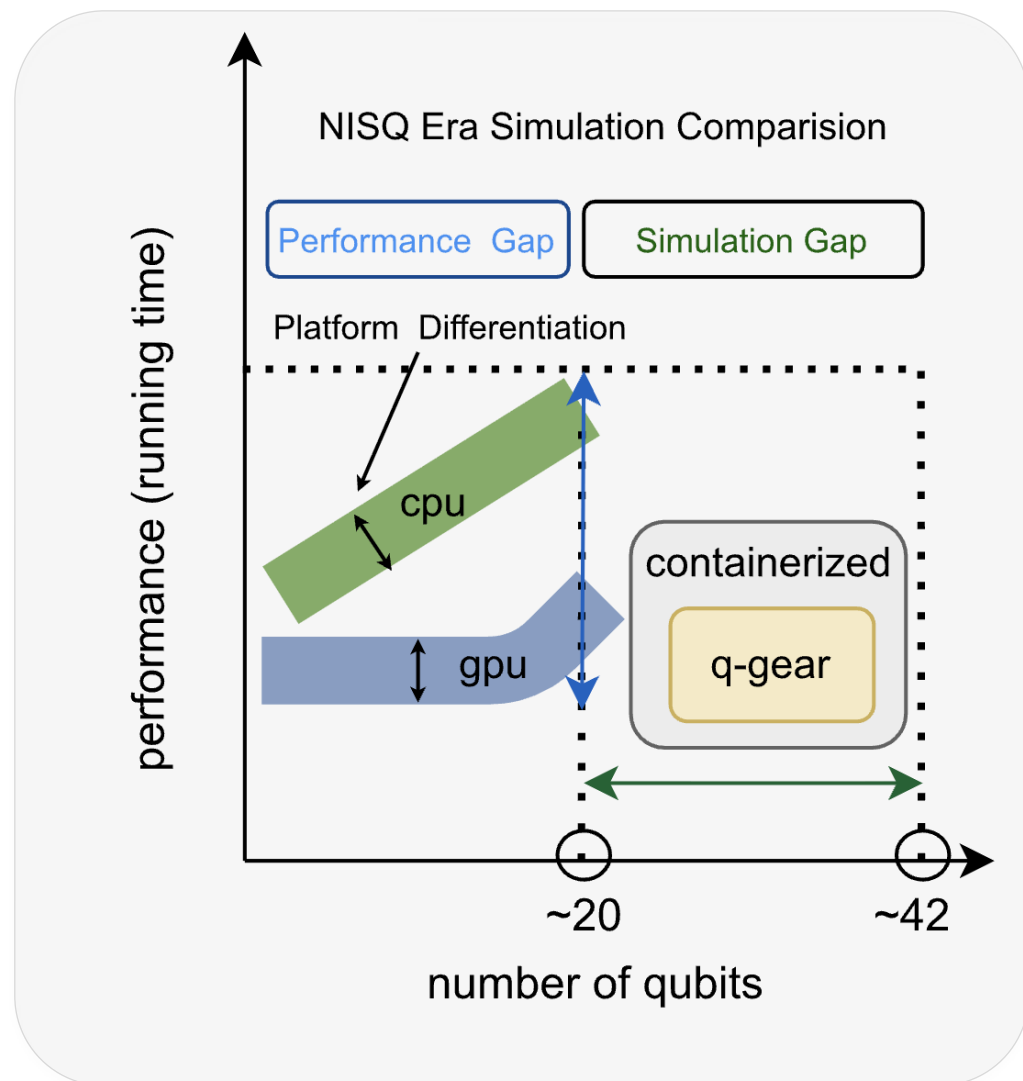
05 · RQ5 · Trust

QEC

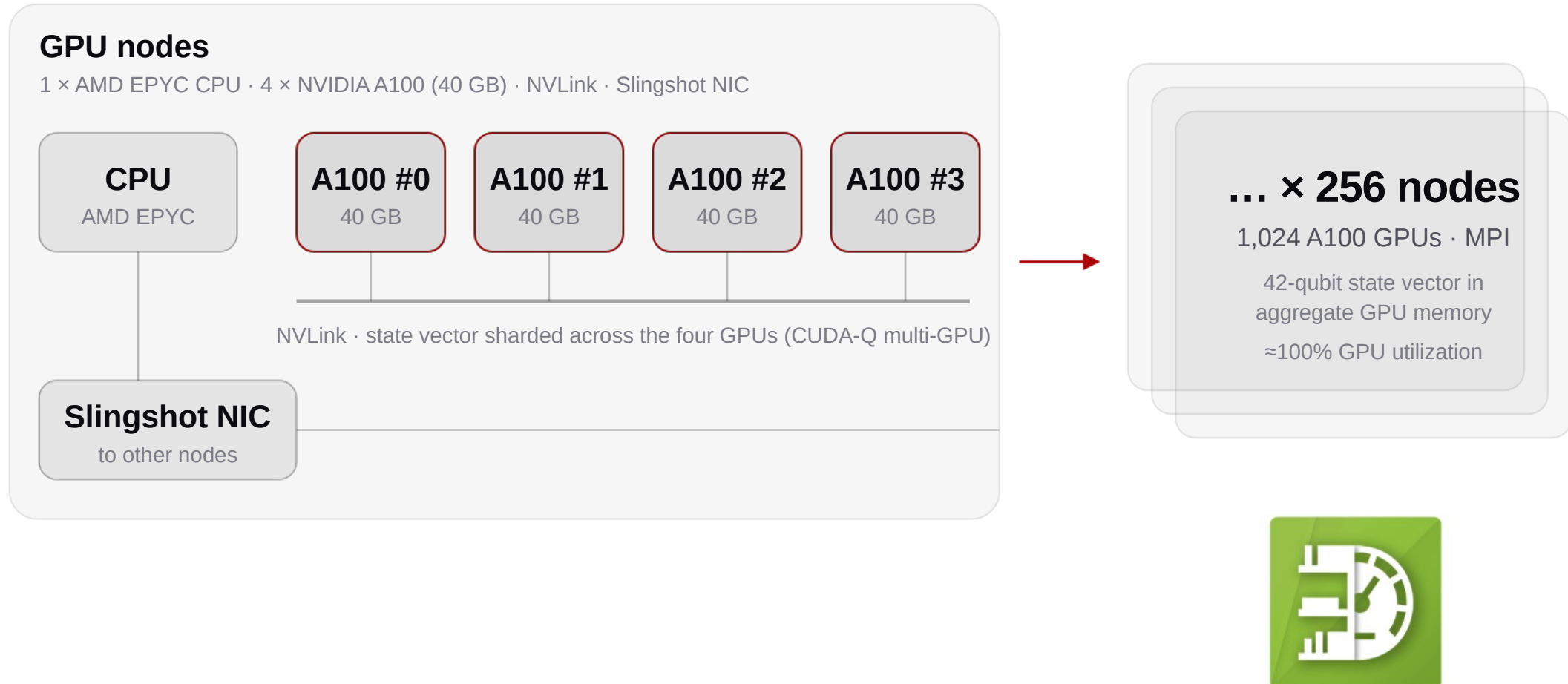


Auditable QEC and
post-quantum
cryptography

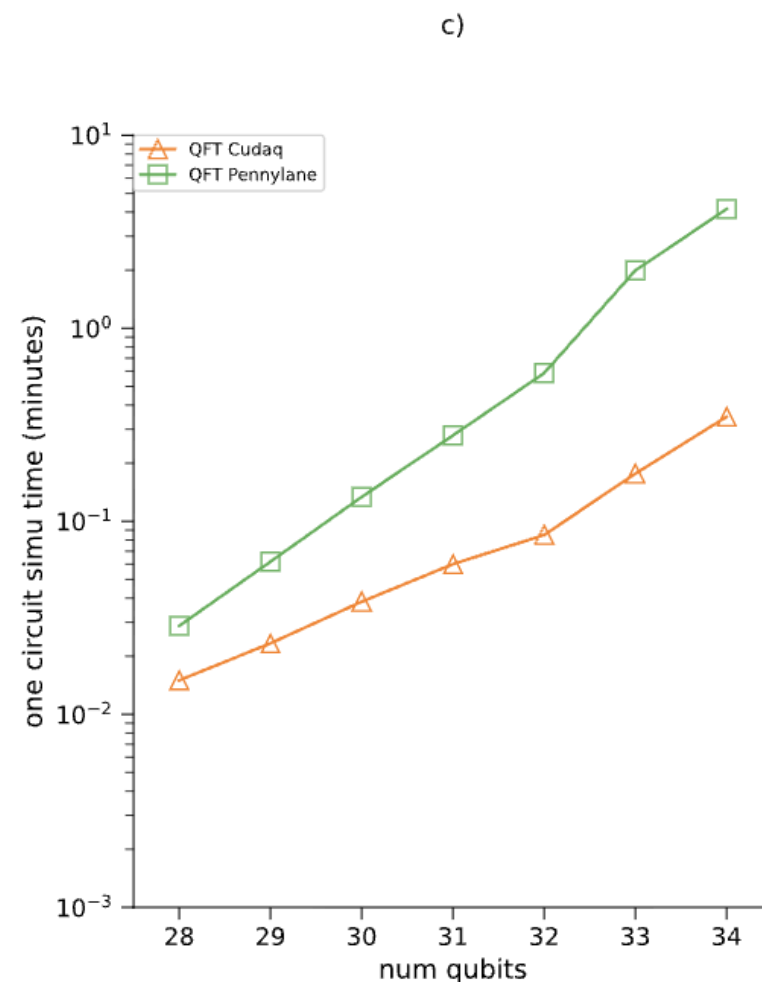
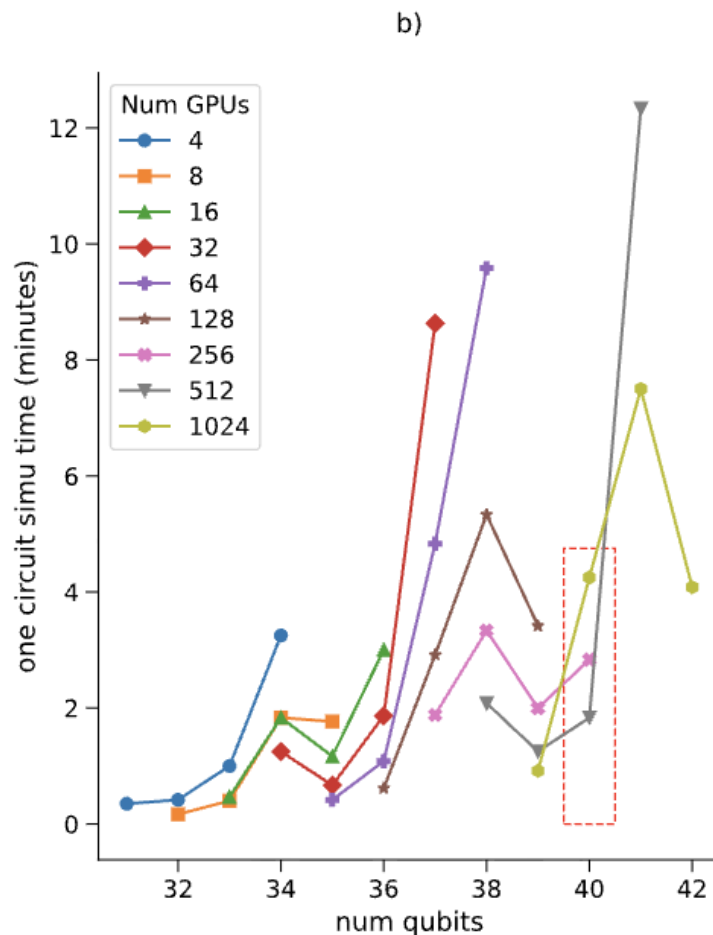
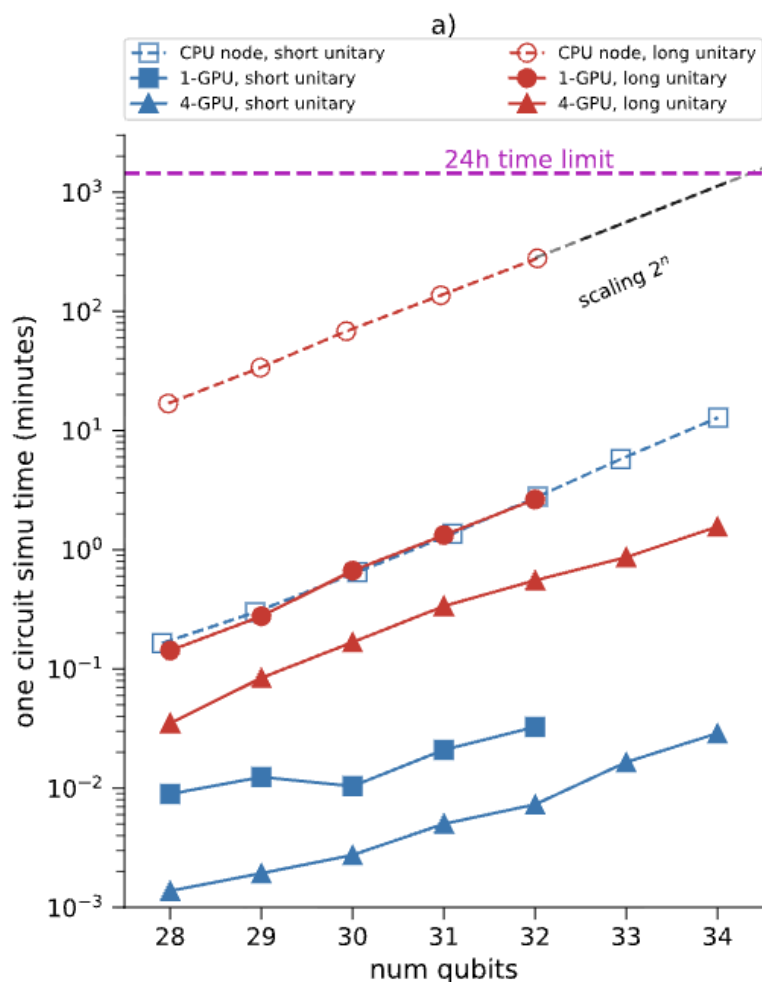
Simulation



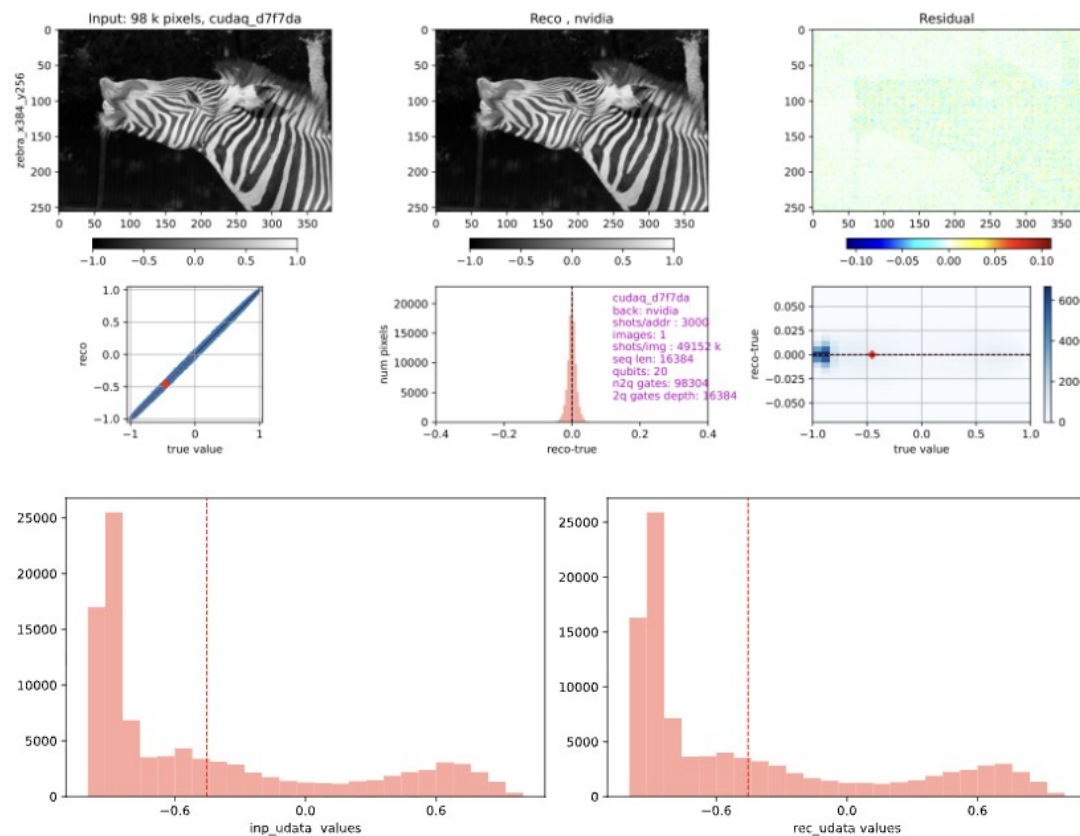
Simulation



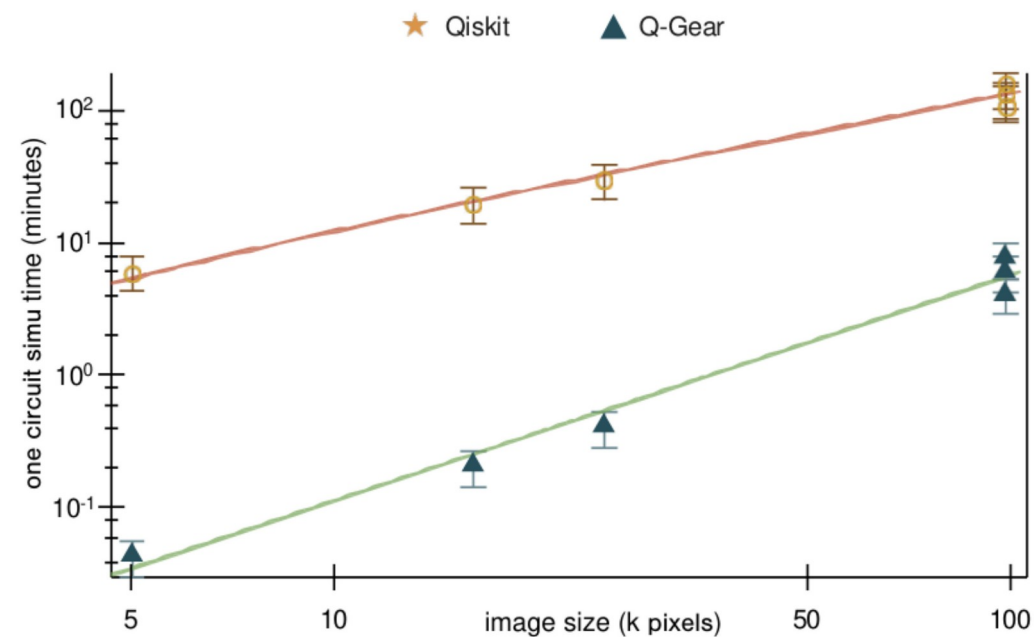
Simulation & Results



Simulation



Main take away



podman

Nvidia
collaboration
redesign CudaQ
framework

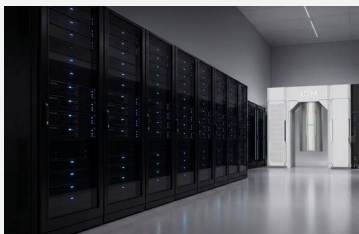
Mathematical
Tensors store gate-by-gate
kernel transformation

Applications & Research



01 · RQ1 · Scale

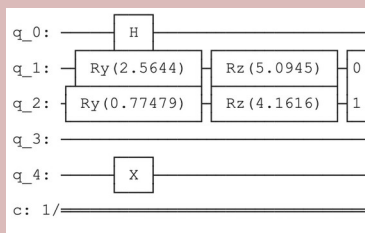
Simulation



GPU / QPU
simulation beyond
hardware reach

02 · RQ2 · QIP

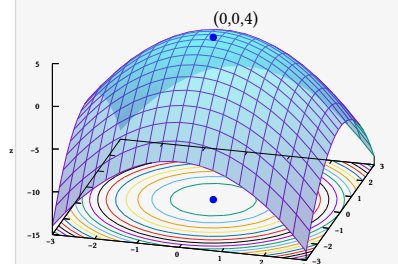
Algorithms



Structure from
problem and device,
not black-box search

03 · RQ4 · Synthesis

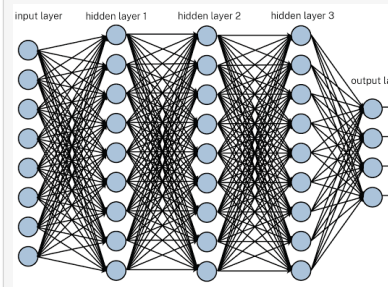
Optimization



AI/HPC synthesis
within fault-tolerant
budgets

04 · RQ3 · LLM

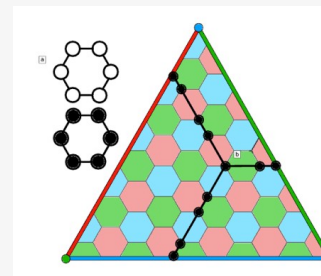
LLMs



AI for quantum

05 · RQ5 · Trust

QEC



Auditable QEC and
post-quantum
cryptography

Algorithms

Home / Quantum Information Processing / Article

A survey

Published: 11 Decem
Volume 15, pages 1-

Article | [Open access](#) | Published: 11 May 2022

Quantum pixel representations and compression for N-dimensional

Article | [Open access](#) | Published: 10 February 2024

Quantum-parallel vectorized data encodings and computations on trapped-ion and transmon QPUs



66	204	64	194	79	190	84	112
116	185	65	226	65	204	75	211
205	174	60	196	147	63	111	201
211	124	44	148	115	61	207	122

ascii

θ_0 00	θ_1 01
θ_2 10	θ_3 11

$$|I\rangle = \frac{1}{2} [(\cos \theta_0 |0\rangle + \sin \theta_0 |1\rangle) \otimes |00\rangle + (\cos \theta_1 |0\rangle + \sin \theta_1 |1\rangle) \otimes |01\rangle + (\cos \theta_2 |0\rangle + \sin \theta_2 |1\rangle) \otimes |10\rangle + (\cos \theta_3 |0\rangle + \sin \theta_3 |1\rangle) \otimes |11\rangle]$$

Algorithms

$$|I\rangle = \frac{1}{2^n} \sum_{i=0}^{2^{2n}-1} |c_i\rangle \otimes |i\rangle$$

$$|c_i\rangle = \cos \theta_i |0\rangle + \sin \theta_i |1\rangle,$$

$$\mathcal{H}(|0\rangle^{\otimes 2n+1}) = \frac{1}{2^n} \otimes \sum_{i=0}^{2^{2n}-1} |i\rangle = |H\rangle$$

$$R_y(2\theta_i) = \begin{pmatrix} \cos \theta_i & -\sin \theta_i \\ \sin \theta_i & \cos \theta_i \end{pmatrix}$$

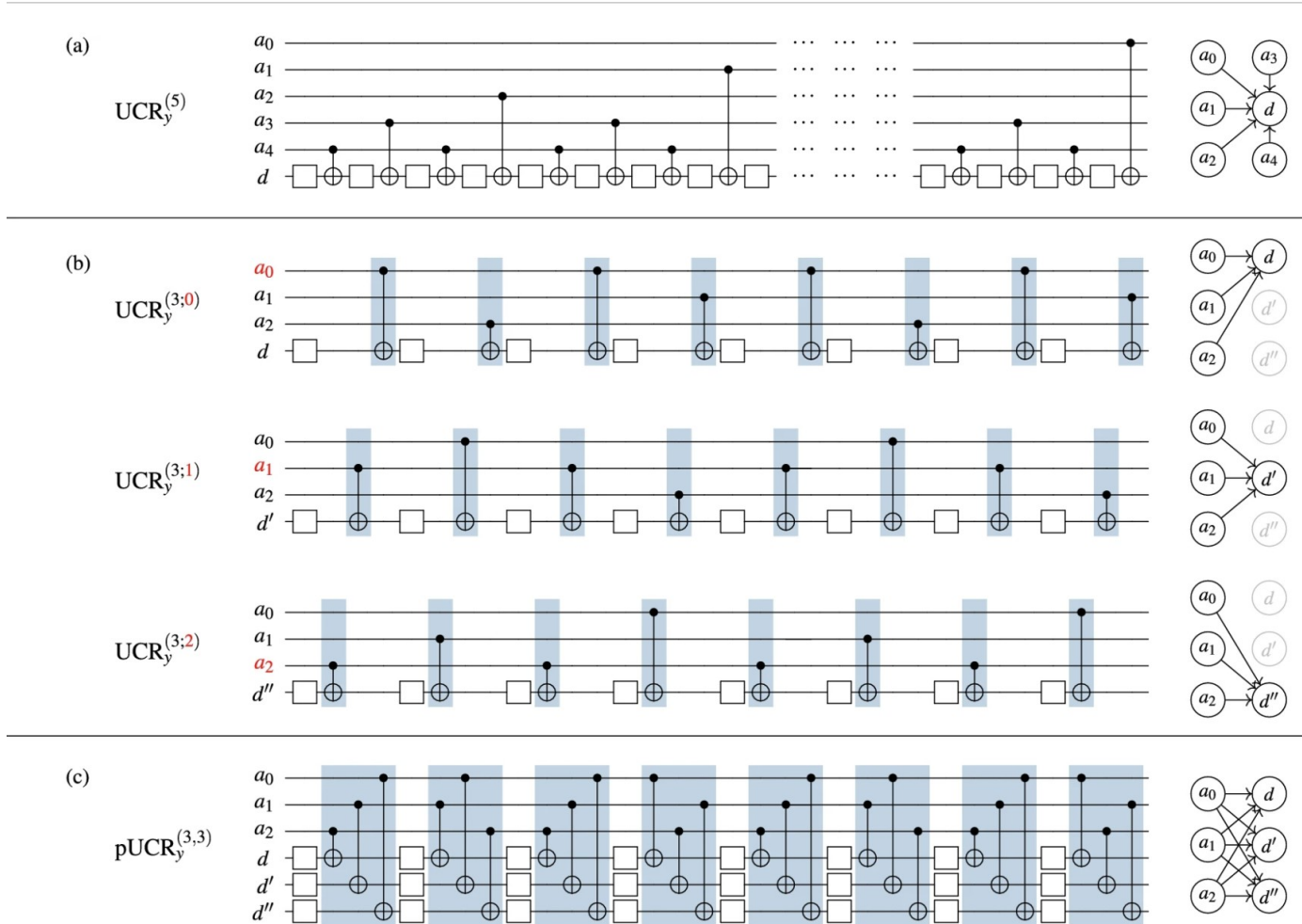
$$R_i = \left(I \otimes \sum_{j=0, j \neq 1}^{2^{2n}-1} |j\rangle \langle j| \right) + R_y(2\theta_i) \otimes |i\rangle \langle i|$$

$$R_y(\phi) := e^{-iY\phi/2} = \begin{bmatrix} \cos \frac{\phi}{2} & -\sin \frac{\phi}{2} \\ \sin \frac{\phi}{2} & \cos \frac{\phi}{2} \end{bmatrix}$$

$$\begin{aligned} |C_{\text{RGB}\alpha}^i\rangle = & \cos \theta_{\text{R}}^i |000\rangle + \cos \theta_{\text{G}}^i |001\rangle + \cos \theta_{\text{B}}^i |010\rangle + \cos \theta_{\alpha} |011\rangle \\ & + \sin \theta_{\text{R}}^i |100\rangle + \sin \theta_{\text{G}}^i |101\rangle + \sin \theta_{\text{B}}^i |110\rangle + \sin \theta_{\alpha} |111\rangle \end{aligned} \quad \text{UCR}_y(\vec{\alpha}) = \begin{bmatrix} R_y(\alpha_0) & & \\ & \ddots & \\ & & R_y(\alpha_{2^{n_a}-1}) \end{bmatrix}$$

Algorithms

$$\begin{aligned} |\psi_{\text{FRQI}}(\vec{\alpha})\rangle &= \text{UCR}_y(\vec{\alpha}) (H^{\otimes n_a} \otimes I) |0\rangle^{\otimes (n_a+1)}, \\ &= \sum_i |i\rangle \otimes \left(\cos \frac{\alpha_i}{2} |0\rangle + \sin \frac{\alpha_i}{2} |1\rangle \right). \end{aligned}$$

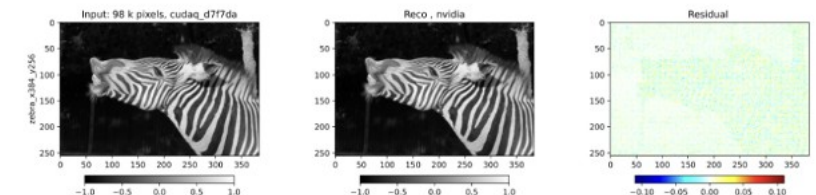
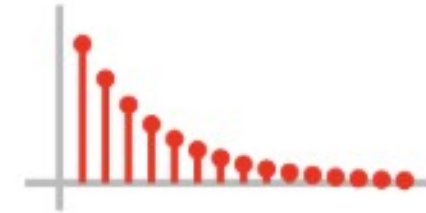


Decoding

$$[c_0, s_0, \dots, c_{2^{n_a}-1}, s_{2^{n_a}-1}]^2$$

$$\alpha_i^{\text{meas}} = 2 \arctan \sqrt{\frac{|s_i|^2}{|c_i|^2}}, \quad i \in [2^{n_a}],$$

Probability density function (PDF)



Algorithms

Quantum computing is linear how do we enable **universal** quantum capability for classical computation?

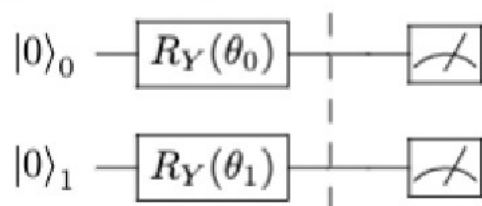


Quantum arithmetic

$$x_i = \cos \theta_i = \langle Z \rangle_i.$$

X+Y?
X*Y

input encoding



Arccos(x)

{Cos[θ0], Cos[θ1]}

$$\left\{ \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}, \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}, \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \right\}$$

(a) EVEN encoding and decoding

$$|0\rangle \xrightarrow{R_y(\theta)} |\phi\rangle \longrightarrow |\phi\rangle \xrightarrow{Z} \hat{y} = \text{EV}(\sigma_z) \simeq x$$

(b) product-with-memory (multiplication)

$$\begin{array}{l} |0\rangle_0 \xrightarrow{R_y(\theta_0)} \\ |0\rangle_1 \xrightarrow{R_y(\theta_1)} \end{array} \xrightarrow{\begin{array}{c} \text{CNOT} \\ R_z(\frac{\pi}{2}) \\ \text{CNOT} \end{array}} \xrightarrow{\oplus} \xrightarrow{Z} \hat{y}_1 = \text{EV}(I \otimes \sigma_z) \simeq x_0 x_1$$

(c) weighted sum (addition) $\hat{y}_0 = \text{EV}(\sigma_z \otimes I) \simeq w x_0 + (1 - w) x_1$

$$\begin{array}{l} |0\rangle_0 \xrightarrow{R_y(\theta_0)} \\ |0\rangle_1 \xrightarrow{R_y(\theta_1)} \end{array} \xrightarrow{\begin{array}{c} \text{CNOT} \\ R_z(\frac{\pi}{2}) \\ \text{CNOT} \end{array}} \xrightarrow{\begin{array}{c} R_y(\frac{\alpha}{2}) \\ \oplus \\ R_y(-\frac{\alpha}{2}) \end{array}} \xrightarrow{Z} \hat{y}_0$$

(d) sign change (negation)

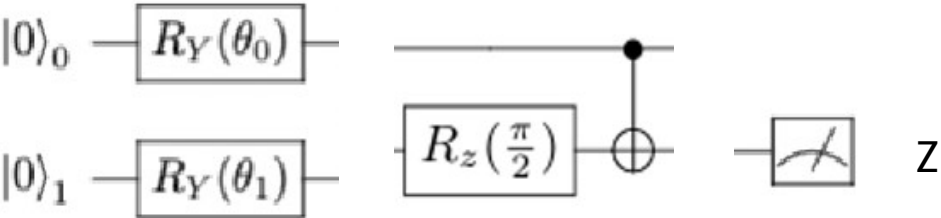
$$|0\rangle \xrightarrow{R_y(\theta)} \xrightarrow{X} \xrightarrow{Z} \hat{y} \simeq -x$$

(e) parity flip

$$\begin{array}{l} |0\rangle_{\text{anc}} \xrightarrow{H} \end{array} \xrightarrow{\begin{array}{c} \text{CNOT} \\ \text{CNOT} \end{array}}$$

Algorithms

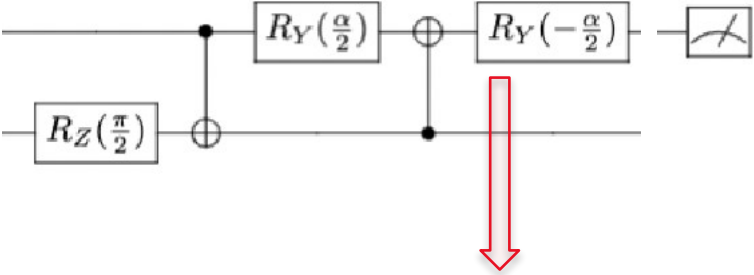
$$\arccos(x_1)) * \arccos(x_2)$$



$$\left\{ \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & i & 0 & 0 \\ 0 & 0 & 0 & i \\ 0 & 0 & 1 & 0 \end{pmatrix}, \begin{pmatrix} \cos\left[\frac{\theta_0}{2}\right] & \cos\left[\frac{\theta_1}{2}\right] \\ i \cos\left[\frac{\theta_0}{2}\right] & \sin\left[\frac{\theta_1}{2}\right] \\ i \sin\left[\frac{\theta_0}{2}\right] & \sin\left[\frac{\theta_1}{2}\right] \\ \cos\left[\frac{\theta_1}{2}\right] & \sin\left[\frac{\theta_0}{2}\right] \end{pmatrix} \right\}$$

$$x_0 \ x_1$$

$$w * \arccos(x_1)) * (1-w) * \arccos(x_2)$$



$$\text{Cos}[\alpha] \rightarrow 1 - 2w, \text{Sin}[\alpha] \rightarrow \text{Sqrt}[w(1-w)]$$

$$\langle Z \rangle_{\text{out}} = \underbrace{\frac{1 - \cos \alpha}{2}}_{\sin^2(\alpha/2)} x_1 + \underbrace{\frac{1 + \cos \alpha}{2}}_{\cos^2(\alpha/2)} x_2.$$

$$w = \frac{1 - \cos \alpha}{2}$$

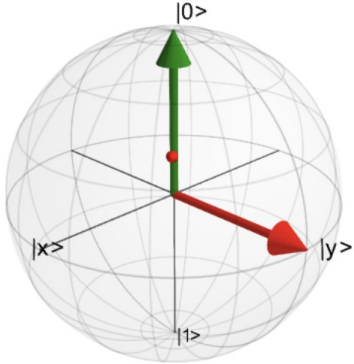
$$\begin{pmatrix} \frac{1}{2} & \sqrt{1+x_0} & \sqrt{1+x_1} \\ \frac{1}{2} & \sqrt{1+x_0} & \sqrt{1-x_1} \\ \frac{1}{2} & \sqrt{1-x_0} & \sqrt{1+x_1} \\ \frac{1}{2} & \sqrt{1-x_0} & \sqrt{1-x_1} \end{pmatrix}$$

$$\{w(x_0 - x_1) + x_1, x_0 x_1\}$$

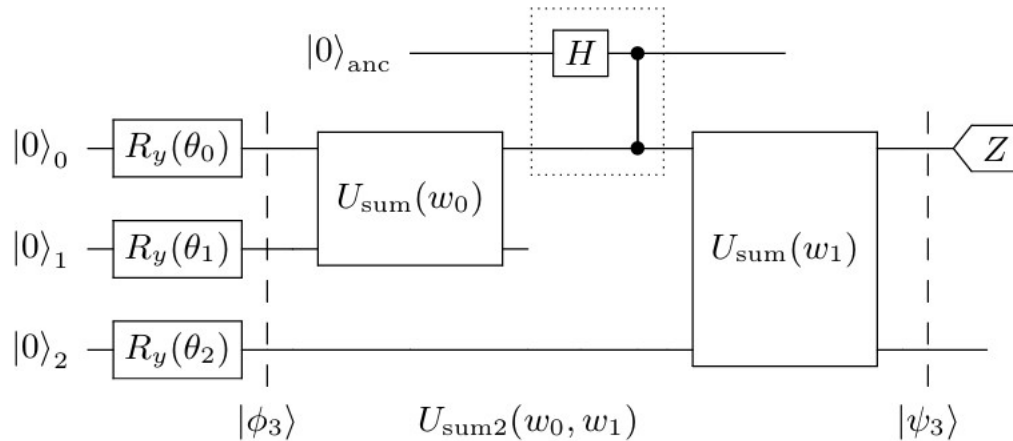
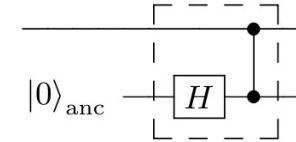
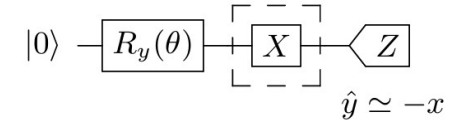
Weight	Gate angle	Encoded output
$w = 0$	$\alpha = 0$	x_2
$w = \frac{1}{2}$	$\alpha = \frac{\pi}{2}$	$(x_1 + x_2)/2$
$w = 1$	$\alpha = \pi$	x_1

Algorithms

Algorithms



Why Ry rotation?

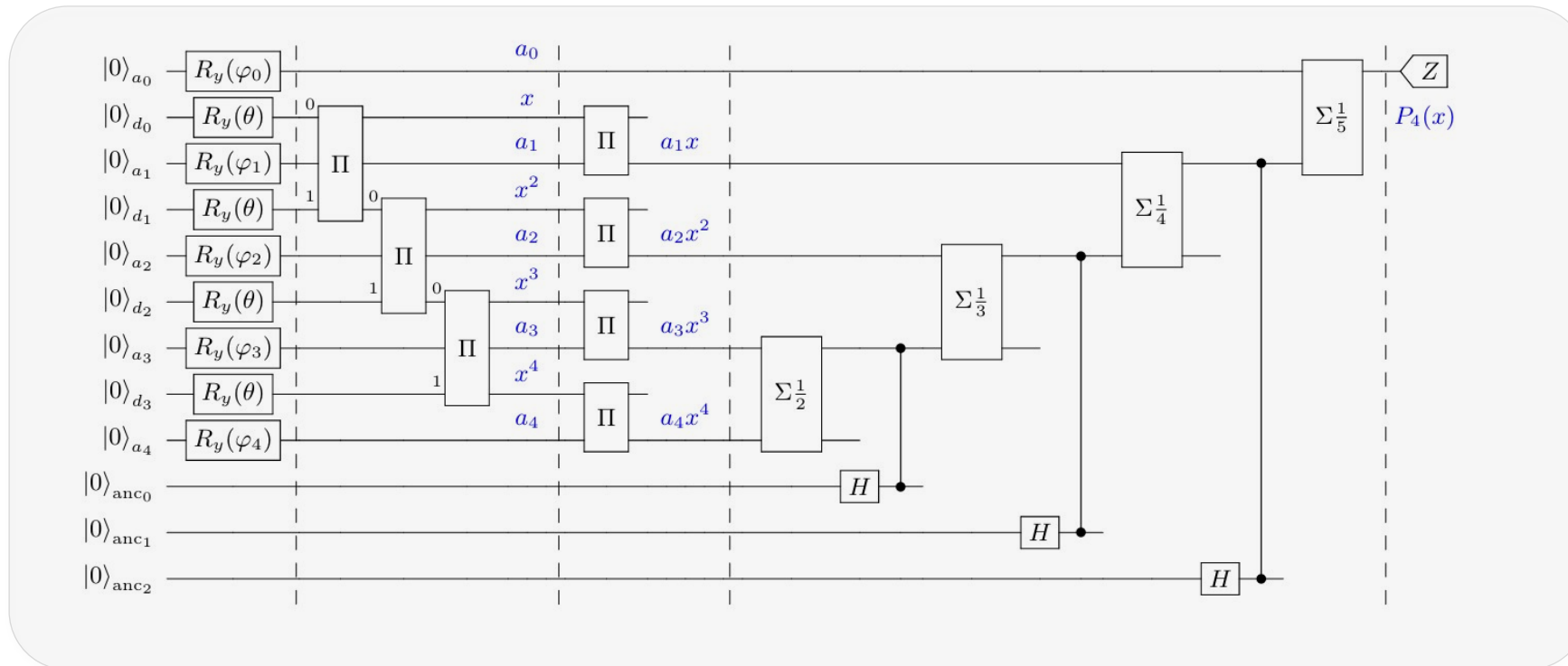


$$\langle \psi_3 | O_{s3} | \psi_3 \rangle = w_1 [w_0 x_0 + (1 - w_0) x_1] + (1 - w_1) x_2$$

$$w_1 [w_0 x_0 + (1 - w_0) x_1] + (1 - w_1) x_2 + \frac{1}{2} \sqrt{(-1 + w_1) w_1} \sqrt{1 - x_0^2} \sqrt{1 - x_1^2} \sqrt{1 - x_2^2}.$$

Algorithms

Putting it all together for Polynomial nonlinear function



$$P_d(x) = \frac{1}{d+1} (a_0 + a_1x + a_2x^2 + \cdots + a_dx^d)$$

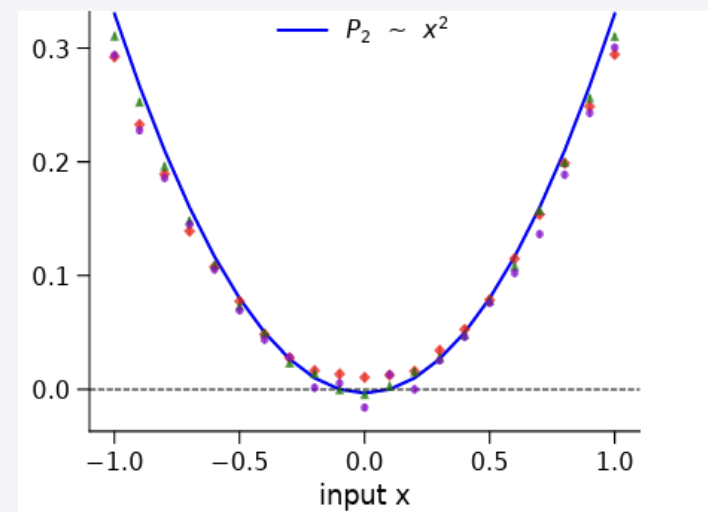
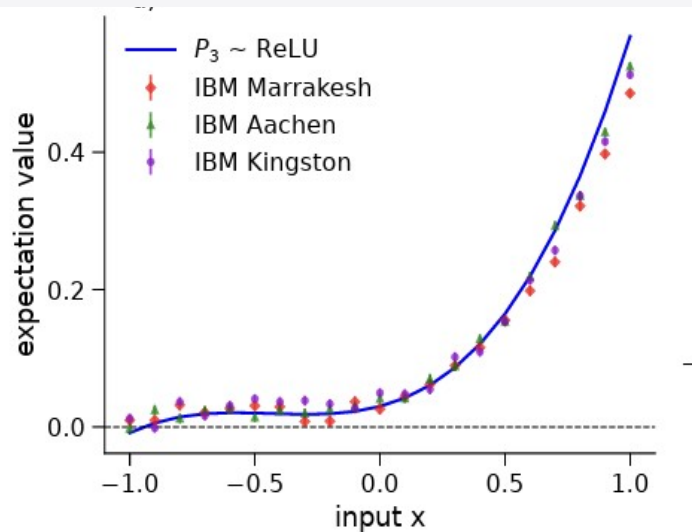
Algorithms

1.46×10^{-3}

median CZ error, IBM Heron-3 (best device used)

$\approx 41-43$

CZ gates before 20-qubit circuits degrade on Heron



CAN WE ACHIEVE QUANTUM INFORMATION ADVAANTAGE

Algorithms

Quick review for modern LLM architecture

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

LLM has linear operation

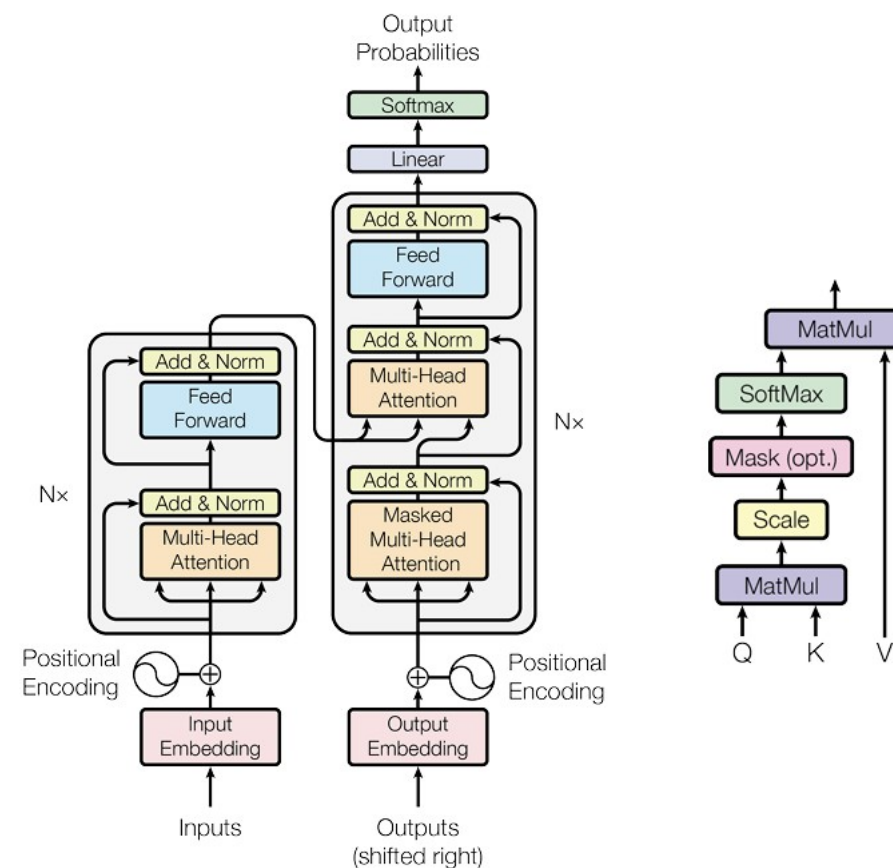
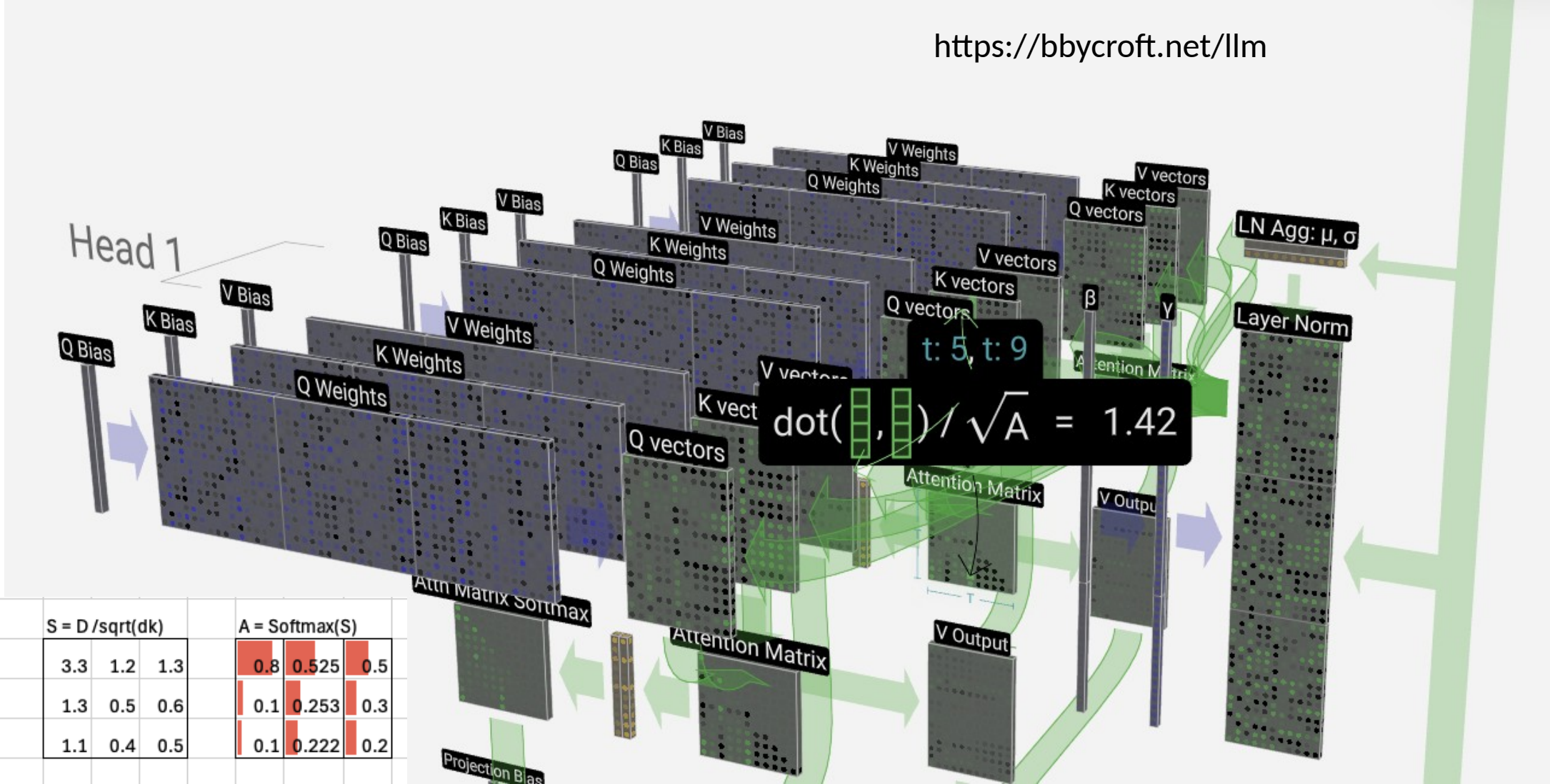


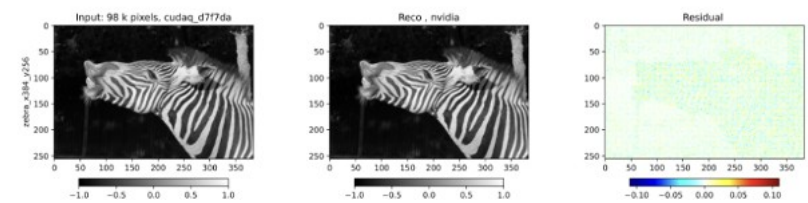
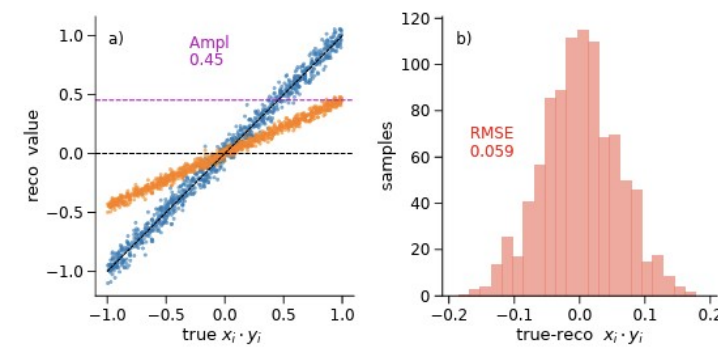
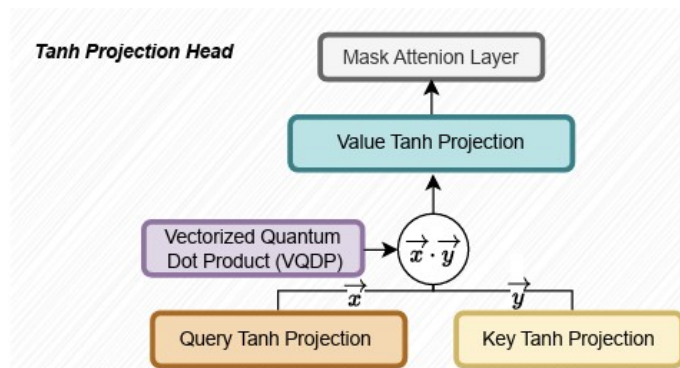
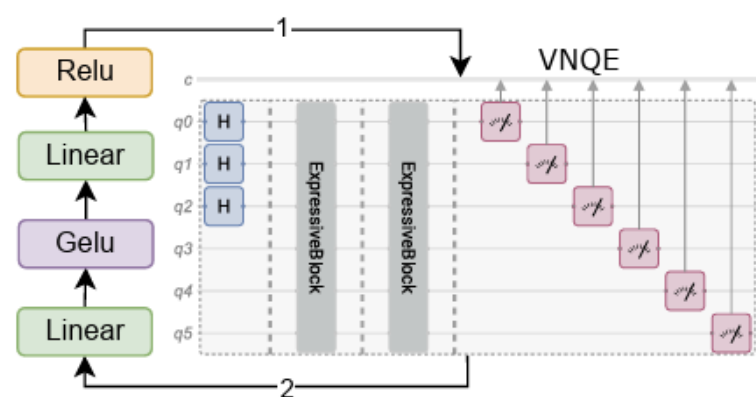
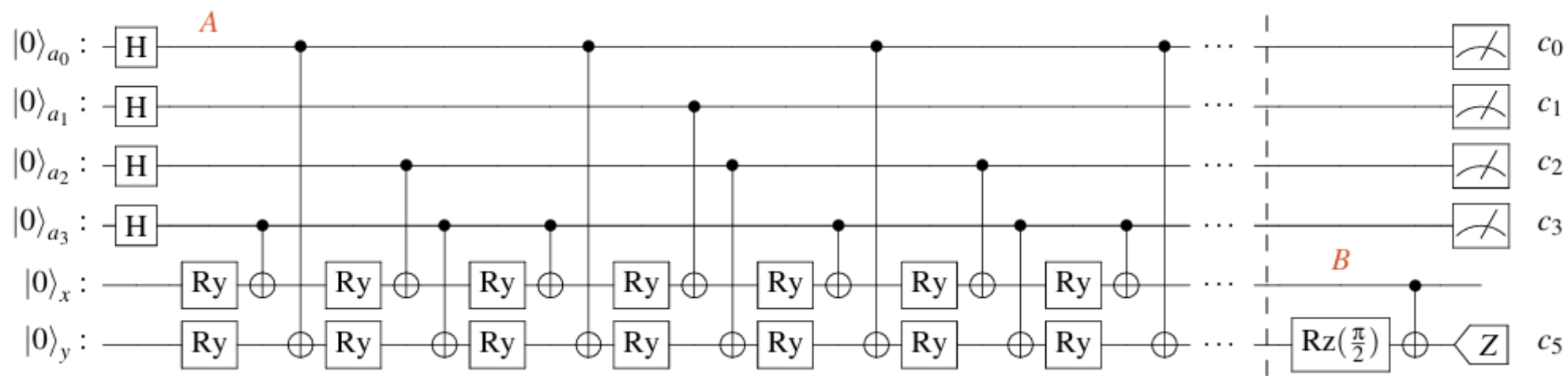
Figure 1: The Transformer - model architecture.

Algorithms

<https://bbycroft.net/llm>



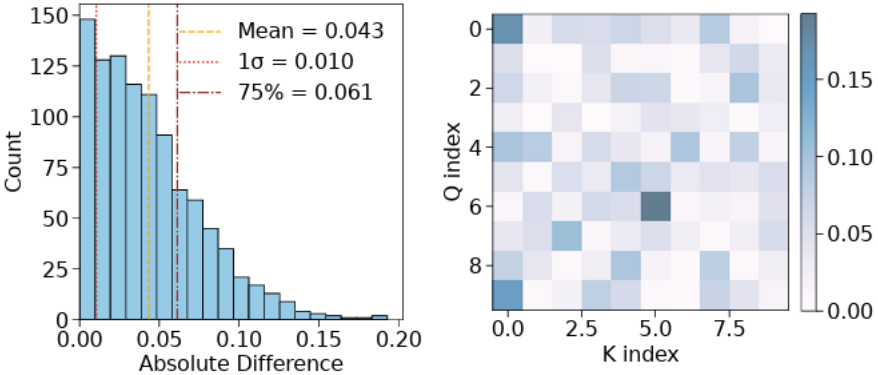
Algorithms



$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

Algorithms

Algorithms



	FLOPS	CLOPS
	Classical matmul	VQDP
Time	$O(BT^2d)$	$O(\log(BT^2)shots)$
Computation cost	1 fused-FLOP	$s \cdot (gt_{1Q} + 2t_{CX}) + t_{ro}$
Runtime	$\lesssim 0.2\text{ ms}$	$\sim 0.1\text{ s}$
Parallel scale	SIMD / GPU cores	feature per circuit

Vectorized attention dot product

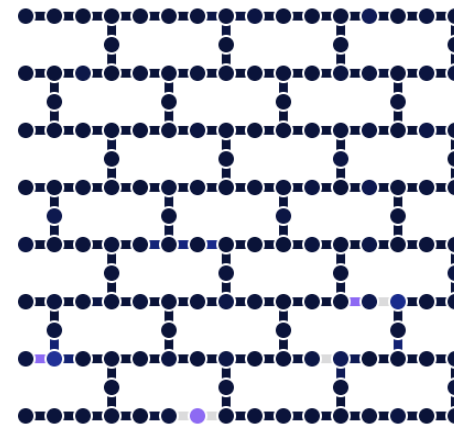
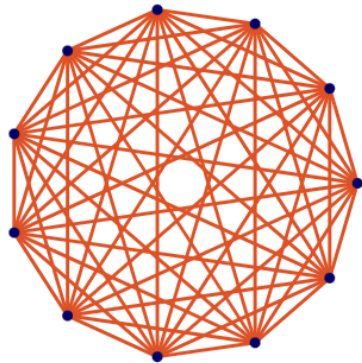
Model	QPL	Loss	Param	Qubit
NanoGPT [†] 2023	92.5	0.94	125K	—
Q-LSTM 2022	125.3	1.18	★	6
Quixer 2024	117.1	1.61	★	●
Hybrid QT 2025	127.6	1.08	★	●
VQT (default)	105.4	1.12	★	●
VQT (large encoder)	108.2	1.08	★	12

Hyperparameter	Default	Large
Vocabulary size (V)	100	100
Sequence length (T)	6	8
Embedding dimension (d)	32	32
Feed-forward hidden size (d_{ff})	128	128
VQT blocks (B)	1	1
Number of QSE (H)	2	2
Address, data qubits (nq_{addr}, nq_{data})	3, 3	6, 6
Encoded dimension Q_{dim}	24	384
Shots per nq_{addr} (S)	1024	1024
AngleMLP hidden size (d_{mlp})	128	128
Dropout (ρ)	0	0
Optimizer	AdamW	AdamW
Learning rate (η)	1×10^{-3}	1×10^{-3}
Batch size (B_{train})	5	5

RESULT

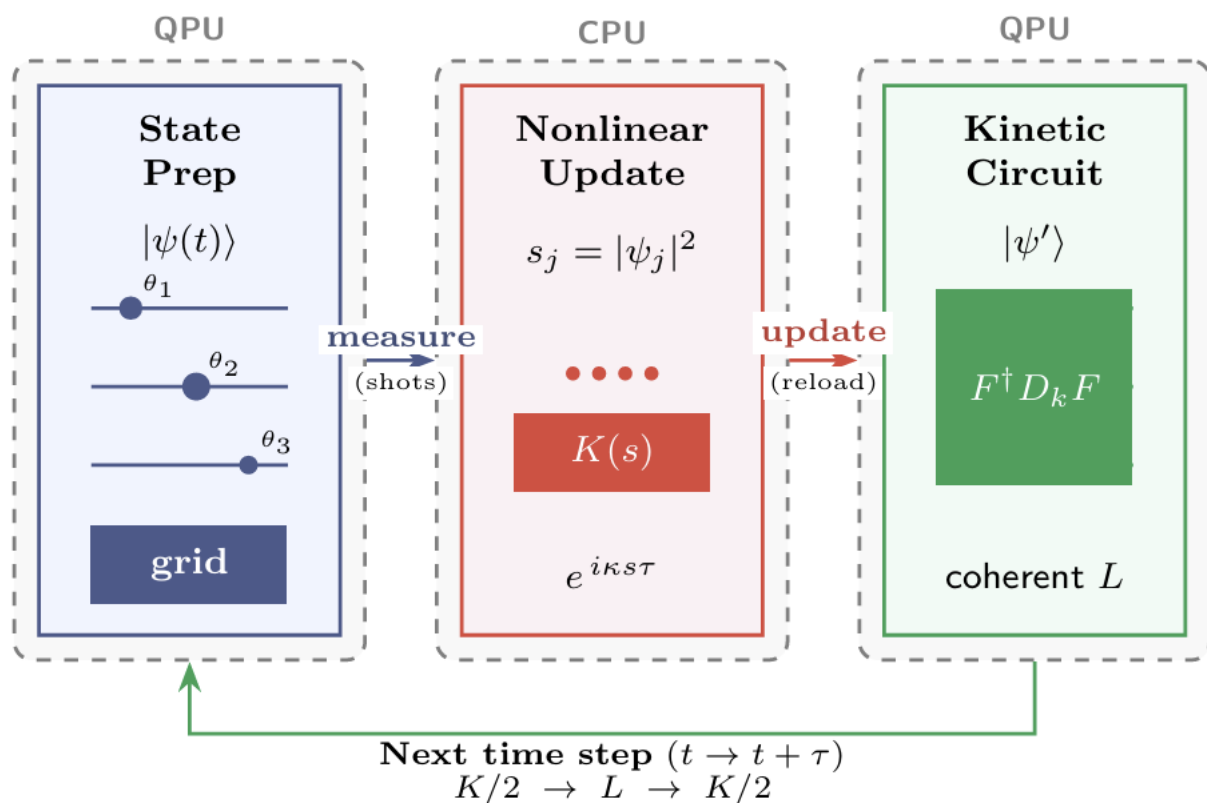
Algorithms

batch size	num qubits	num shots	Ideal			IBM Kingston			IonQ Aria-1 [†]
			num CX ⁺	CX depth	RMSE	num CZ	CZ depth	RMSE	RMSE
4	4	10K	9	5	0.017	18	14	0.022	0.057
8	5	20K	17	9	0.016	29	25	0.023	0.155
16	6	40K	33	17	0.017	78	69	0.037	0.215
32	7	80K	65	33	0.017	164	121	0.059	–
64	8	160K	129	65	0.016	348	249	0.274	–
128	9	320K	257	129	0.016	704	514	–	–

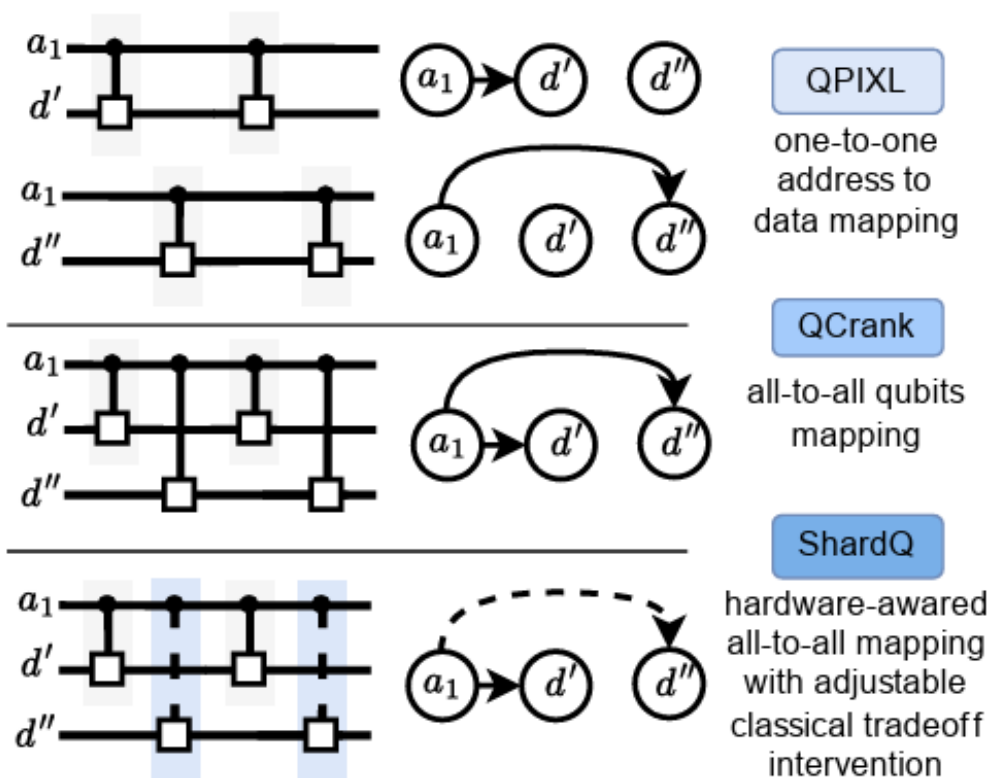


Algorithms

Reloadable nonlinear function solver with quantum circuit simulation



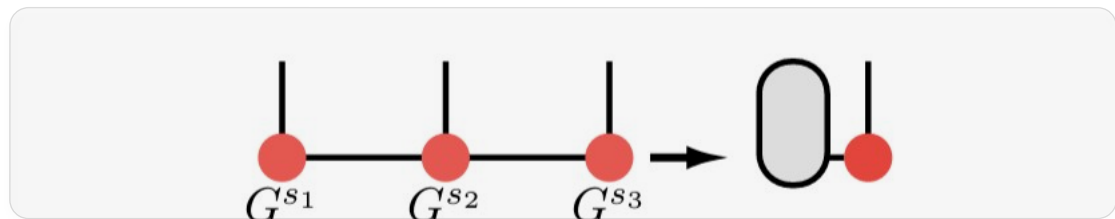
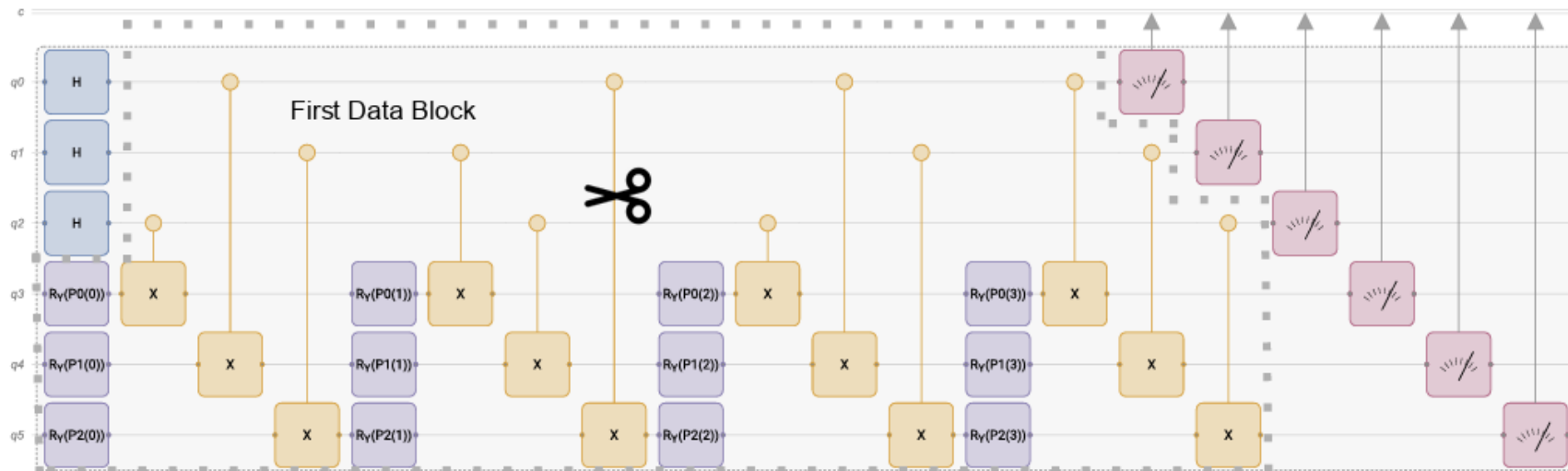
Circuit Cutting and Knitting



Algorithms

Two options:

1. Postprocess with MPS
2. MPS cutting



$$q_c \xrightarrow{\text{CNOT}} q_t = \begin{matrix} \rightarrow \\ \rightarrow \end{matrix} + \begin{matrix} \boxed{Z} \\ \boxed{X} \end{matrix} + \sum_{a_1, a_2} a_1 a_2 \left\{ \begin{matrix} \boxed{\frac{I+a_2 Z}{2}} \rightarrow \boxed{H S^{a_1} H} \\ \boxed{H S^{a_1} H} \rightarrow \boxed{\frac{I+a_2 X}{2}} \end{matrix} \right\}$$

local operations and classical communication (LOCC)



Gate	Decomposition	Gate	Decomposition	Gate	Decomposition
X	$H \cdot Z \cdot H$	R_Z	$S^\dagger$	Π_Y	$H \cdot S^\dagger \cdot H \cdot P_0 \cdot H \cdot S \cdot H$
Y	$H \cdot Z \cdot H \cdot Z$	R_{YZ}	$H \cdot S^\dagger \cdot H \cdot Z$	Π_Z	P_0
Z	S^2	R_{ZX}	$S^\dagger \cdot H \cdot S^\dagger \cdot H \cdot S^\dagger$	Π_{YZ}	$S \cdot H \cdot S \cdot H \cdot P_0 \cdot H \cdot S \cdot H \cdot S^\dagger$
R_X	$H \cdot S^\dagger \cdot H$	R_{XY}	$H \cdot Z \cdot H \cdot S^\dagger$	Π_{ZX}	$H \cdot S^\dagger \cdot H \cdot P_0 \cdot H \cdot S \cdot H \cdot Z$
R_Y	$S \cdot H \cdot S^\dagger \cdot H \cdot S^\dagger$	Π_X	$S \cdot H \cdot S \cdot H \cdot P_0 \cdot H \cdot S^\dagger \cdot H \cdot S^\dagger$	Π_{XY}	$P_0 \cdot H \cdot Z \cdot H$

- [33] K. Mitarai and K. Fujii, “Constructing a virtual two-qubit gate by sampling single-qubit operations,” *New Journal of Physics*, vol. 23, no. 2, p. 023021, 2021.
- [34] T. Peng, A. W. Harrow, M. Ozols, and X. Wu, “Simulating large quantum circuits on a small quantum computer,” *Physical review letters*, vol. 125, no. 15, p. 150504, 2020.
- [35] D. P. DiVincenzo, “Two-bit gates are universal for quantum computation,” *Physical Review A*, vol. 51, no. 2, p. 1015, 1995.

NISQ RESULTS

Algorithms

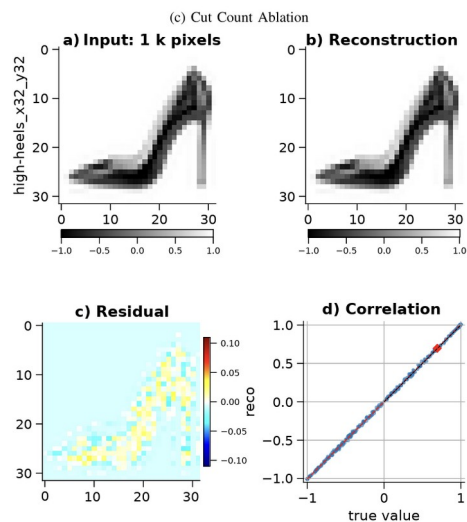
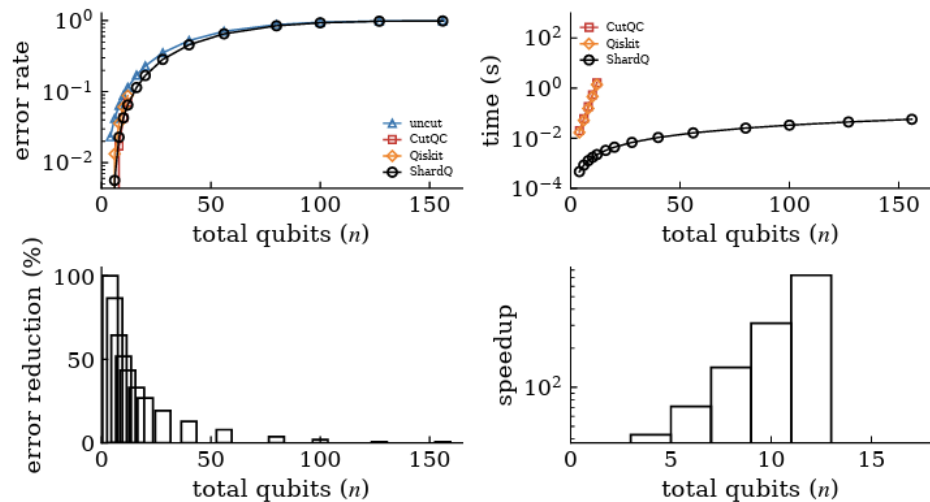
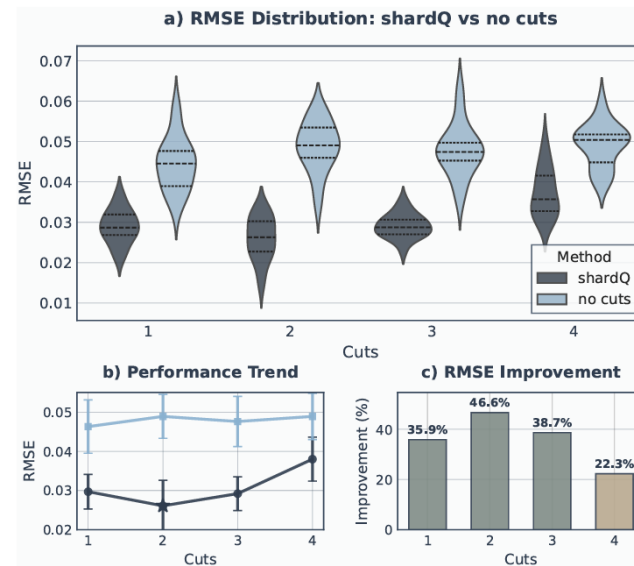


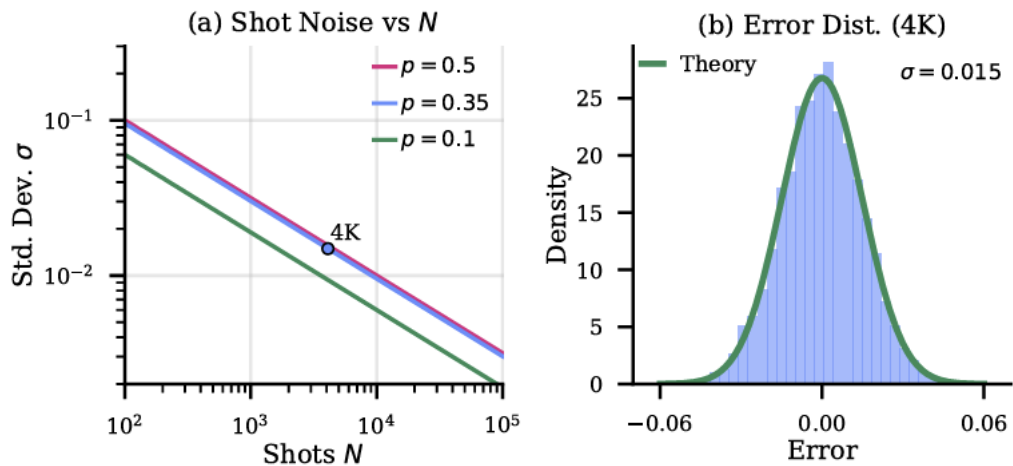
TABLE I: Summary of experiments on IBM Heron QPUs. RVF is the reconstruction vector fidelity defined in Eq. (22).

Exp. Method	#1 QCrack	#2 shardQ	#3 QCrack	#4 shardQ	#5 shardQ
Device	ibm_pittsburgh				marrakesh
CZ err	(1.46×10^{-3})				—
n_a	4	4	5–7	5–7	4
n_d	8	8	8	8	8
# Addr.	16	16	32–128	32–128	16
Input (b)	128	128	256–1k	256–1k	128
RVF	0.79	0.90	0.75	0.88	0.75



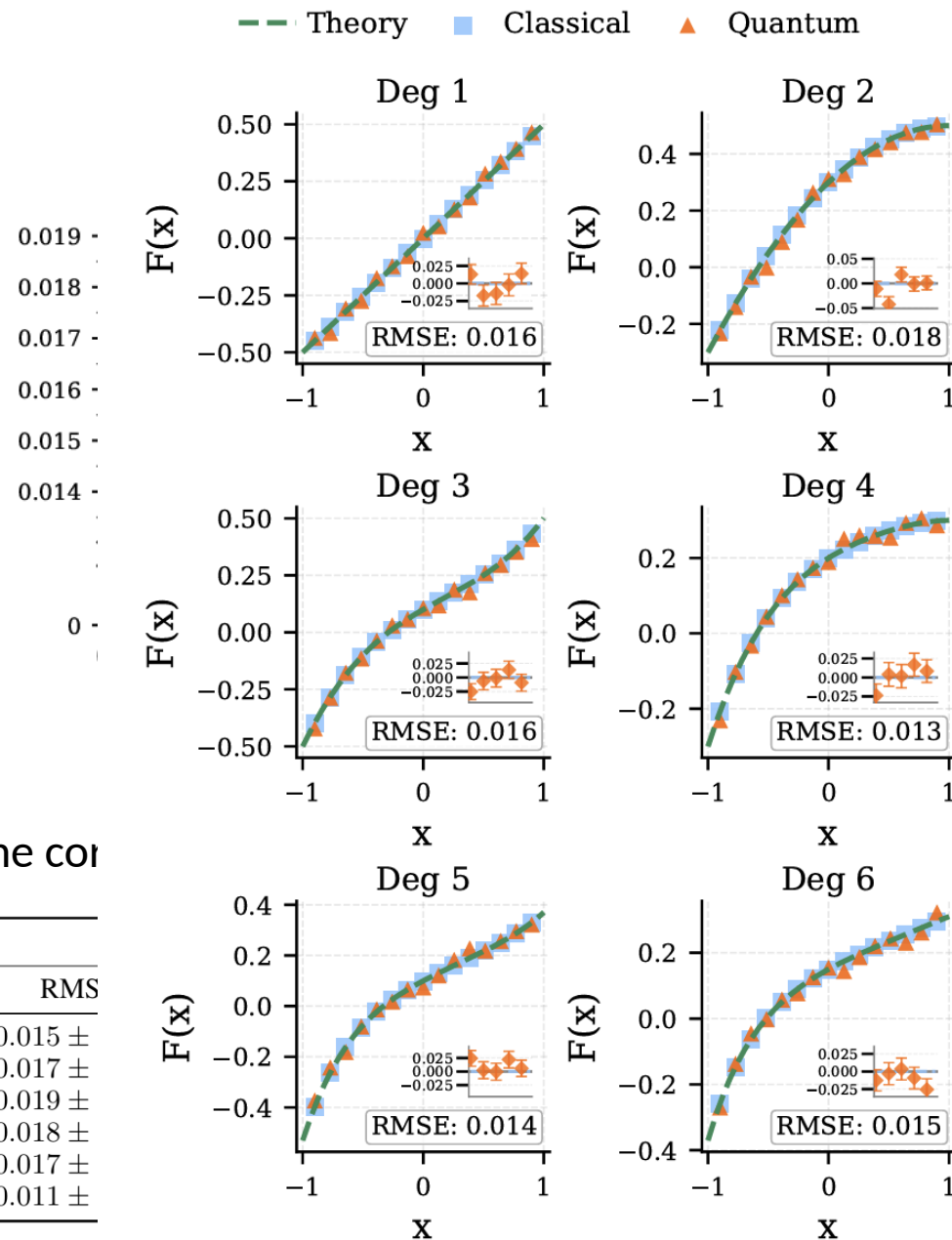
Algorithms

$$|\mathcal{F}(x) - \hat{P}_d(x)| \leq \underbrace{\|\mathcal{F} - P_d\|_\infty}_{\epsilon_{\text{classical}}} + \underbrace{O(1/\sqrt{N})}_{\epsilon_{\text{shot}}}$$



We collaborate with quantum lab (UMD) for getting the cor

Degree (d)	AerSimulator			IBM Heron3			RMS
	RMSE	Corr.	Pass %	RMSE	Corr.	Pass %	
1 (Linear)	0.016 ± 0.002	0.999	95.6	0.021 ± 0.003	0.999	84.0	$0.015 \pm$
2 (Quadratic)	0.018 ± 0.001	0.997	91.1	0.020 ± 0.004	0.998	86.0	$0.017 \pm$
3 (Cubic)	0.016 ± 0.001	0.998	95.6	0.022 ± 0.003	0.998	82.7	$0.019 \pm$
4 (Quartic)	0.013 ± 0.001	0.997	97.8	0.023 ± 0.004	0.996	79.3	$0.018 \pm$
5 (Quintic)	0.014 ± 0.002	0.998	97.8	0.022 ± 0.004	0.998	82.7	$0.017 \pm$
6 (Sextic)	0.015 ± 0.001	0.996	95.6	0.024 ± 0.004	0.995	74.0	$0.011 \pm$

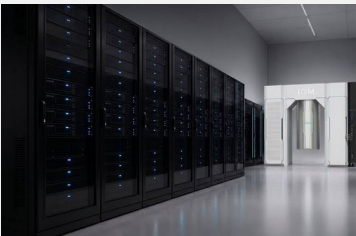


Applications & Research



01 · RQ1 · Scale

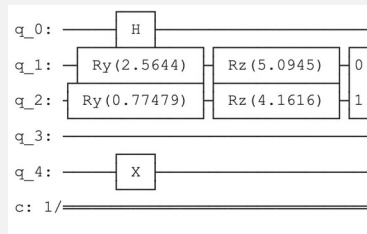
Simulation



GPU / QPU
simulation beyond
hardware reach

02 · RQ2 · QIP

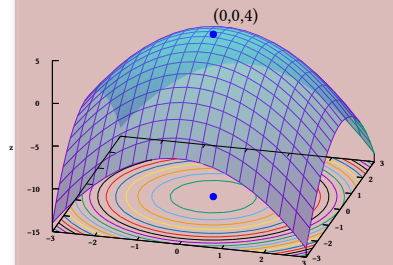
Algorithms



Structure from
problem and device,
not black-box search

03 · RQ4 · Synthesis

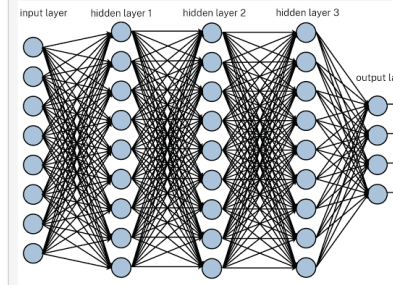
Optimization



AI/HPC synthesis
within fault-tolerant
budgets

04 · RQ3 · LLM

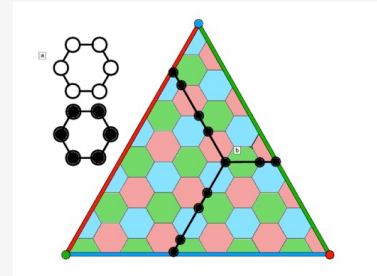
LLMs



AI for quantum

05 · RQ5 · Trust

QEC



Auditable QEC and
post-quantum
cryptography

Optimization

A Quantum Approximate Optimization Algorithm

Edward Farhi, Jeffrey Goldstone, Sam Gutmann

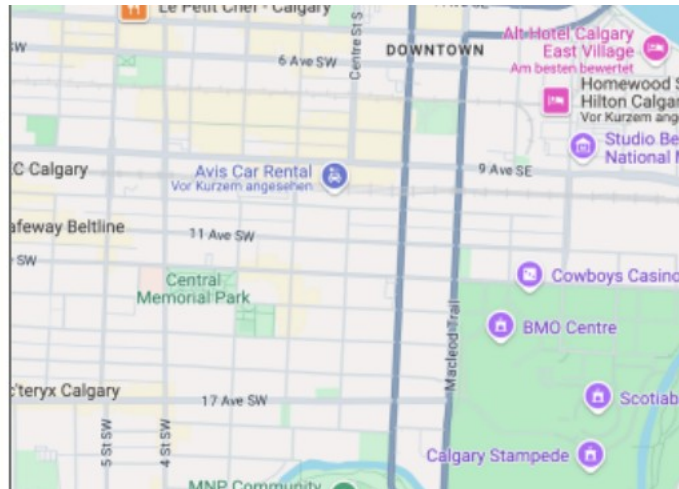
We introduce a quantum algorithm that produces approximate solutions for combinatorial optimization problems. For a fixed positive integer p and the quality of the approximation improves as p is increased. The quality of the approximation is measured by the expected value of the cost function.



Universal computation by quantum walk

Andrew M. Childs

In some of the earliest work on quantum mechanical computers, Feynman showed how to implement the dynamics of a time-independent Hamiltonian. I show that this remains possible even if the Hamiltonian entries equal to 0 or 1, i.e., the adjacency matrix of a low-degree graph. Thus quantum walk can be used for universal computation.



$$H_C|z\rangle = C(z)|z\rangle.$$

Optimization

n bits m clauses

$$C(z) = \sum_{i=1}^m C_i(z)$$

$$z = z_1 \dots z_n$$

$$C_2(z) = \max_z C(z)$$

$$\frac{C(z)}{C_{\max}} \uparrow \text{big}$$

Boaz Barak

Ankur Moitra

Ryan O'Donnell

Prasad Raghavendra

Oded Regev

David Steurer

Luca Trevisan

Aravindan Vijayaraghavan

David Witmer

John Wright

Johan Håstad

Algorithm: $(\frac{1}{2} + \frac{\text{const}}{D})$ (2000)

QAOA: $(\frac{1}{2} + \frac{\text{const}}{D^{3/4}})$ (2014)

classic Algorithm $(\frac{1}{2} + \frac{\text{const}}{D^{1/2}})$ (2015)

D here is the graph degrees

n is total number of bits ($n = D+1$)

Optimization

Classical and Quantum Bounded Depth Approximation Algorithms

M. B. Hastings

2019

We consider some classical and quantum approximate optimization algorithms with bounded depth. First, we define a class of optimization algorithms and show that a single step version of these algorithms can achieve the same performance as the original algorithms in the literature.

Local classical MAX-CUT algorithm outperforms $p = 2$ QAOA on high-girth regular graphs

Kunal Marwaha

Bounds on approximating Max kXOR with quantum and classical local algorithms

Kunal Marwaha, Stuart Hadfield

We consider the power of local algorithms for approximately solving Max kXOR, a generalization of two constraint satisfaction problems previously studied with classical and quantum algorithms (MaxCut and Max E3LIN2). In Max kXOR each constraint is the XOR of exactly k variables and a parity bit. On instances with either random signs (or expected satisfying fraction of the depth-1 QAOA from Hirvonen et al [arXiv:1402.2543]). Notably

for combinatorial optimization on noisy beyond $p = 1$. We analyze QAOA₂ for the maximum cut on any D-regular graph of girth > 5 (i.e.

The Quantum Approximate Optimization Algorithm at High Depth for MaxCut on Large-Girth Regular Graphs and the Sherrington-Kirkpatrick Model

Joao Basso, Edward Farhi, Kunal Marwaha, Benjamin Villalonga, Leo Zhou

[Submitted on 21 Sep 2020 (v1), last revised 16 Jun 2021 (this version, v4)]

Warm-starting quantum optimization

proximate solutions to combinatorial optimization problems. Its performance on MaxCut on large-girth D-regular graphs. We give an iterative formula to evaluate the performance on these graphs, at optimal parameters and as D goes to infinity, we find that the

Daniel J. Egger, Jakub Mareš

There is an increasing interest in solving such problems employ relaxation techniques. Valued problems (semidefinite programming) available classically in polynomial time.

Spin-Boson Mapping of the Quantum Approximate Optimization Algorithm

Sami Boulebnane^{1,*}, Abid Khan^{1,*}, Minzhao Liu^{1,†}, Jeffrey Larson², Dylan Herman¹, Ruslan Shaydulin^{1,‡}, and Marco Pistoia¹

Show more

$$P_* \equiv \lim_{n \rightarrow \infty} \frac{\mathbb{E}_J[\max_s C(s)]}{n} \approx 0.763166$$

Optimization

Beating the random assignment on
constraint satisfaction problems of bounded degree

Boaz Barak* Ankur Moitra[†] Ryan O'Donnell[‡] Prasad Raghavendra[§]
Oded Regev[¶] David Steurer^{||} Luca Trevisan[§] Aravindan Vijayaraghavan^{**}
David Witmer[‡] John Wright[‡]

August 12, 2015



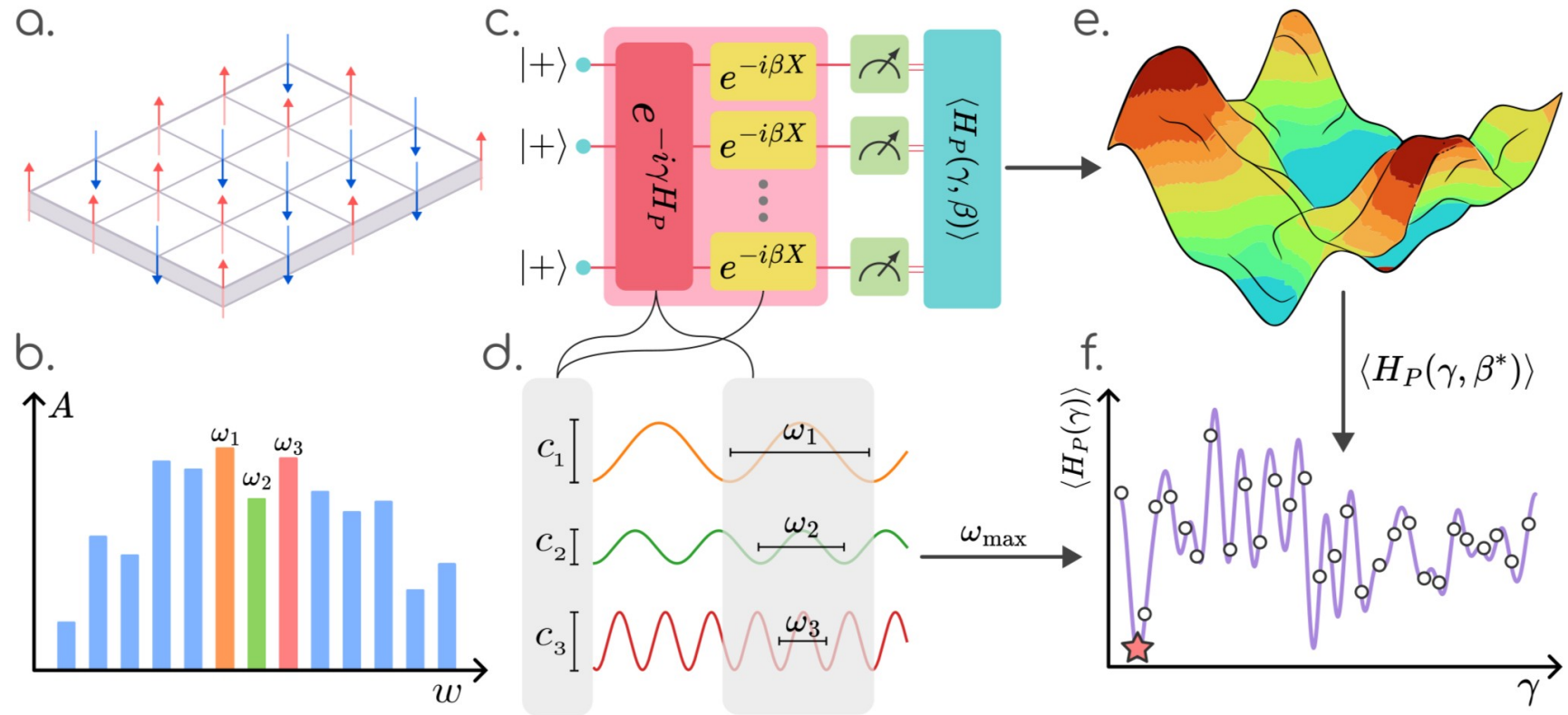
« [What I believe](#)

[BQP/LHC collision](#) »

Quantum computing news items (by reader request)

Within the last couple months, there was a major milestone in the quest to build a scalable quantum computer, and also a major milestone in the quest to figure out what you would *do* with a quantum computer if you had one. As I've admitted many times, neither of these two quests is really the reason why I got into quantum

Optimization



Near-Optimal Parameter Tuning of Level-1 QAOA for Ising Models

V Vijendran^{1,2,3}, Dax Enshan Koh^{2,4,5,6}, Eunok Bae^{7,8}, Hyukjoon Kwon⁷,
Ping Koy Lam^{2,1,3}, and Syed M Assad^{2,3}

$$|\psi(\gamma, \beta)\rangle = \underbrace{e^{-i\beta \sum_j X_j}}_{\text{mix candidates}} \underbrace{e^{-i\gamma H_C}}_{\text{encode scores into phases}} |+\rangle^{\otimes n}$$

Optimization

Algorithm 1 DEAL with Dynamic Physical Mapping

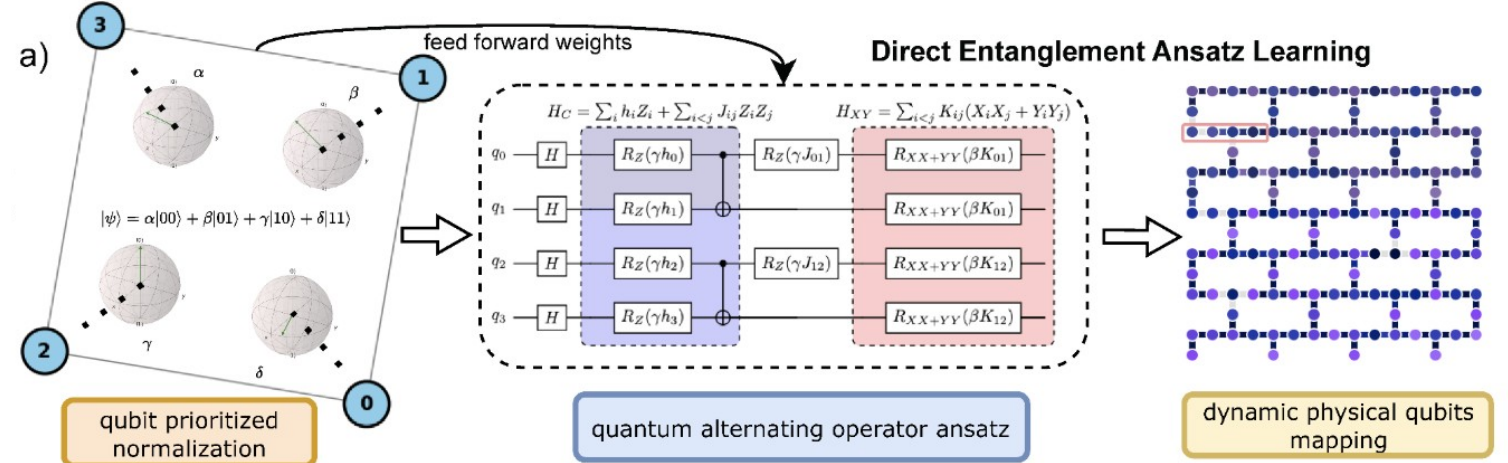
Require: QUBO matrix Q , depth p , device topology

G_{device} , error rates E_{ij}

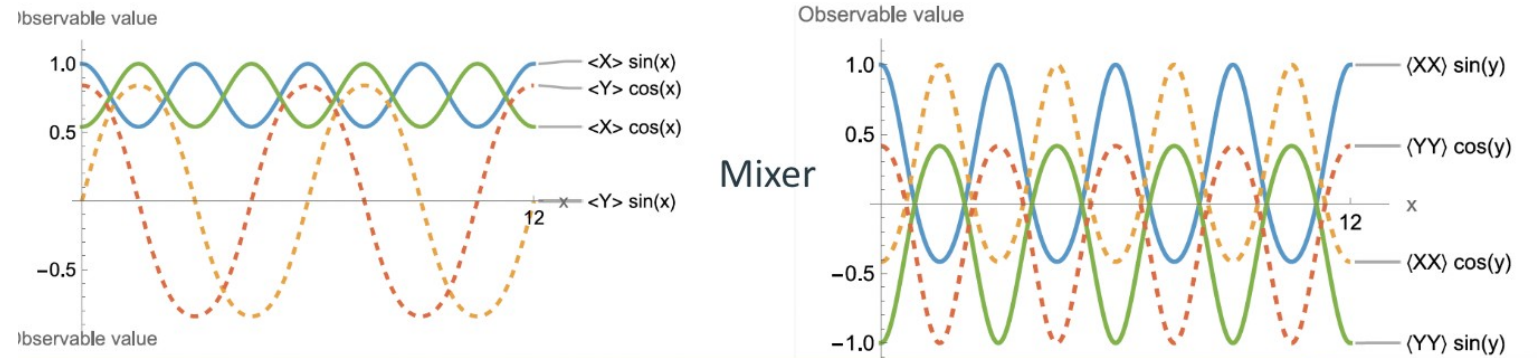
Ensure: optimized θ^* , physical mapping π

- 1: **QPN:** Compute $s_i = \sum_j |Q_{ij}|$, normalize $w_i = s_i / \sum_\ell s_\ell$
- 2: Initialize $\gamma_k^0 = \frac{\lambda_\gamma k}{np} \sum_i \arccos(1 - 2w_i)$, $\beta_k^0 = \frac{\lambda_\beta (p-k)}{np} \sum_i \arcsin(\sqrt{w_i})$
- 3: **Mapping:** Sort qubits by w_i , minimizing $\sum_{ij} w_{ij} \cdot d(\pi(i), \pi(j)) \cdot E_{\pi(i)\pi(j)}$
- 4: **Ansatz:** Set $\theta^0 = (\gamma_1^0, \dots, \gamma_p^0, \beta_1^0, \dots, \beta_p^0)$
- 5: **for** $t = 1$ to optimizer budget **do**
- 6: Evaluate $f(\theta_t) = \langle \psi(\theta_t) | H_C | \psi(\theta_t) \rangle$ with S shots
- 7: Update $\theta_{t+1} \leftarrow \text{OptimizerStep}(\theta_t, f(\theta_t))$
- 8: **end for**
- 9: **return** θ^*, π

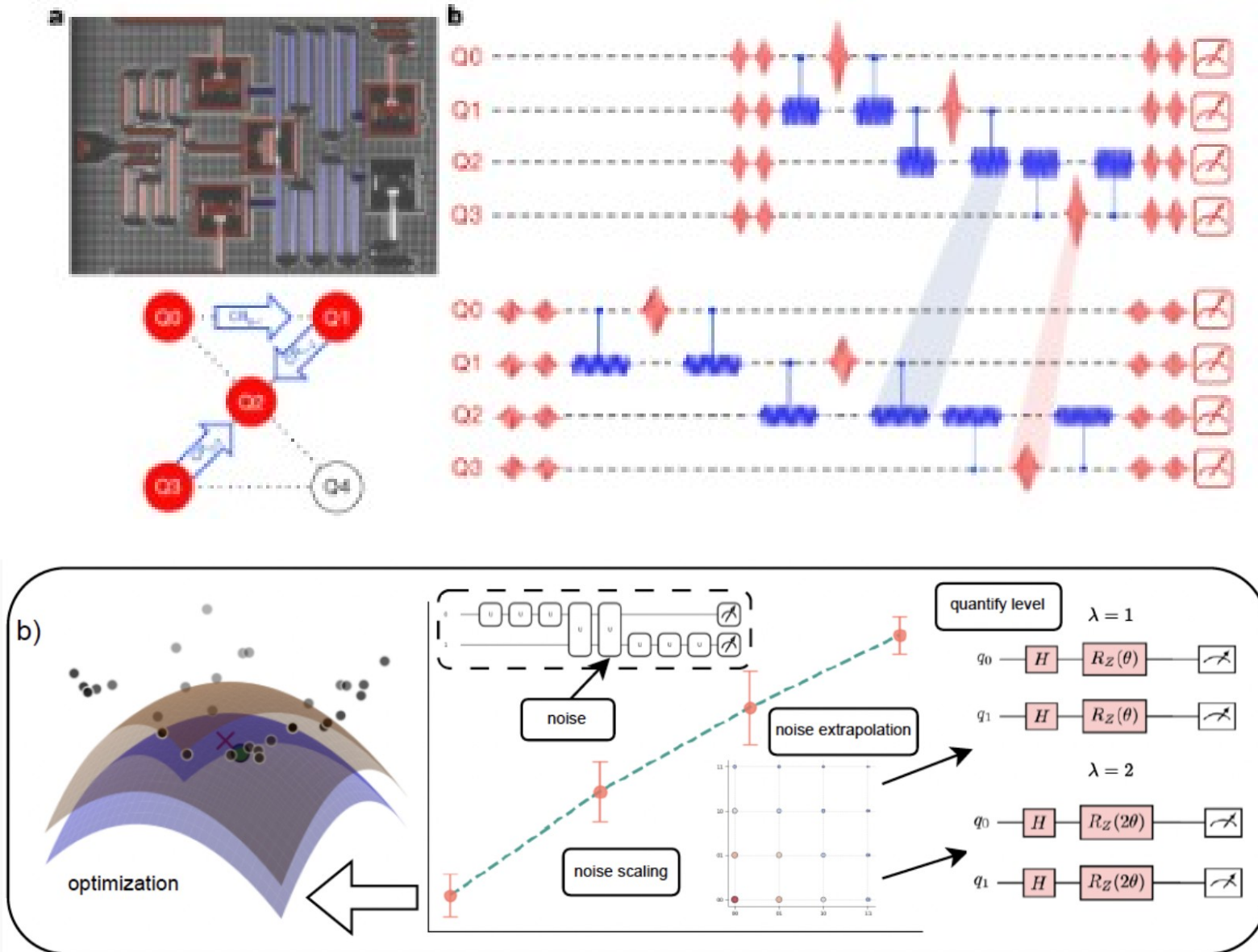
Can we map the problem directly into quantum circuit based on the clause (constraint)?



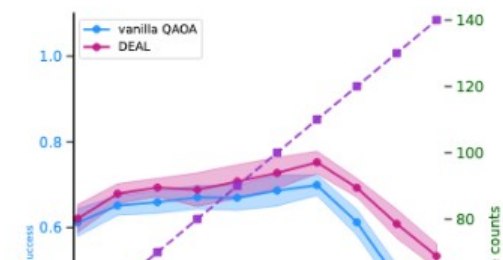
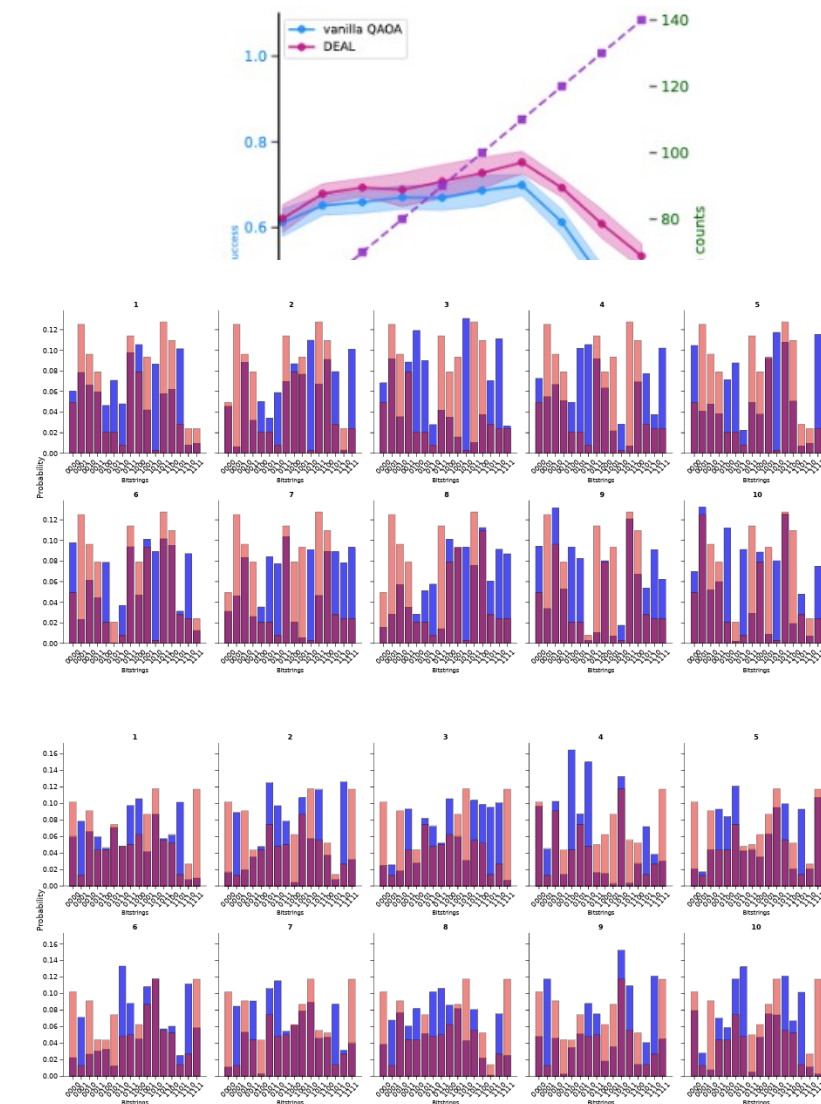
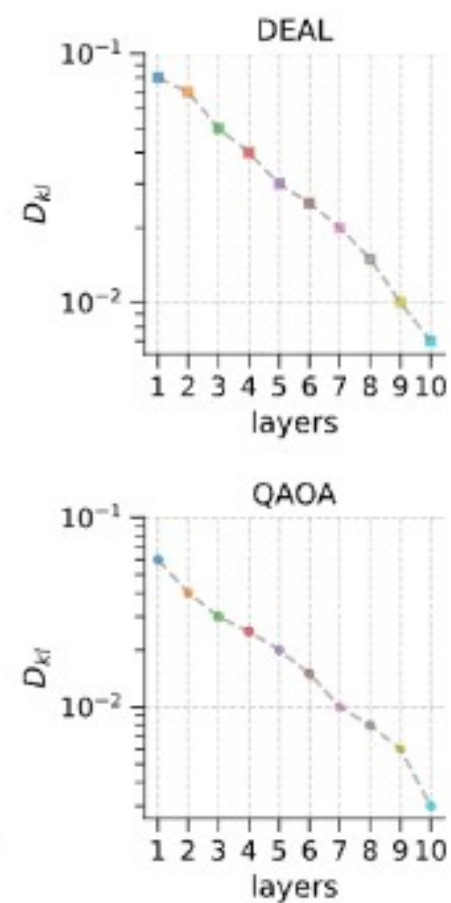
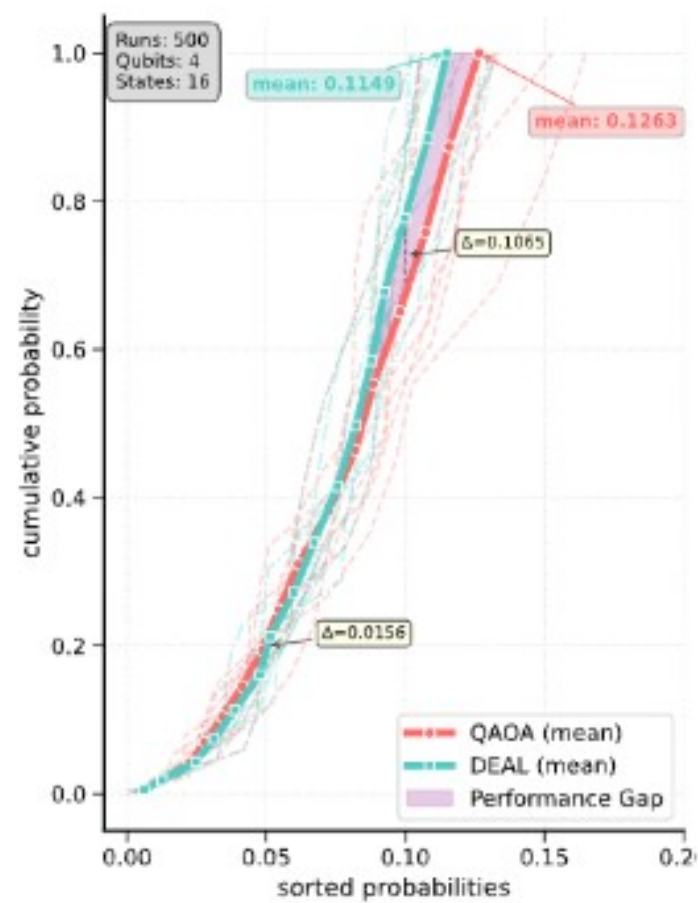
Qubit prioritization normalization(QPN)



Optimization

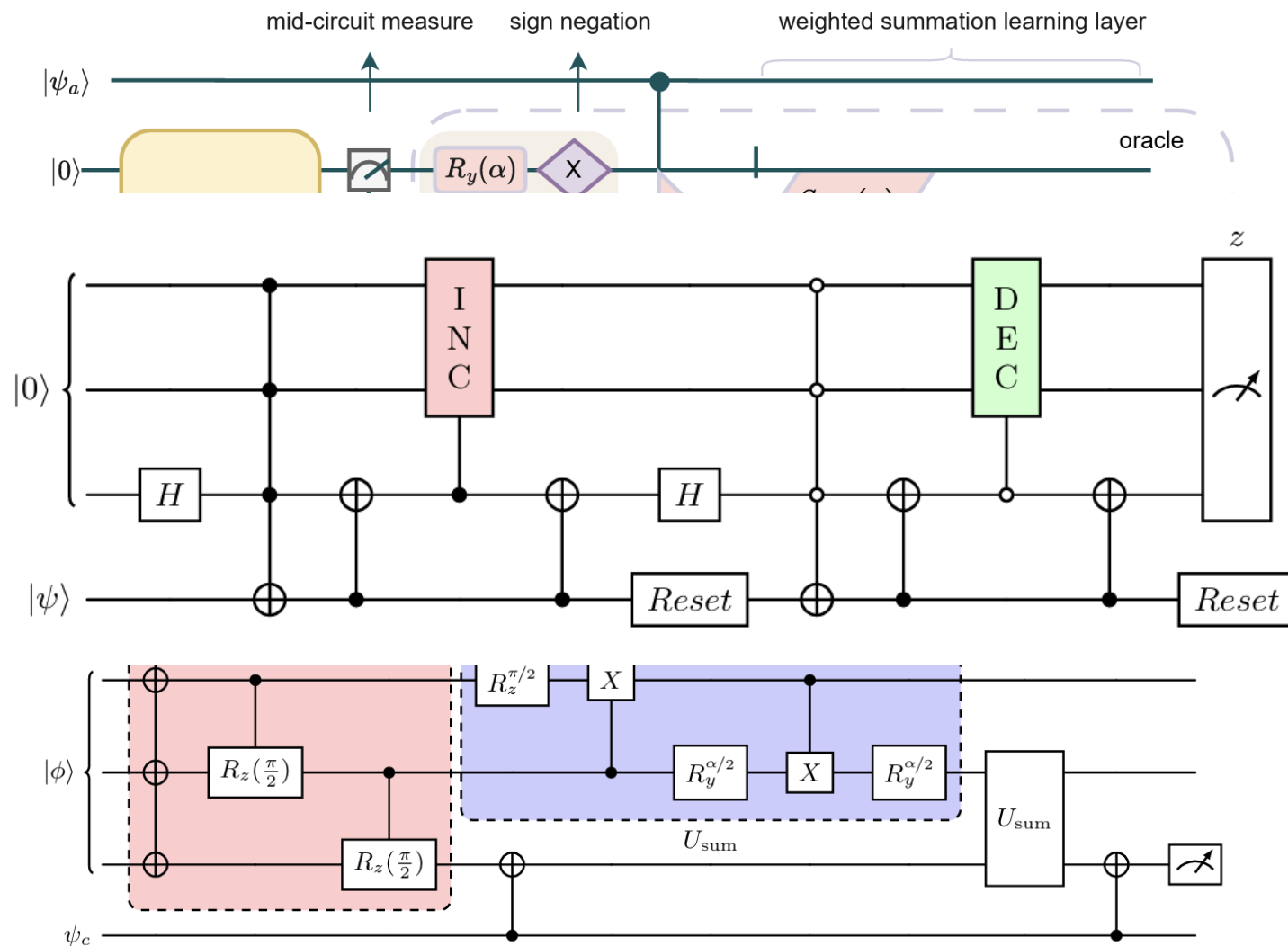


Optimization

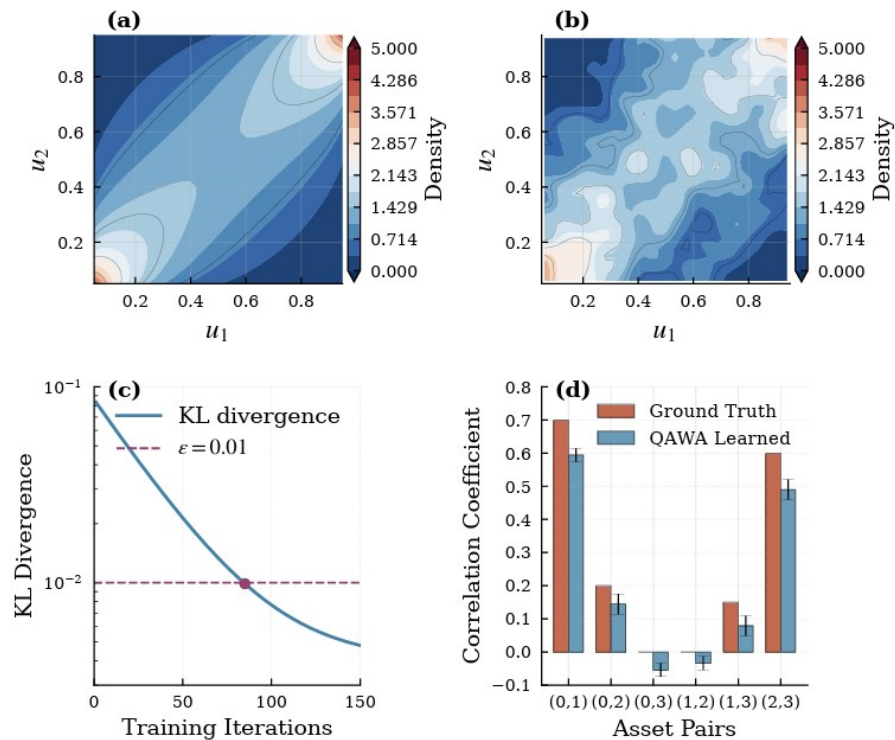
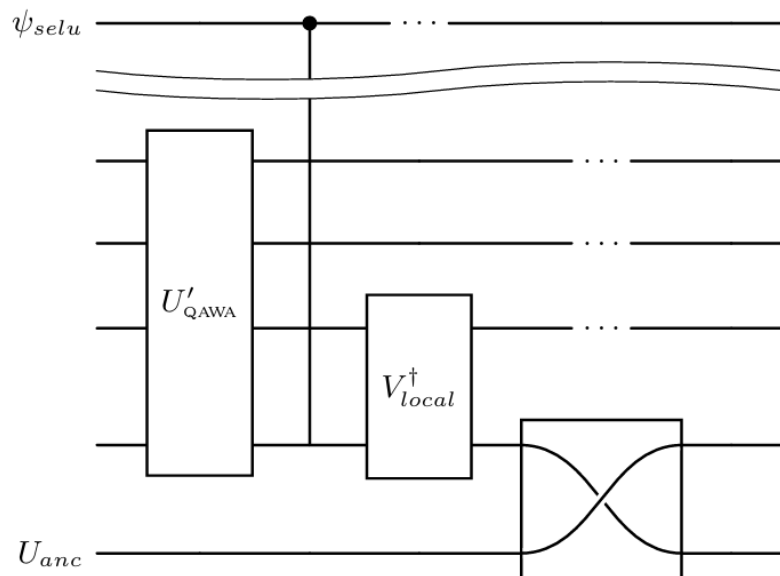


Optimization

$$s_3 = w_1 [w_0 x_0 + (1 - w_0) x_1] + (1 - w_1) x_2 = \underbrace{w_1 w_0}_{a_0} \cdot x_0 + \underbrace{w_1 (1 - w_0)}_{a_1} \cdot x_1 + \underbrace{(1 - w_1)}_{a_2} \cdot x_2,$$



Optimization



Learning Shallow Quantum Circuits

Hsin-Yuan Huang[†]
 California Institute of Technology
 Pasadena, USA
 Google Quantum AI
 Venice, USA
 hsinyuan@caltech.edu

Yunchao Liu[†]
 University of California, Berkeley
 Berkeley, USA
 yunchaoliu@berkeley.edu

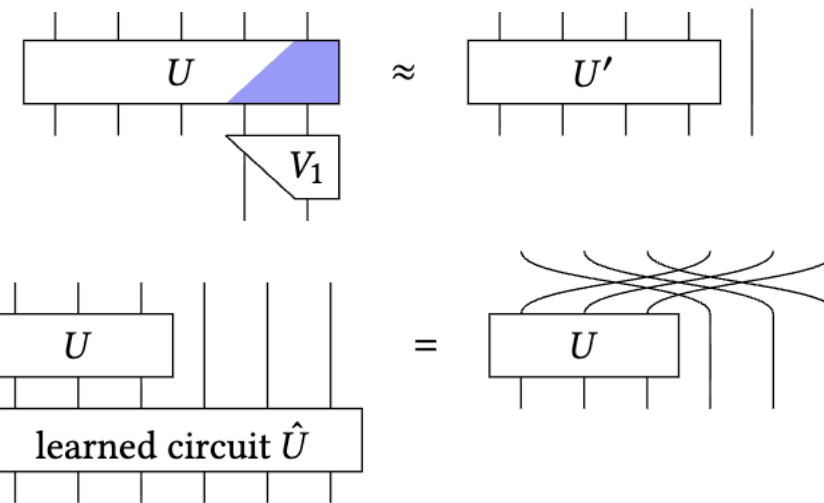
Michael Broughton
 Google Quantum AI
 Santa Barbara, USA
 michael.broughton99@gmail.com

Isaac Kim
 University of California, Davis
 Davis, USA
 isaac.kim.quantum@gmail.com

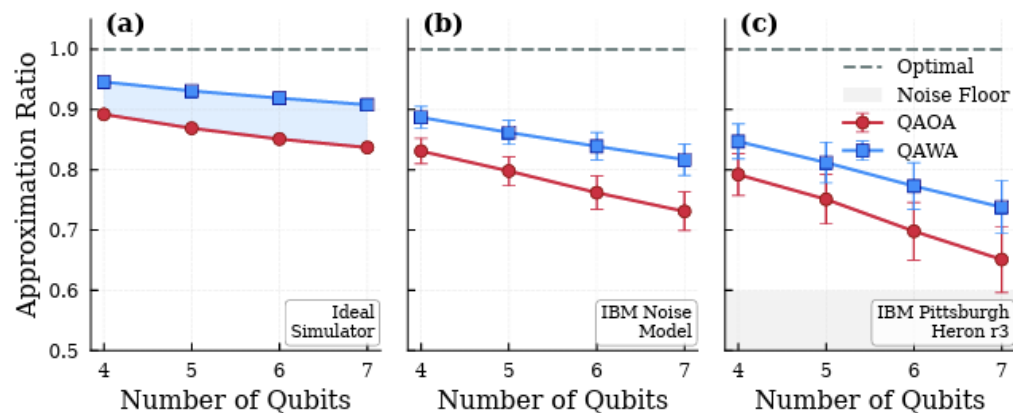
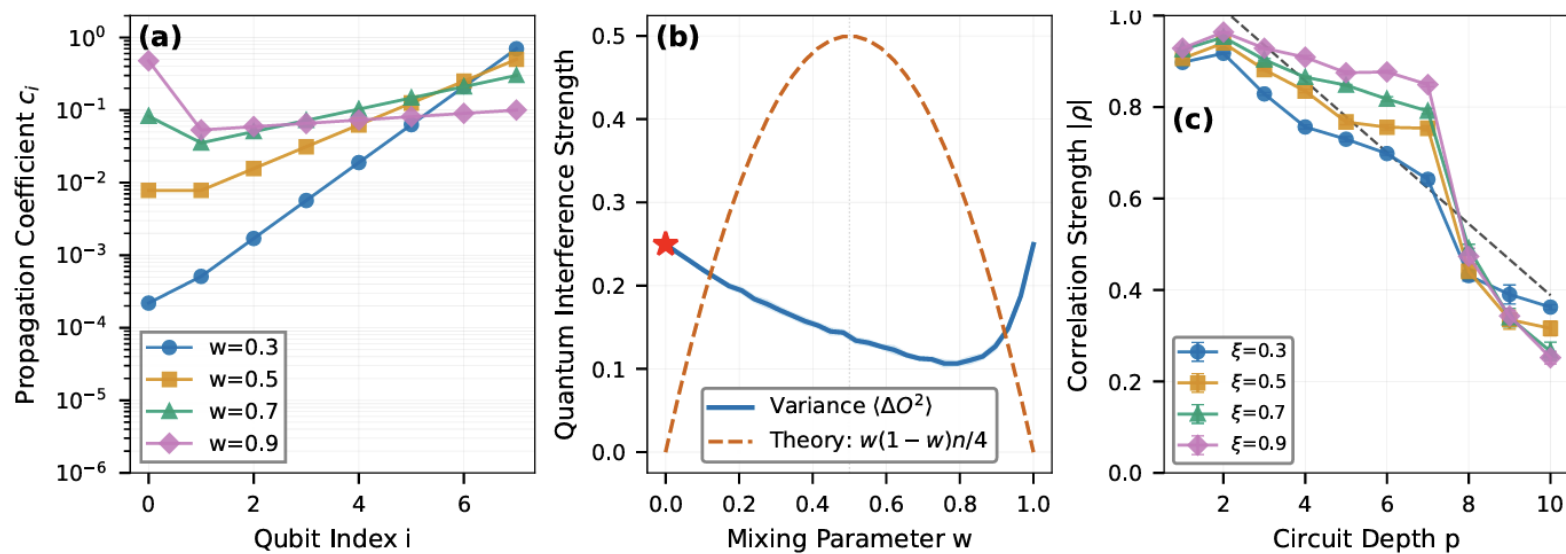
Anurag Anshu
 Harvard University
 Cambridge, USA
 anuraganshu@fas.harvard.edu

Zeph Landau
 University of California, Berkeley
 Berkeley, USA
 zeph.landau@gmail.com

Jarrod R. McClean
 Google Quantum AI



Optimization

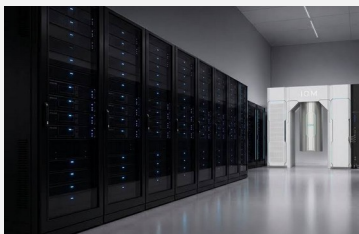


Applications & Research



01 · RQ1 · Scale

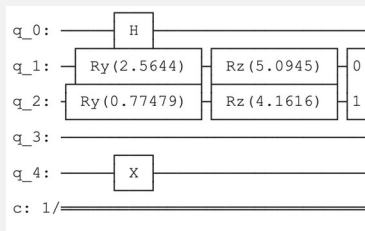
Simulation



GPU / QPU
simulation beyond
hardware reach

02 · RQ2 · QIP

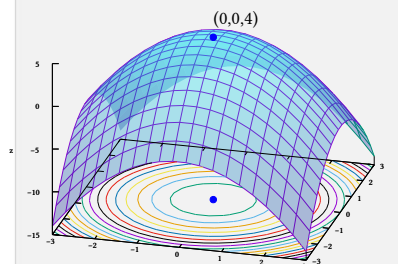
Algorithms



Structure from
problem and device,
not black-box search

03 · RQ4 · Synthesis

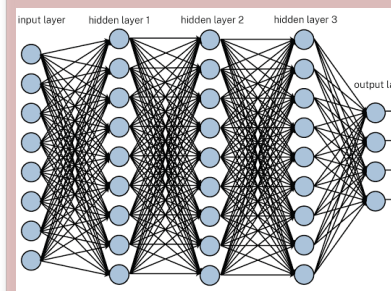
Optimization



AI/HPC synthesis
within fault-tolerant
budgets

04 · RQ3 · LLM

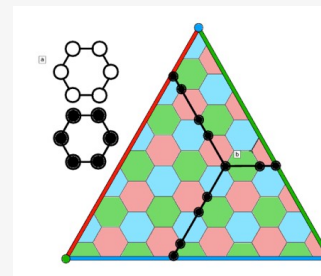
LLMs



AI for quantum

05 · RQ5 · Trust

QEC



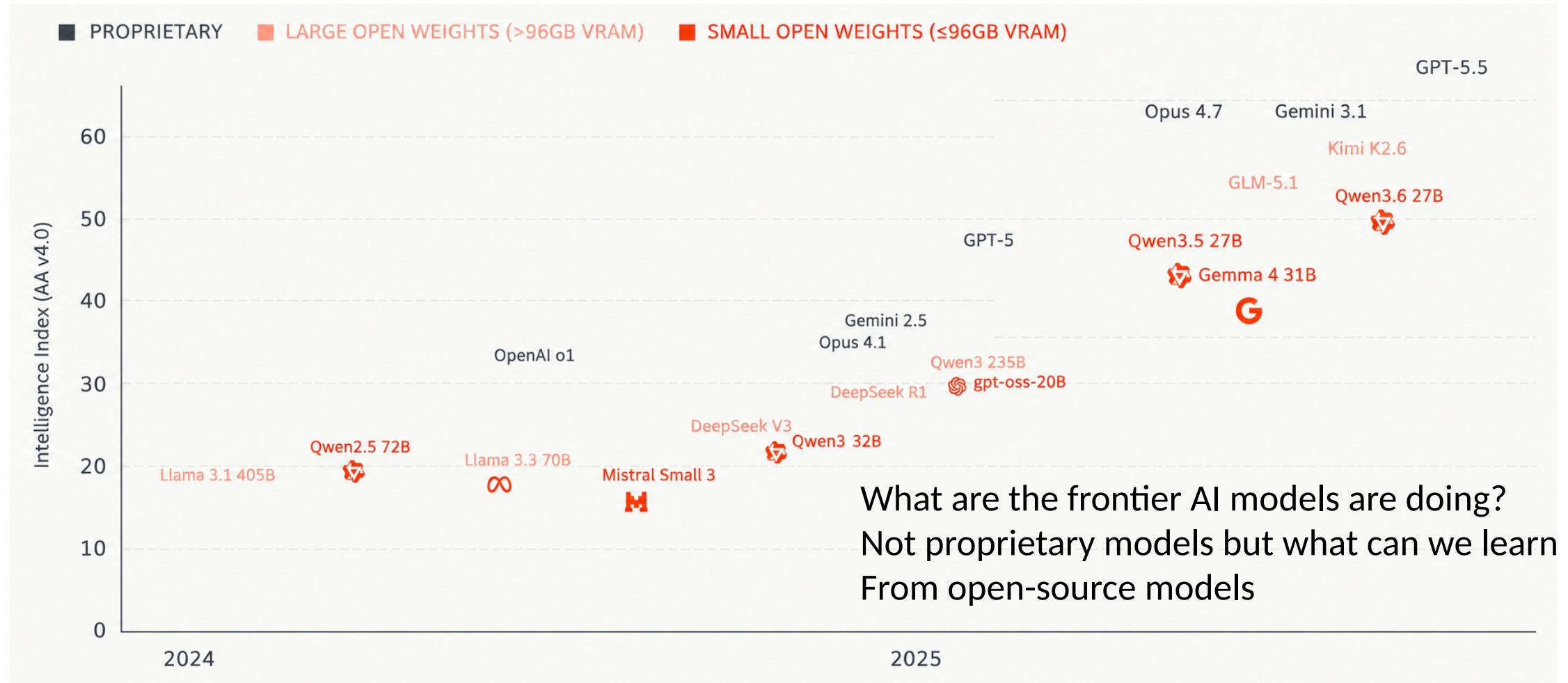
Auditable QEC and
post-quantum
cryptography

LLMs

$$o_t = \sum_{j=1}^t p_{t,j} v_j, \quad p_{t,j} = \frac{\exp(q_t k_j^\top / \sqrt{d_k})}{\sum_{\ell=1}^t \exp(q_t k_\ell^\top / \sqrt{d_k})}$$

$$\text{Attention}(Q, K, V) = \text{softmax}_{\text{row}} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V, \quad M_{t,j} = \begin{cases} 0, & j \leq t, \\ -\infty, & j > t. \end{cases}$$

LLMs



LLMs

	q1	q2	q3	q4	q5	q6	q7	q8	q9	q10	q11
Q	1.2	1.3	1.1	1.1	1	0.7	1.2	1.5	1	0	1
	1.1	1.2	0.4	1	0.8	0.8	1.4	1.5	1	1	0
	1.3	1.4	1.2	1.1	0.8	0.7	1.3	1.6	0	1	1
	0.6	0.7	0.4	0.6	0.7	0.5	0.8	0.8	0	0	0
	k1	k2	k3	k4	k5	k6	k7	k8	k9	k10	k11
K	1.4	1.5	0.7	1.4	1.2	1	1.7	1.8	1	1	0
	1	1.1	1.1	1	0.8	0.7	1	1.2	0	1	1
	1.3	1.5	1.2	1.3	1.1	1	1.4	1.6	1	0	1
	1.1	1.2	1.1	0.9	1	0.7	1.1	1.3	0	0	0
	v1	v2	v3	v4	v5	v6	v7	v8	v9	v10	v11
V	1.3	1.4	1	1.2	0.7	0.8	1.4	1.6	1	0.7	0.8
	1.4	1.6	0.9	1.2	1	0.8	1.5	1.9	1.4	1	1.1
	1.4	1.6	1.2	1.3	1.2	0.9	1.5	1.7	1.5	0.9	1.3
	1.1	1.2	0.9	0.9	1.1	0.7	1.1	1.3	1.4	0.9	1.2
	1.2	1.3	0.6	1.2	1.1	0.9	1.5	1.6	1.6	1	1.2
	1.4	1.6	1	1.4	1.2	1.1	1.7	1.9	1.6	0.9	1.2

K^T					
k1	0.7	1.5	1.42	1.1	
k2	1	1.6	1.18	1.2	
k3	1	2.1	1.89	1.5	
k4	1.1	2	1.78	1.5	
k5	1.2	2.3	1.97	1.6	
k6	0.8	1.5	1.46	1.2	
k7	0.9	1.4	0.88	0.9	
k8	1.1	2.1	1.97	1.6	
k9	1	0	1	0	
k10	1	1	0	0	
k11	0	1	1	0	

D = K^T Q / sqrt(dk)											

Delta Net

write gates			
b9	b10	b11	
0.4	0.4	0.5	
Sin			
S8			
1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1
0	0	0	0
0	0	0	0

how can a model remember earlier tokens without repeatedly searching an ever-growing history

LLMs

rentention gates

a9	a10	a11
1	0.5	1
1	1	0.5
1	0.5	1
1	1	0.5

write gates

b9	b10	b11
0.4	0.4	0.5

Sin

S8

State matrix

1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1
0	0	0	0
0	0	0	0

Long Short-Term Memory

Sepp Hochreiter

Fakultät für Informatik, Technische Universität München, 80290 München, Germany

Jürgen Schmidhuber

IDSIA, Corso Elvezia 36, 6900 Lugano, Switzerland

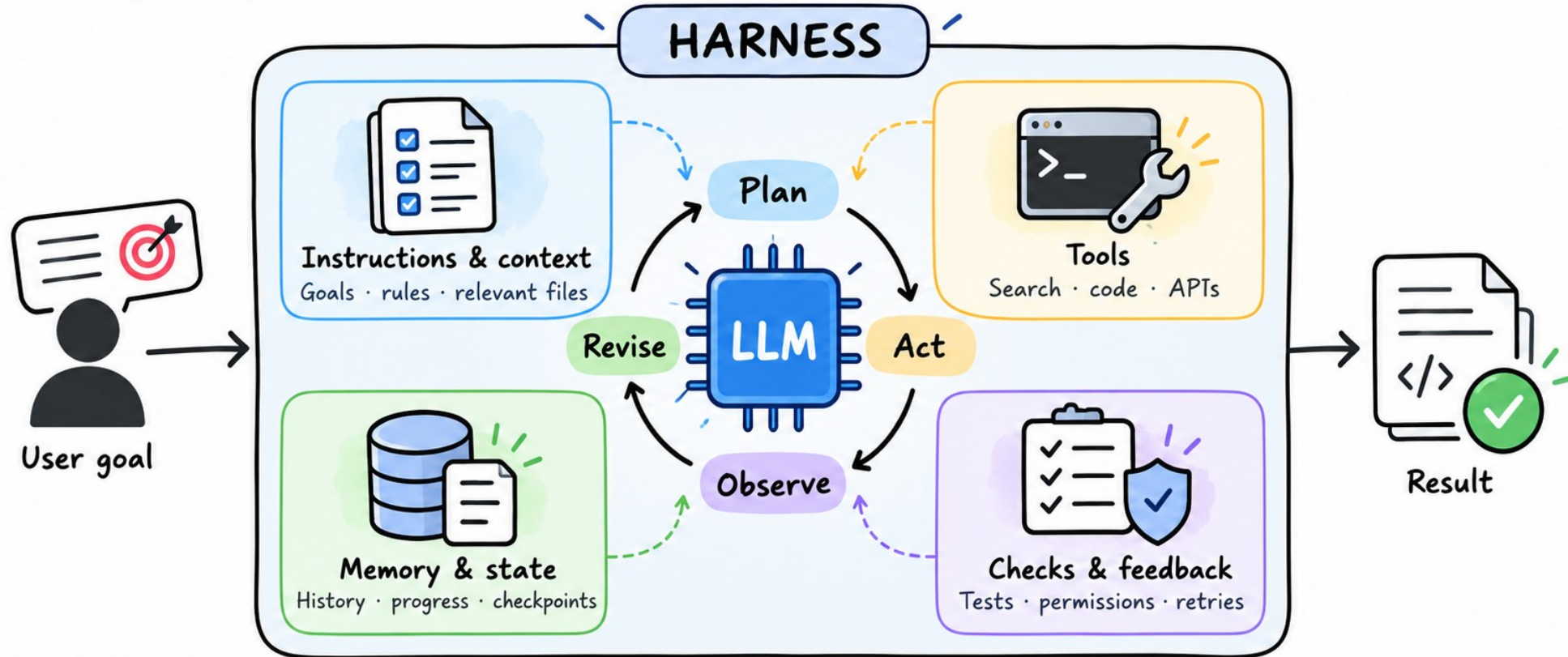
Learning to store information over extended time intervals by recurrent backpropagation takes a very long time, mostly because of insufficient, decaying error backflow. We briefly review Hochreiter's (1991) analysis of this problem, then address it by introducing a novel, efficient, gradient-based method called long short-term memory (LSTM). Truncating the

decay						predict	residual	scale		k9	1	0	1	0
S'						vs	dv=vs-v	V' = bt(dv)						
1	0	0	0			1	1	0.4			0.4	0	0.4	0
0	1	0	0			0	0	0			0	0	0	0
0	0	1	0			1	3	1.2			1.2	0	1.2	0
0	0	0	1			0	0	0			0	0	0	0
0	0	0	0			0	2	0.8			0.8	0	0.8	0
0	0	0	0			0	-2	-1			-1	0	-1	0

LLMs

Harness engineering for LLMs

Building the system around the model



Artificial Intelligence for Quantum Computing

Yuri Alexeev, Marwa H. Farag, Taylor L. Patti, Mark E. Wolf, Natalia Ares, Alán Aspuru-Guzik, Simon C. Benjamin, Zhenyu Cai, Shuxiang Cao, Christopher Chamberland, Zohim Chandani, Federico Fedele, Ikko Hamamura, Nicholas Harrigan, Jin-Sung Kim, Elica Kyoseva, Justin G. Lietz, Tom Lubowe, Alexander McCaskey, Roger G. Melko, Kouhei Nakaji, Alberto Peruzzo, Pooja Rao, Bruno Schmitt, Sam Stanwyck, [Norm M. Tubman](#), Hanrui Wang, Timothy Costa

Artificial intelligence (AI) advancements over the past few years have had an unprecedented and revolutionary impact across everyday application areas. Its significance also extends to technical challenges within science and engineering, including the nascent field of quantum computing (QC). The counterintuitive nature and high-dimensional mathematics of QC make it a prime candidate for AI's data-driven learning capabilities, and in fact, many of QC's biggest scaling challenges may ultimately rest on developments in AI. However, bringing leading techniques from AI to QC requires drawing on disparate expertise from arguably two of the most advanced and esoteric areas of computer science. Here we aim to

arXiv:2411.10406v1 [quant-ph] 15 NOV 2024

How to Build a Quantum Supercomputer: Scaling Challenges and Opportunities

Masoud Mohseni

Artur Scherer

K. Grace Johnson

masoud.mohseni@hpe.com
Hewlett Packard Labs, CA, USA

1QB Information Technologies (1QBit),
BC, Canada

Hewlett Packard Labs, CA, USA

Quantum circuit synthesis with diffusion models

Florian Furrutter, Gorka Muñoz-Gil,^{*} and Hans J. Briegel

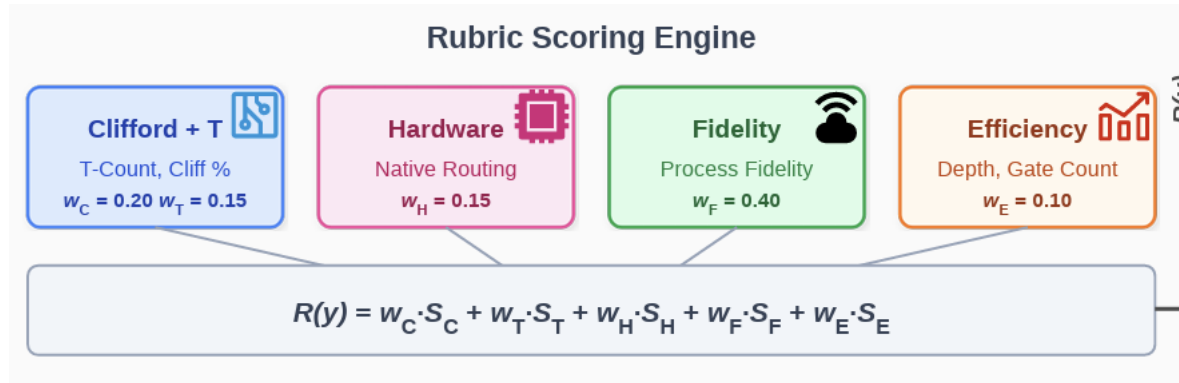
Institute for Theoretical Physics, University of Innsbruck, Technikerstr. 21a, A-6020 Innsbruck, Austria

Quantum computing has recently emerged as a transformative technology. Yet, its promised advantages rely on efficiently translating quantum operations into viable physical realizations. In this work, we use generative machine learning models, specifically denoising diffusion models (DMs).

A good way to simulate quantum circuit is to make it explainable

LLMs

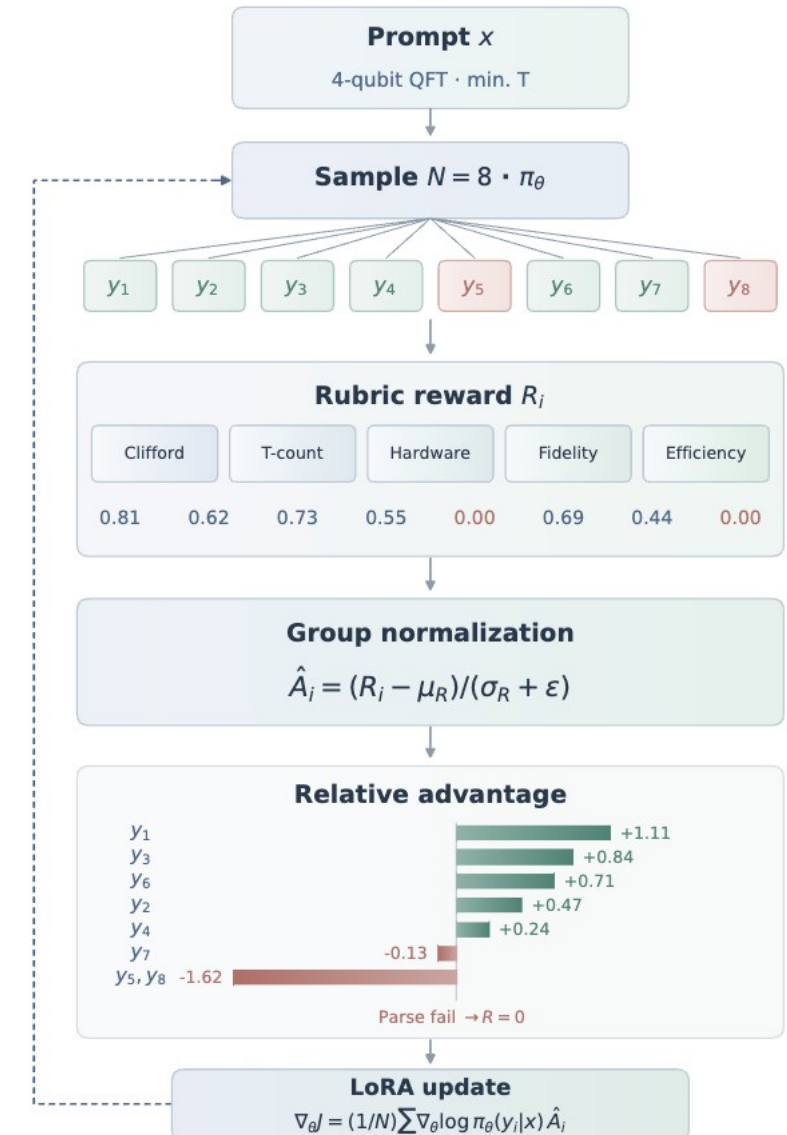
reinforcement learning with verifiable rewards (RLVR)



Clifford, T-count, hardware, fidelity, efficiency

Method	Comp. ($\uparrow$)	MSE ($10^{-3} \downarrow$)	GPU-h
<i>Dataset: UnitaryHack</i>			
Qiskit O3	1.42 ± 0.08	—	<0.01
TKET O2	1.68 ± 0.11	—	<0.01
Sparse-PPO	2.11 ± 0.12	1.12	14.6
RubriQ	3.31 ± 0.01	0.69	9.2
<i>Dataset: qsynth-bench</i>			
Qiskit O3	1.39 ± 0.09	—	<0.01
TKET O2	1.72 ± 0.13	—	<0.01
Sparse-PPO	1.98 ± 0.16	1.35	16.1
RubriQ	3.32 ± 0.01	0.67	9.2
<i>Dataset: QData</i>			
Qiskit O3	1.35 ± 0.07	—	<0.01
TKET O2	1.61 ± 0.10	—	<0.01
Sparse-PPO	2.05 ± 0.14	1.40	17.3
RubriQ	3.30 ± 0.02	0.65	9.2

Comp is T-gate compression (C) as defined in evaluation metrics.

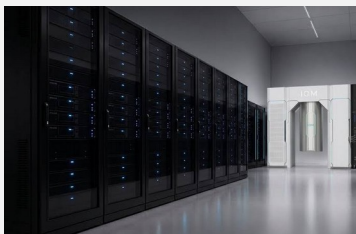


Applications & Research



01 · RQ1 · Scale

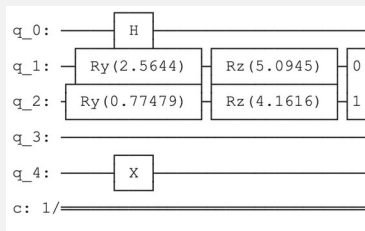
Simulation



GPU / QPU
simulation beyond
hardware reach

02 · RQ2 · QIP

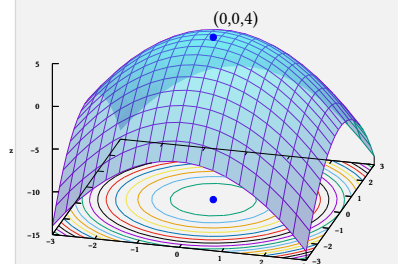
Algorithms



Structure from
problem and device,
not black-box search

03 · RQ4 · Synthesis

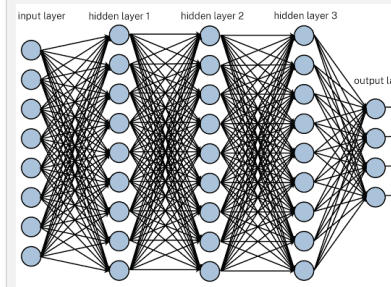
Optimization



AI/HPC synthesis
within fault-tolerant
budgets

04 · RQ3 · LLM

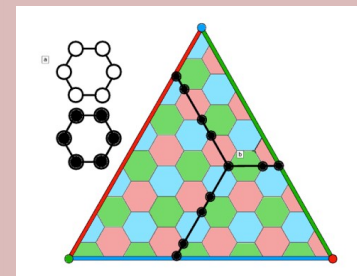
LLMs



AI for quantum

05 · RQ5 · Trust

QEC



Auditable QEC and
post-quantum
cryptography

QEC

Advances in QEC Experiments

Alexandre Bourassa · (2025)

Google Scholar

Copy Bibtex

Abstract

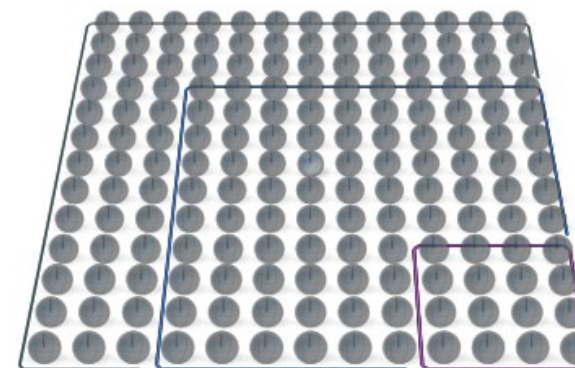
Quantum error correction (QEC) is critical for achieving useful quantum computers, since it allows us to combine many noisy physical qubits into one high-quality logical qubit with exponentially decreasing logical error rate. In this talk, we will discuss Google's latest error correction results [1] where we achieved below threshold surface code performance with logical qubits at distances=(3, 5, 7). In a logical memory demonstration, we show that each increases in distance reduces errors by a factor of 2.14. Additionally, we report the ability to decode these QEC experiments in real-time for up to 1 million rounds. Finally, we present a 10,000x reduction in the rare correlated errors by measuring the repetition code in the very low error regime. Ultimately, our results show device performance that, if scaled, could realize the operational requirements of large scale fault-tolerant quantum algorithms.

Scalable logical qubits

Matthias Troyer, Microsoft Quantum
Chetan Nayak, Microsoft Quantum
John Martinis, QOLAB

Abstract

Utility-scale quantum computing will require executing long, complex algorithms with end-to-end error rates far below what physical qubits can support directly. Error-corrected logical qubits are needed to achieve this goal. To make the progress on logical qubits measurable and comparable we introduce the definition of *scalable logical qubits*: logical qubits preserved for long computations by repeated quantum error correction, capable of fault-tolerant universal operations with low-latency real-time decoding and feedback, and replicable to the hundreds or thousands, as required by applications. We characterize these scalable logical qubits along four coupled dimensions: reliability, scale, capability, and performance, and discuss trade-offs among



Quantum error correction below the surface code threshold

Google Quantum AI and Collaborators
(Dated: August 27, 2024)

Quantum error correction [1–4] provides a path to reach practical quantum computing by combining multiple physical qubits into a logical qubit, where the logical error rate is suppressed exponentially as more qubits are added. However, this exponential suppression only occurs if the physical error rate is below a critical threshold. In this work, we present two surface code memories operating below this threshold: a distance-7 code and a distance-5 code integrated with a real-time decoder. The logical error rate of our larger quantum memory is suppressed by a factor of $\Lambda = 2.14 \pm 0.02$ when increasing the code distance by two, culminating in a 101-qubit distance-7 code with $0.143\% \pm 0.003\%$ error per cycle of error correction. This logical memory is also beyond break-even, exceeding its best physical qubit's lifetime by a factor of 2.4 ± 0.3 . We maintain below-threshold performance when decoding in real time, achieving an average decoder latency of 63 μs at distance-5 up to a million cycles, with a cycle time of 1.1 μs . To probe the limits of our error-correction performance, we

QEC experiments in real-time for up to 1 million rounds.

QEC

Gates / Layers:

- Standard Clifford Randomized Benchmarking [[Phys. Rev. A 85, 042311](#) (2012)]
- Interleaved Randomized Benchmarking [[Phys. Rev. Lett. 109, 080505](#) (2012)]
- Compressive Gate Set Tomography [[PRX Quantum 4, 010325](#) (2023)]
(Optional dependencies required)
- Mirror Randomized Benchmarking [[Phys. Rev. Lett. 129, 150502](#) (2022)]
- Error Per Layered Gate [[arXiv:2311.05933 \[quant-ph\]](#) (2023)]

Holistic:

- Quantum Volume [[Phys. Rev. A 100, 032328](#) (2019)]
- CLOPS [[arXiv:2110.14108 \[quant-ph\]](#) (2021)]

Entanglement:

- GHZ State Fidelity [[arXiv:0712.0921 \[quant-ph\]](#) (2007)]
- Graph State Bipartite Entanglement [[Adv. Quantum Technol., 2100061](#) (2021)]

Optimization:

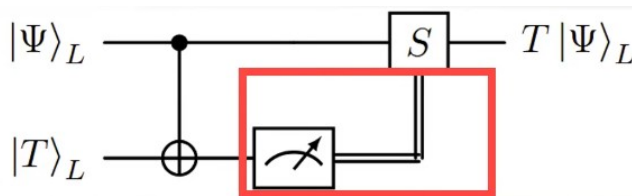
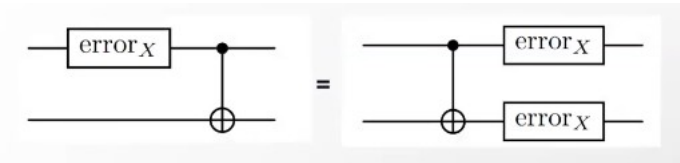
- Q-Score [[IEEE Trans. Quantum Eng., 2](#) (2021)]

QEC

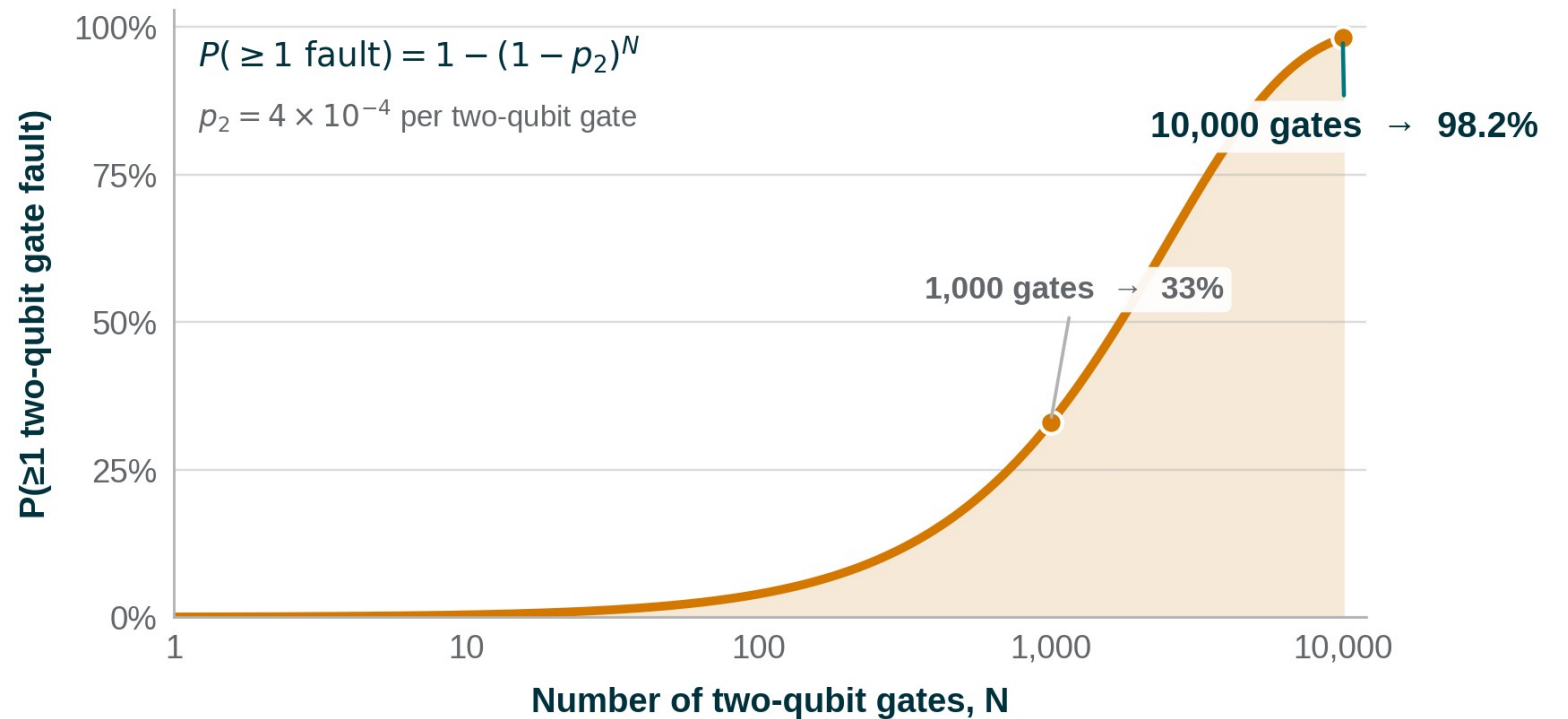
Why quantum error correction (QEC) is necessary

Long computations require reliability beyond the fidelity of any single physical operation.

QEC lets a computation continue through faults instead of requiring a fault-free run.



<https://www.nature.com/articles/s41586-025-09848-5/figures/1>

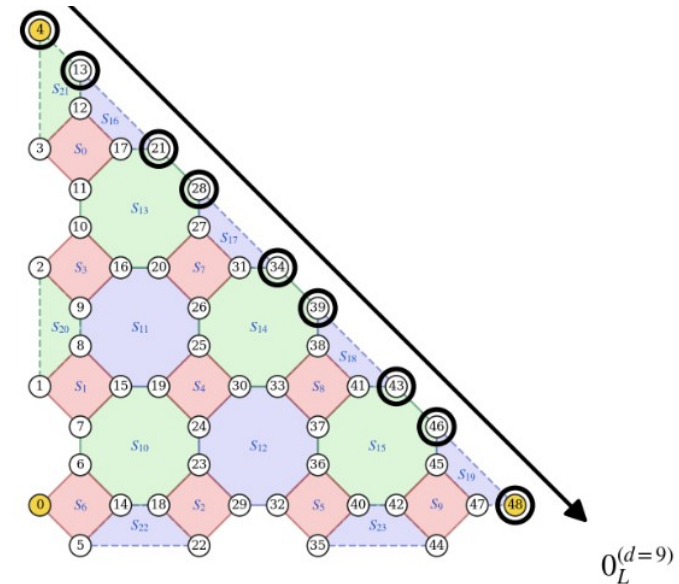
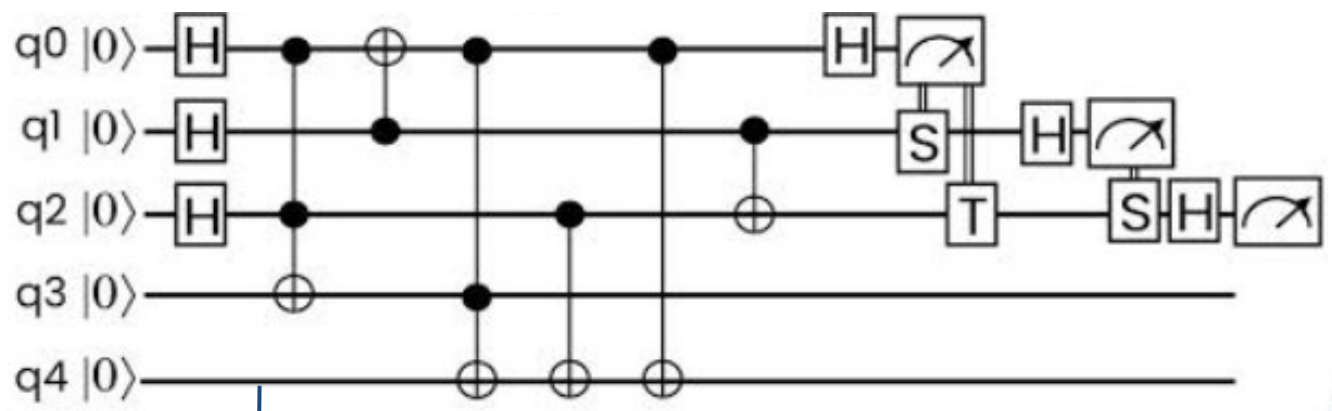


Independent-fault model: $P(\geq 1 \text{ fault}) = 1 - (1 - p_2)^N$; $p_2 = 4 \times 10^{-4}$.

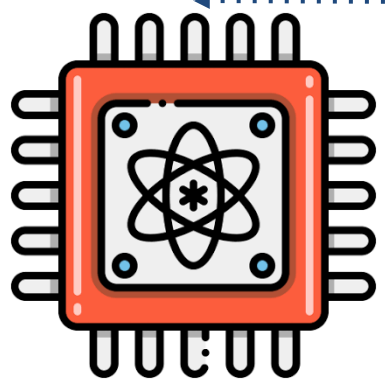
Visual reference: Nature 656, 47–53 (2026), Extended Data Fig. 1.

QEC

Fault-tolerant quantum computations



real time QEC ?

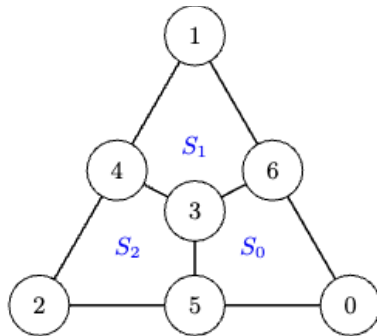
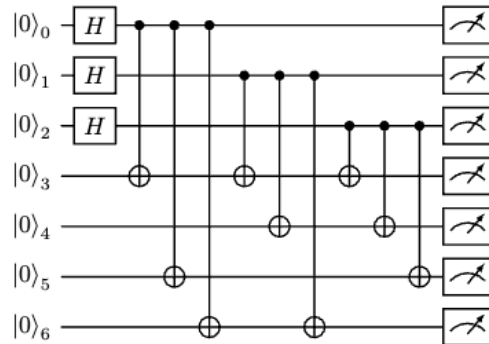


FPGA/ASIC

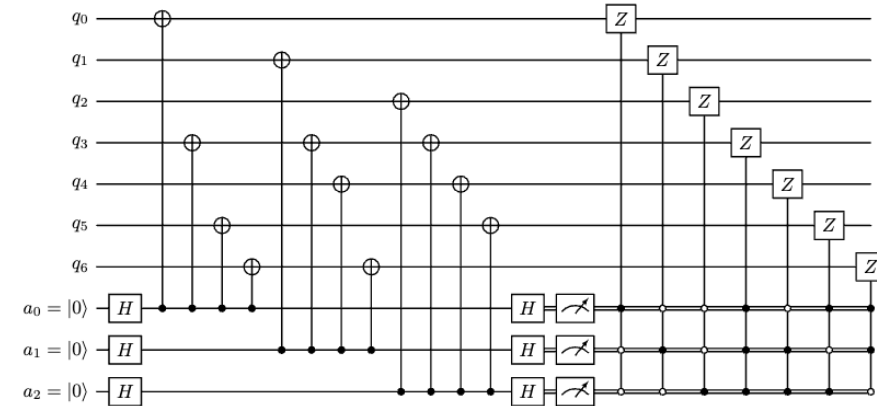
QEC

Encoder-> Stabilizer -> Decoder

Example: 4.8.8 $[[7, 1, 3]]$ color code (steane)



$$|\bar{0}\rangle = \frac{1}{\sqrt{8}} \left(|0000000\rangle + |1010101\rangle + |0110011\rangle + |1100110\rangle \right. \\ \left. + |0001111\rangle + |1011010\rangle + |0111100\rangle + |1101001\rangle \right) \\ |\bar{1}\rangle = \frac{1}{\sqrt{8}} \left(|1111111\rangle + |0101010\rangle + |1001100\rangle + |0011001\rangle \right. \\ \left. + |1110000\rangle + |0100101\rangle + |1000011\rangle + |0010110\rangle \right).$$



Readout bit strings

$$\mathbf{m} = (m_0, m_1, m_2, m_3, m_4, m_5, m_6) \in \mathbb{F}_2^7.$$

$$H \equiv H_7 = \begin{array}{c|ccccccc} & q_0 & q_1 & q_2 & q_3 & q_4 & q_5 & q_6 \\ \hline S_0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ S_1 & 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ S_2 & 0 & 0 & 1 & 1 & 1 & 1 & 0 \end{array}.$$

Raw decoder

$$\ell_{\text{raw}} = \lambda \cdot \mathbf{m} = m_0 \oplus m_1 \oplus m_6, \quad \lambda = (1, 1, 0, 0, 0, 0, 1).$$

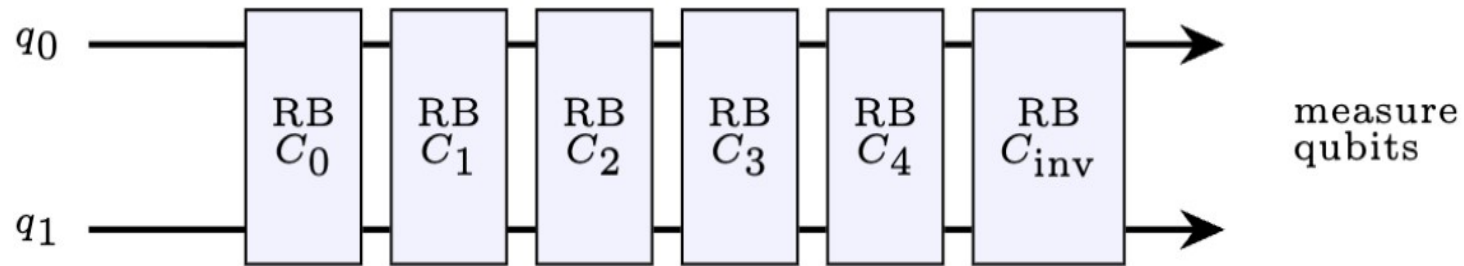
distance	5	7	9	11	13	15
data qubits	17	31	49	71	97	127
num of corr	2	3	4	5	6	7

QEC

Randomized Benchmarking (RB) estimates fidelity

A random Clifford group followed by its exact inverse has a known ideal answer.

<https://arxiv.org/pdf/2408.12064>



One-qubit Clifford group: 24 combinations
Two-qubit : 11,520 combinations
DRBG

The 4 Pauli Gates: I, X, Y, Z

The Basic Generators: $H, S, S^\dagger$ (where $S^\dagger$ is essentially $Z \cdot S$)

Two-Gate Combinations: $H \cdot S, S \cdot H, H \cdot X, H \cdot Y$, etc.

$|00\rangle_L$

logical error rate (LER)
 $= 1 - F_L$
err: $|01\rangle_L, |10\rangle_L, |11\rangle_L$

100 RB **sequences** $\times$ 1,000 **shots**
 $=$ 100,000 shots per RB

01 **Encode**

two $|0\rangle_L$ blocks

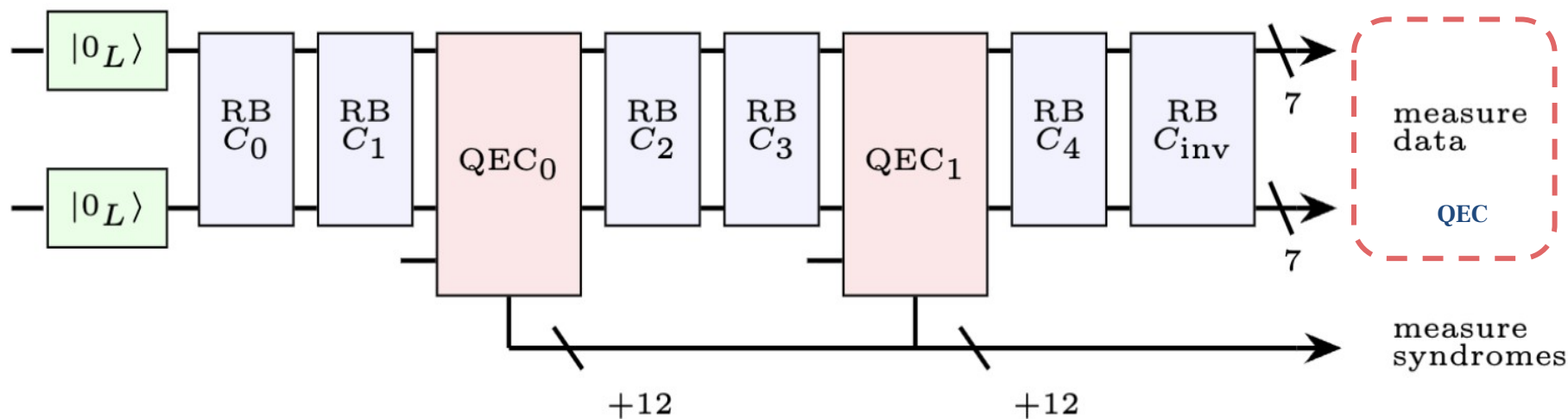
02 **Compute**

m random Cliffords

QEC

Two logical blocks run the same random workload with full X+Z QEC cycles inserted.

2 Logical 0L with QEC rounds = 2, RB rounds = 4



In real RB, we choose 240 length because it has many divisors 2, 3, 4, 5, 6, 8, 10, 12, ...

With $q = R + 1$ equal circuit segments, q divides m and each segment contains exactly m/q random Cliffords.

Pauli frame tracking in post processing

Noise model: $p_1 = 10^{-4}$; $p_2 = 4 \times 10^{-4}$; ideal reset and readout.

03 **Correct**

R full QEC cycles

04 **Decode**

data + syndrome history

QEC

Lookup table for decode?

N = detector bits in the run. One QEC round of a 4.8.8 color code emits n – 1 of them, one per stabilizer generator.

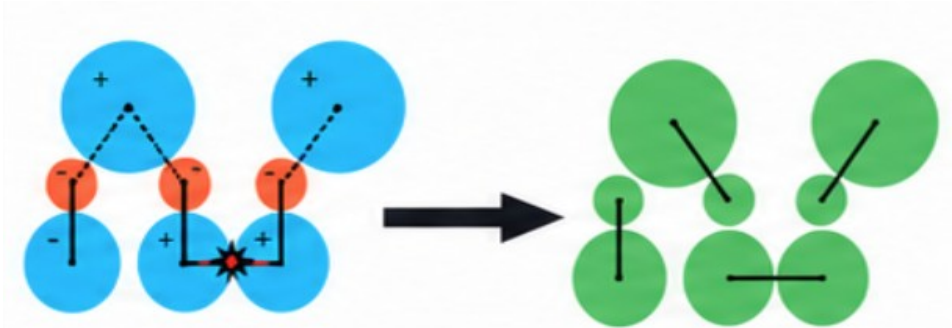
code	detector bits, 1 round	table rows
[[17,1,5]] d = 5	16	$2^{16} = 6.6 \times 10^4$
[[49,1,9]] d = 9	48	$2^{48} = 2.8 \times 10^{14}$
[[127,1,15]] d = 15	126	$2^{126} = 8.5 \times 10^{37}$

Qubits 1+2	Qubits 2+3	Qubits 3+4	How to fix the error
0	0	0	Do nothing
1	0	0	Flip qubit 1
1	1	0	Flip 2
0	1	0	Flip 1+2
0	0	1	Flip 4
1	0	1	Flip 2+3
1	1	1	Flip 1+3
0	1	1	Flip 3

d=15, 2 logical -> 252 detector bits per round of QEC
R = 19: 252 x 19 = 4788 bits per shot
 $2^{4788} \approx 10^{1441}$ RAM



Chromobius decoder (used in this work)

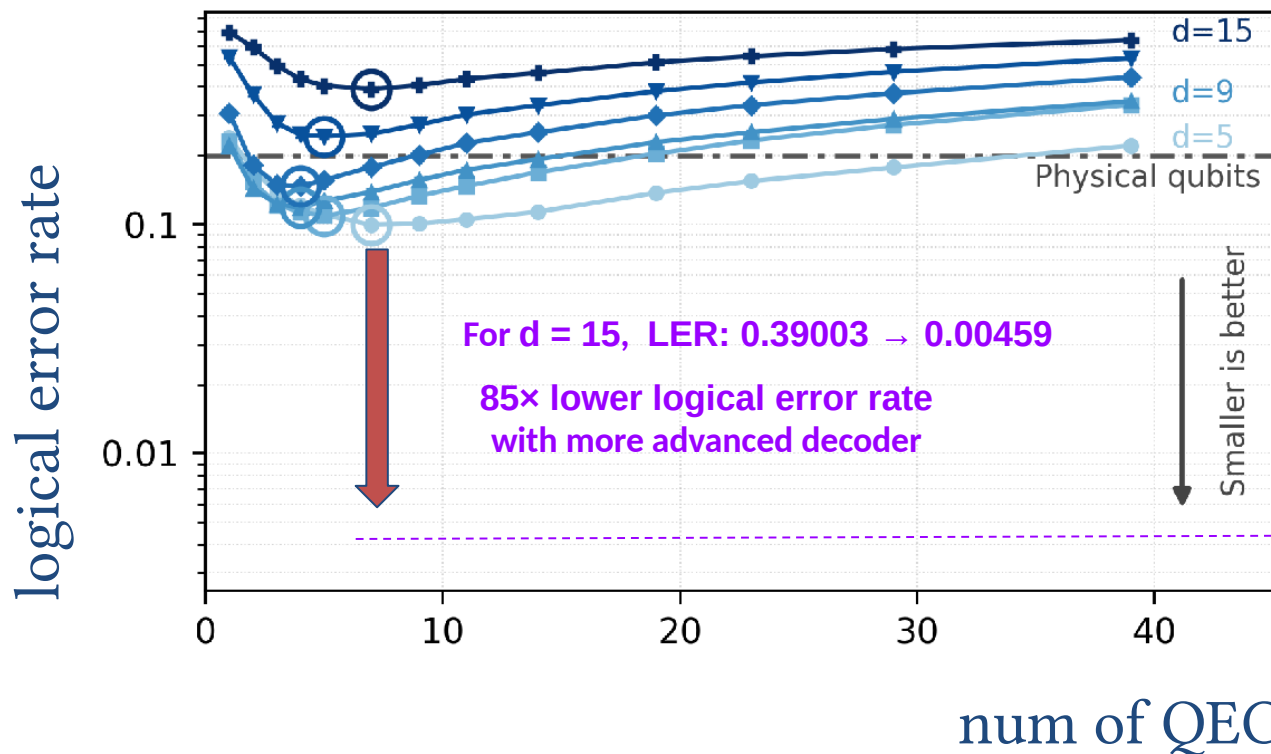


QEC

Logical error versus number of QEC rounds

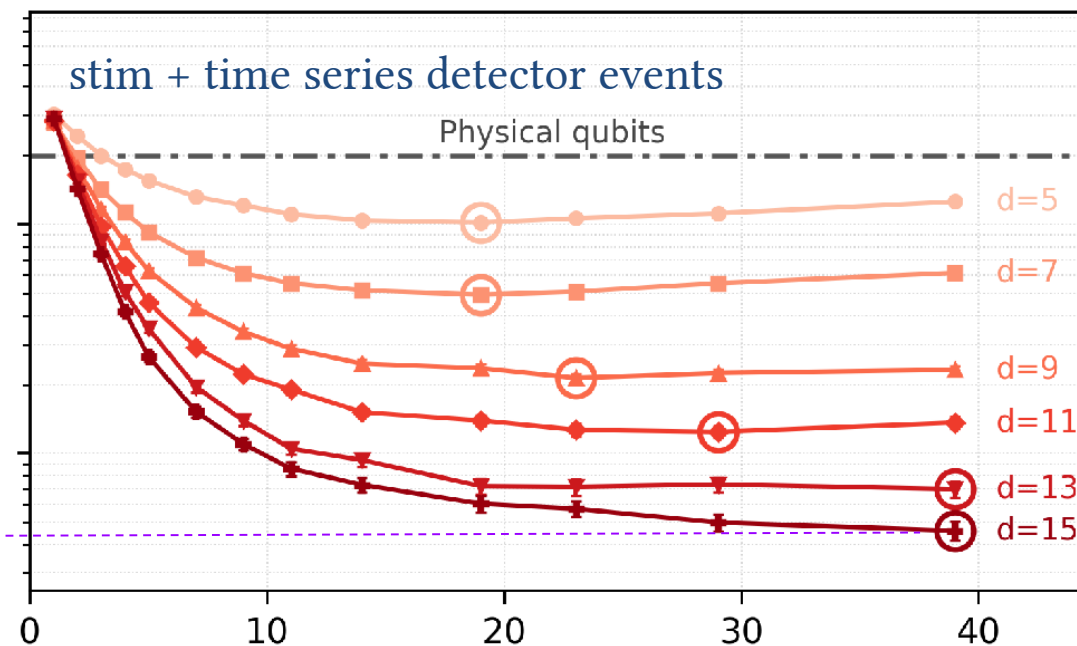
Naive MWPM decoder

(c) RB 240 — MWPM



Advanced Chrombious decoder

(d) RB 240 — Chromobius



Circles mark the minimum LER at each distance.

Chromobius minima decrease from $d = 5$ to 15.

runtime and scaling

SAME RB-240 WORKLOAD (same podman image)

platform	workers	wall time
Laptop	8	49h m*
Perlmutter CPU	100	4h

*Laptop value is projected from Perlmutter worker scaling; it was not measured on a laptop.

MEASURED PERLMUTTER JOB

4.8M shared shots · RB = 240 · d = 11, 13, 15
 · 16 values of R · two decoders · ≈390 worker-hours
 390/100 ≈ 4h

WHAT EACH FACTOR DOES TO WALL TIME

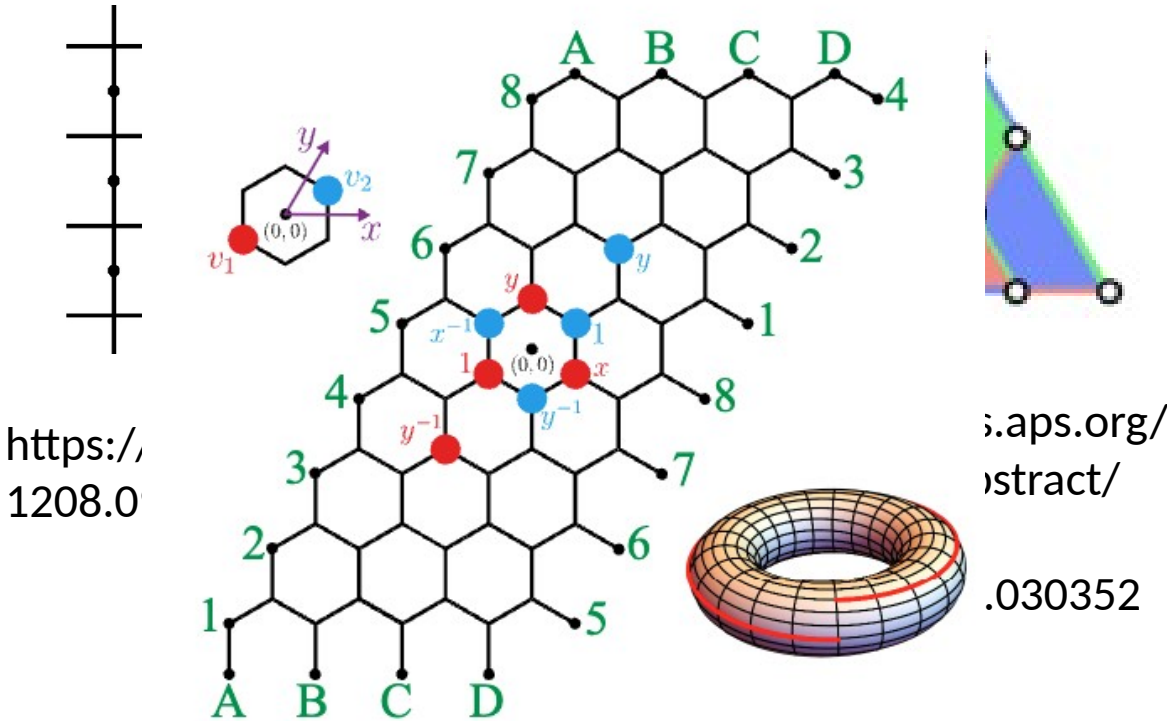
factor	range in this study	exact count that grows	effect on wall time
shots per RB point	25k → 100k	independent samples per point: 4×	linear — 4× shots, 4× time
workers	8 → 100	the same shots split across workers	inverse-linear because shots are independent
RB depth m	120 → 480	noisy 1q + CNOT gate slots per shot: 4× more	linear in m
QEC rounds R	1 → 19	QEC gate slots + detector bits: 126 R per block at d = 15	linear in R
code distance d	5 → 15	$n = (d^2 + 2d - 1)/2$: 17 → 127 qubits, 16 → 126 checks	gate slots grow with n, so quadratic in d
noise strength p1, p2	10 ⁻⁴ , 4 × 10 ⁻⁴	nothing — gate and detector counts are unchanged	negligible
decoder-model build	once per (d, R)	one compile per configuration	negligible against 100k shots

Every count above is exact. The shots and workers rows are linear by construction because shots are independent; the wall-time slopes for m, R and d were not timed factor by factor. CNOT = controlled-NOT gate. We also note that RB circuits we use HPC to **pregenerate** all.

QEC

Self-dual Calderbank–Shor–Steane (CSS) quantum low-density parity-check (QLDPC) block with $[[n, k \geq 2, d]]$. we want high rate: $[7,1,3] \ 1/7 \rightarrow [16,4,4] \rightarrow 1/4$

logical qubits	code distance	
k	d	Code
2	$\sqrt{n}$	Kitaev toric
$\Theta(n)$	$\Theta(\log n)$	2D hyperbolic surface
$\Theta(n)$	$\Omega(n^{1/10})$	Guth-Lubotzky
2	$\Omega(\sqrt{n \log n})$	Freedman-Meyer-Luo
$\Theta(n)$	$\Theta(\sqrt{n})$	hypergraph product
$\Theta(\sqrt{n} / \log n)$	$\Omega(\sqrt{n} \log n)$	high-dimensional expander (HDX)
$\Theta(\sqrt{n})$	$\Omega(\sqrt{n} \log^c n)$	tensor-product HDX
$\Theta(n^{3/5} / \text{polylog}(n))$	$\Omega(n^{3/5} / \text{polylog}(n))$	fiber-bundle
$\Theta(\log n)$	$\Omega(n / \log n)$	lifted-product (LP)
$\Theta(n^{4/5})$	$\Omega(n^{3/5})$	balanced product (BP)
$\Theta(n)$	$\Theta(n)$	expander LP
$\Theta(n)$	$\Theta(n)$	quantum Tanner
$\Theta(n)$	$\Theta(n)$	Dinur-Hsieh-Lin-Vidick



Quantum Entropy

Certified Randomness from Quantum Supremacy

Scott Aaronson*

Shih-Han Hung†

Abstract

We propose an application for near-term quantum devices: namely, generating cryptographically certified random bits, to use (for example) in proof-of-stake cryptocurrencies. Our protocol repurposes the existing “quantum supremacy” experiments, based on random circuit sampling, that Google and USTC have successfully carried out starting in 2019. We show that, whenever the outputs of these experiments pass the now-standard Linear Cross-Entropy Benchmark (LXEB), under plausible hardness assumptions they necessarily contain $\Omega(n)$ min-entropy, where

Quantum computational advantage in random-circuit sampling on IBM superconducting quantum computers

Tigran Sedrakyan,¹ Yuxuan Zhang,^{1,2} Hovnatan Karapetyan,¹
Joshua D. Baktay,^{1,3} Hrant Gharibyan,¹ and Hayk Tepanyan¹

¹BlueQubit, San Francisco, CA 94105, USA

²Institute of Physics, École Polytechnique Fédérale de Lausanne, Lausanne, Switzerland

³XPRIZE Foundation, Culver City, CA 90230, USA

(Dated: September 25, 2026)

We report forward random-circuit sampling (RCS) on the 120-qubit Nighthawk r2 superconducting processor (*ibm.phoenix*) with square-lattice connectivity, using 61 qubits, native CZ gates, and the standard cloud execution stack with no benchmark-specific calibration. Two independent fidelity estimators—mirror benchmarking and three- and four-patch cross-entropy benchmarking (XEB)—agree with each other at every measured depth, the mirror from 4 to 40 cycles and the patched estimators from 20 to 40 cycles, across more than two orders of magnitude of fidelity decay, and exceed the first-generation Nighthawk r1 device by more than an order of magnitude at fixed depth. The 36-cycle circuits sit at the depth where tensor-network contraction cost saturates at system size; a contraction cost estimator validated against the published Supremacy and Zuchowzki

ACCESS BY UNIVERSITY LIBRARIES | VIRGINIA TECH

Zeno Effect for Quantum Computation and Control

[Gerardo A. Paz-Silva](#)^{1,4}, [A. T. Rezakhani](#)^{1,4,5}, [Jason M. Dominy](#)^{1,4}, and [D. A. Lidar](#)^{1,2,3,4}

Show more ▾

Synthetic Data



Gretel 

To train a LLM what considers as the most significant important

QEC

Recently NIST update the rules **FIPS140-3**

RSA / EC - > ML-KEM / ML-DSA

FIPS 140-3

SP-800-80 A/B/C

However,
Entropy comes from GCP Numpy
Permutation of the classical RNG

CURBy - CU Randomness Beacon

The University of Colorado randomness beacon (CURBy) is a public randomness service that regularly broadcasts a set of random bits. It is composed of a classical periodic source of randomness as well as a quantum source of randomness based on the unpredictable [correlations arising from measuring distant quantum particles](#). CURBy is tied together through a network of distributed blockchains we call [Twine](#) that allows the random outputs to be fully traced, audited, and certified.

You can browse the past and present random values below:

Classical	Quantum
Viewing Pulse ID: b.....idxd5ve.i Mixing in with 5 other chains	
72e127c2a7c0e04cf609b55e2d1c45e6115a22c92a4e5b1fa5389934c12f3476 b15d0c5a47ba2be1086c1b85695f69b210043c414e1afe330ee79ba0771f6a44	
first	<< < 1538056 > >> last
released: 17:41:00 on Monday, September 28, 2026 pulse: 1538056 / 1538056	

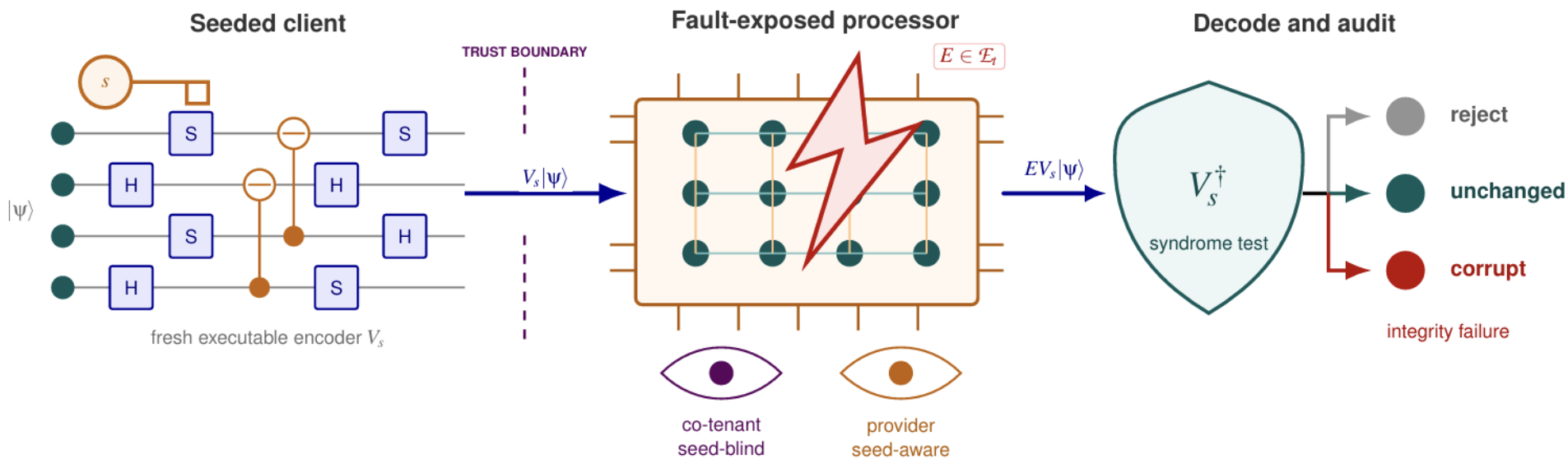
Outshift Quantum Random Number Generator (QRNG)

Quantum randomness is nature's ultimate wildcard that taps into the inherent unpredictability of nature at a subatomic level—no patterns, no biases, just pure, unrepeatable randomness. This makes it an essential building block for ultra-secure encryption, reliable simulations, and cutting-edge innovations. Whether you're

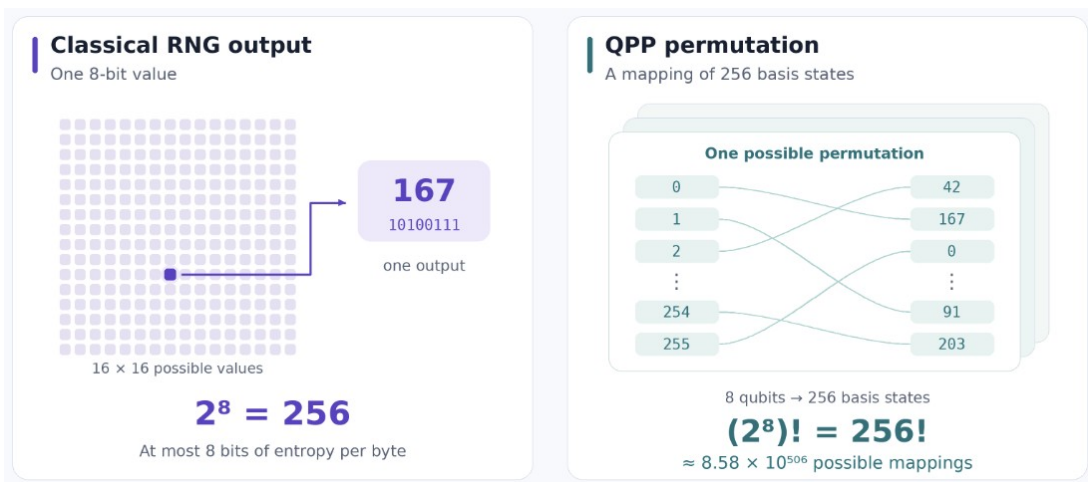


QEC

The circuit was submitted to the cloud based on the fixed structure.



AES-256



$$256! \approx 2^{1684}$$

Summary



- Introduce a large scale quantum **simulation** paradigm
- Showed that we could use quantum information theory to approximate nonlinearity and solve the **arithmetic** problems more efficiently in terms of quantum encoding space. We emphasize that $BQP \neq NP$
- Our proposed Large **Language Model framework** enables a variety of applications such quantum circuit synthesis, randomize circuit benchmarking, solving polynomial roots.
- Entropy enhanced quantum error correction encoder allows an unpredictable error propagation, which increase the cloud security.

ÇÇ

Outlook

- With my collaborators, I have introduced multiple quantum simulation and tools on github that are public accessible; feel free to join and use.

To that end: <https://github.com/gzquse/qgear>

- Richard Hamming said working on something meaningful (it might be hard) but solvable problems. Such we cannot teleport people however, we could use teleportation mechanism for quantum error correction as well as key distribution.
- Hard / interesting but solvable problems:
 - Nonlinearity solver
 - Quantum Synthetic Data
 - Running high dimension system at the scale
 - Exploit quantum algorithm entanglement properties

Acknowledgement



- Professor Pan (Research Advisor)
- Committee Members: Professor Lu, Professor Mengel, and Professor Zhan.
- NERSC Quantum Members: Jan Balewski, Daan, Kaitie, Ding, and etc.
- UMD Collaborators Alex Khan, Prof. Jabeen, Frenz (Quantum Lab)
- NVIDIA folks: Zohim, Dr. Nguyen, Jin, Sam
- Industrial Collaborators: Randy Kuang, Anthony Lawrence
- Funding and Support: NERSC and AWS Research Fellowship
- Family / Friends

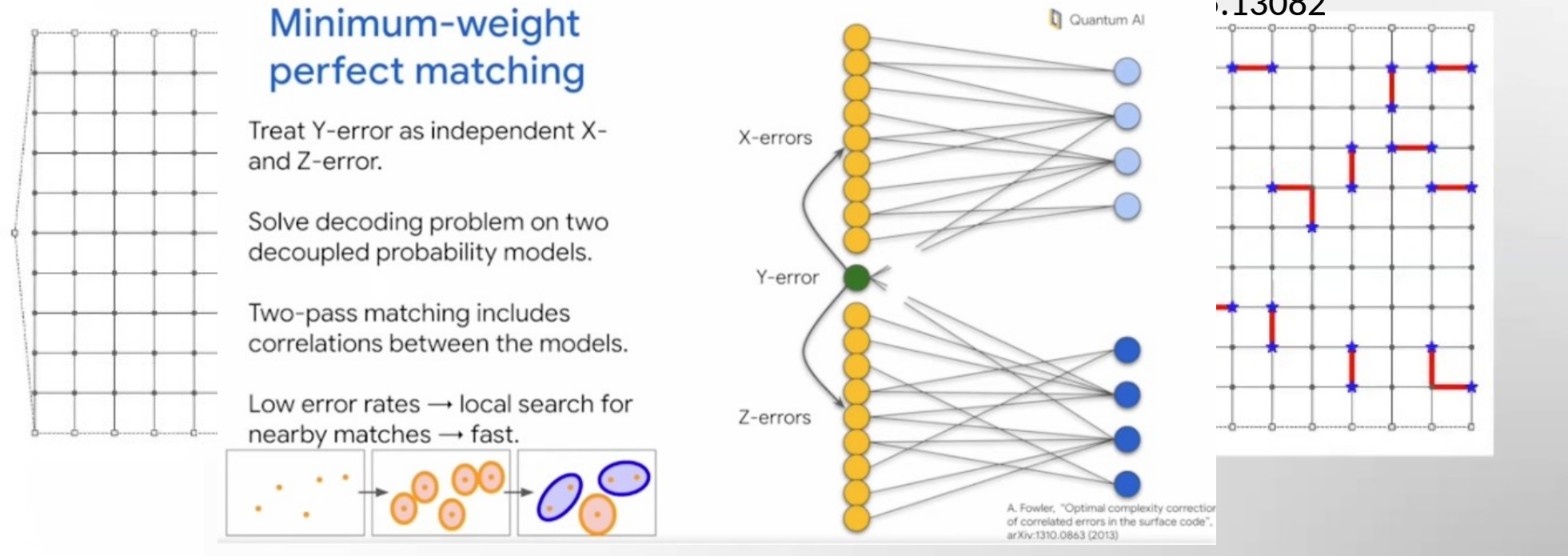


Thank you

Questions?

Ziqing Guo · ziqguo@ttu.edu · github.com/gzquse · ORCID 0009-0006-3999-2221

MWMP



<https://arxiv.org/abs/2105.13082>

<https://ui.adsabs.harvard.edu/abs/2019PhRvB..99i4103P/abstract>

belief propagation

Belief-matching

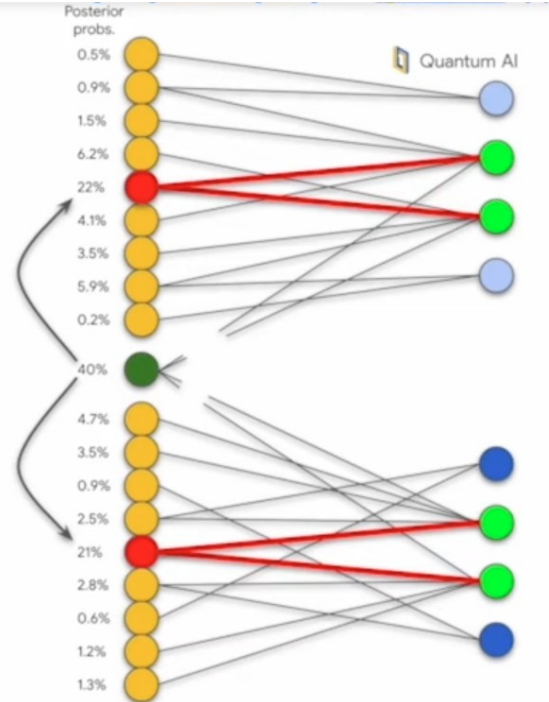
First - run belief propagation passing messages between error mechanisms and detectors.

Updated probabilities are often 'inconsistent' with the detection events due to

- split beliefs
- short loops

Second - post-process with MWPM.

BP parallelizable, fast, but 'touches' the entire probability model.



1. Data qubits say to ancillas: "I believe I am in error with probability p ."
2. Ancillas say to data qubits: "Given my syndrome and what I'm hearing, p is too high/too low."
3. Data qubits update belief based on ancillas' messages.
4. Iterate until converged.

The MWPM algorithm finds a subset of errors $M \subseteq E$ (a matching) that satisfies two conditions:

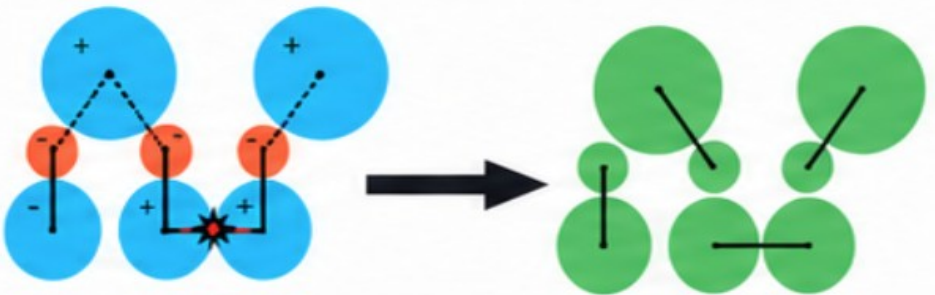
- Step 2: Edge Decomposition
- Step 3: Standard MWPM and Projection

The decoder runs standard MWPM on the lifted set V' to find a matching $M' \subseteq E'$. Finally, an inverse projection maps the matched pairs back to the original 3-element hyperedges to identify the physical errors:

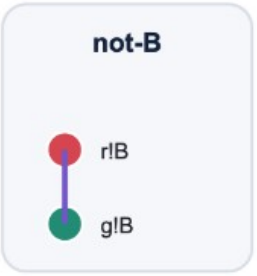
$$M_{final} = f^{-1}(M')$$

detects in V' .

MWPM



A better way to decode use mobius



A public encoder is a reusable target

shared cloud QPU

bounded cloud fault model: weight-one Pauli faults, all $3n$ locations enumerated

your job

$[[n, k, d]]$ encoder E

$|\psi\rangle \rightarrow E|\psi\rangle|0\dots 0\rangle$

syndrome check $\rightarrow$ accept / reject
logical qubit

co-tenant adversary

same chip, same time

injects one Pauli fault

learns the public encoder,
then chooses where to strike

fixed public encoder

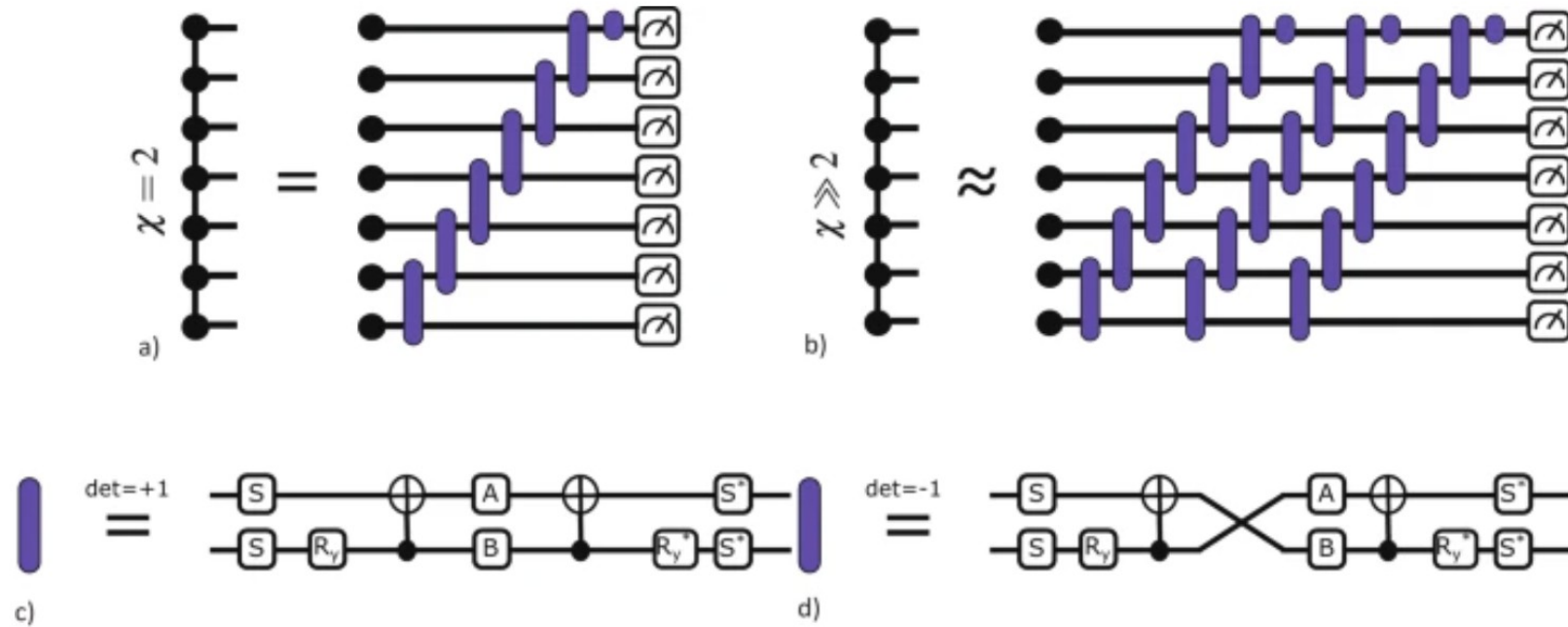
same E every run $\rightarrow$ reusable target
adversary picks the fault after learning E

reseeded encoder

fresh seed per run $\rightarrow$ fault map changes
fault must be fixed before the seed is known

<https://www.nature.com/articles/s41534-024-00805-0#Fig1>

Fig. 1: The circuit structure for preparing an MPS wave function.



<https://arxiv.org/abs/quant-ph/9811052>

The toric code, memory only.

<https://arxiv.org/abs/quant-ph/0110143>

Computation using a stack of tori with transversal interactions.

<https://arxiv.org/abs/quant-ph/0510135>

Computation using a 3D cluster state, only local interactions in 3D.

<https://arxiv.org/abs/quant-ph/0610082>

<https://arxiv.org/abs/quant-ph/0703143>

Building the 3D cluster state layer by layer so only a bi-layer of locally connected 2D hardware is required.

<https://arxiv.org/abs/0803.0272>

A version of the 2D scheme cut down to a single nearest-neighbor coupled 2D layer of hardware. At this point everything was based on the 2D scheme.

<https://arxiv.org/abs/0805.3202>

A simplified version of the 3D cluster state scheme (I found the original papers hard to understand).

<https://arxiv.org/abs/1009.3686>

<https://arxiv.org/abs/1110.5133>

Beginning to learn how to decode the surface code better.

<https://arxiv.org/abs/1111.4022>

Original proposal for lattice surgery on an unrotated surface code.

<https://arxiv.org/abs/1206.0800>

Proof that high levels of error can be suppressed exponentially using the surface code.

<https://arxiv.org/abs/1208.0928>



Hardware used in this dissertation

DEVICE	TYPE	QUBITS	CALIBRATION	FIGURE REPORTED	USED IN
IBM Torino	Heron-1, superconducting	133	—		DEAL (Ch. IV)
IBM Marrakesh	Heron-2, superconducting	156	—		DEAL (Ch. IV), ShardQ (Ch. V)
IBM Pittsburgh	Heron-3, superconducting	156	1.46×10^{-3}	median CZ error	QAWA (Ch. IV), ShardQ (Ch. V)
IBM Boston	Heron-3, superconducting	156	—		NNQA (Ch. V)
IBM Kingston	Heron, superconducting	—	—		VQT (Ch. V), NNQA
IBM Miami	Nighthawk, superconducting	—	—		NNQA (Ch. V)
IonQ Aria-1	trapped ion	—	—		QPIE, VQT (Ch. V)
IonQ Forte / Forte-1	trapped ion	36	—		NNQA, RubriQ validation (Ch. V, VI)
Amazon Braket SV1	state-vector simulator	—	—		QPIE (Ch. V)
NERSC Perlmutter	A100 40 GB / 80 GB nodes	—	—		Q-GEAR, ShardQ, NNQA training, RubriQ (Ch. III, V, VI)

Per-device T1/T2 and error tables are in the individual papers; dashes mark values not reproduced here.



Q-GEAR gate mapping

QISKIT GATE	PARAMETERS	CUDA-Q KERNEL OPERATION
h	—	<code>h(q)</code>
ry	θ	<code>ry(θ, q)</code>
rz	φ	<code>rz(φ, q)</code>
u3	θ, φ, λ	<code>u3(θ, φ, λ, q)</code>
cx	—	<code>x.ctrl(c, t)</code>
cp	λ	<code>r1.ctrl(λ, c, t)</code>
swap	—	<code>swap(a, b)</code>
measure	—	<code>mz(q)</code>

ENCODING

Each gate becomes one row across the three tensors: gate id, qubit indices (control, target), and parameters. Unused parameter slots are left empty.

STORAGE

Tensors are written to HDF5 with the circuit metadata (qubit count, shots, seed) so a run is reproducible on CPU, one GPU or 1,024 GPUs.

NOTE

Operation names follow the CUDA-Q Python API; the exact emitted code is in the qgear repository.

DEAL, in pseudo-code

```
# Direct Entanglement Ansatz Learning (DEAL) – sketch
input  QUBO matrix Q (n = 20), device coupling map, CZ budget ≈ 41–43
output bitstring x*, energy E*

y, β ← direct_map(Q)           # angles from Qii, Qij; no optimizer
p ← bayes_select_layers(Q, budget) # 7 layers ≈ QAOA at 9
for λ in noise_scales:         # adaptive digital ZNE
    C ← entangling_ansatz(y, β, p, coupling_map) # echoed cross-resonance
    C ← insert_delays(C, λ)           # crosstalk / coherence control
    E[λ] ← run(C, shots)
E* ← extrapolate(E, λ → 0)
x* ← argmin_sample(C)
QNRE ← |E* - E_exact| / noise_limit      # reported metric
```

INSTANCES

Travelling salesman, knapsack and MaxCut; Table II optima 8.795 / 7.468, 39.167 and 14.184.

VALIDATION

20-qubit circuits were simulated with the Q-GEAR GPU path before submission to IBM Torino and Marrakesh.

CODE

github.com/gzquse/DEAL_QUBO

Reproducibility and artifacts

WORK	REPOSITORY / ARCHIVE	CONTENTS
Q-GEAR	github.com/gzquse/qgear · gzquse/cudaq-perlmutter	nbdev package, Apache-2.0, 241 commits; multi-node CUDA-Q scripts for Perlmutter
QPIE	github.com/gzquse/QPIE_datacircuit	PennyLane Dockerfile, PyTorch 24.06 Shifter container, inference scripts
DEAL	github.com/gzquse/DEAL_QUBO	distributed training, D-Wave Leap baseline, Qiskit GPU kernel, IBM job tracking
VQT	github.com/gzquse/quantum-transformer · gzquse/paper_AAAI2025-quantum · gzquse/vectorized_encoding	model code, figure reproduction, Mathematica derivations
ShardQ · QAWA · NNQA	github.com/gzquse/ShardQ_v2 · gzquse/qawa_data_circ · gzquse/nnqa	SparseCut and MPS knitting (MIT); QAWA_v2.py; NNQA toolkit and cloud jobs
RubriQ	github.com/gzquse/rubriq-artifact · Zenodo 10.5281/zenodo.21284556	GRPO fine-tuning, 21 evaluator tests; archived artifact, CC-BY-4.0
Nonlinear waves	Zenodo 10.5281/zenodo.21245905 (SplitStepQUDE)	code and data for the split-step solver, CC-BY-4.0 / Apache-2.0
QEC audit	anonymous.4open.science/r/usernix_qec_hse-32FE · gzquse/ExploreQEC_mar	containerized, seeded, deterministic audit; Stim/Qiskit/Guppy toolboxes
QPC SDK	Zenodo 10.5281/zenodo.21895102	v1.0.0, Linux x86_64, 1.1 MB, CC-BY-4.0